국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

보안 과제( ), 일반 과제(●) / 공개(●), 비공개( ), 발간등록번호( )

정보보호글로벌선도 기술개발 사업 최종 보고서

과제번호 2020-0-00666

개별분리 아키텍쳐 기반 5G 데이터 중심 클라우드 컴퓨팅 플랫폼 보안 연구 최종보고서

2023.05.30.

주관연구개발기관 / 한국과학기술원
공동연구개발기관 / 성균관대학교
연구개발기관 외 기관 / CISPA

과학기술정보통신부
정보통신기획평가원

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최종보고서

보안등급: 일반[●], 보안[ ]

중앙행정기관명	과학기술정보통신부	사업명	사업명	정보보호글로벌선도 기술개발 사업
전문기관명(해당 시 작성)	정보통신기획평가원		내역사업명 (해당 시 작성)	정보보호 국제공동연구
공고번호	제2019-0654호	총괄연구개발 식별번호 (해당 시 작성)		
		연구개발과제번호		2020-0-00666

기술분류	국가과학기술 표준분류	공통보안기술 (EE0301)	50 %	Cloud Computing / Grid Computing (EE0110)	30 %	인공지능 (EE0108)	20 %
	부처기술분류 (해당 시 작성)	시스템보안	50 %	클라우드보안	30 %	기타 ICT 보안	20 %

총괄연구개발명 (해당 시 작성)	국문				
	영문				
연구개발과제명	국문	개별분리 아키텍처 기반 5G 데이터 중심 클라우드 컴퓨팅 플랫폼 보안 연구			
	영문	Research on security of 5G-data-intensive computing platforms based on disaggregated architecture			
주관연구개발기관	기관명	한국과학기술원	사업자등록번호	314-82-01980	
	주소	대전 유성구 대학로 291 (34141)	법인등록번호		
연구책임자	성명	강병훈	직위	교수	
	연락처 직장전화	042-350-8322	휴대전화	*******	
	연락처 전자우편	brentkang@kaist.ac.kr	국가연구자번호	*******	

연구개발기간	전체	2020. 04. 01 - 2023. 03. 31 (3년)

연구개발비 (단위: 천원)		정부지원 연구개발비	기관부담 연구개발비		그 외 기관 등의 지원금				합계			연구개발비 외 지원금
					지방자치단체		기타( )					
		현금	현금	현물	현금	현물	현금	현물	현금	현물	합계	
총계		750,000		17,502					750,000	17,502	767,502	
1단계	1년차	250,000		17,502					250,000	17,502	267,502	
	2년차	250,000							250,000		250,000	
	3년차	250,000							250,000		250,000	

공동연구개발기관 등 (해당 시 작성)	기관명	책임자	직위	휴대전화	전자우편	비고 역할	비고 기관유형
공동연구개발기관	성균관대학교	이호준	조교수	*******	hojoon.lee@skku.edu		대학
연구개발기관 외 기관	CISPA	Sven Bugiel	정년 트랙 교수	*******	bugiel@cispa.de		연구소

연구개발담당자 실무담당자	성명	김대경	직위	박사과정	
	연락처 직장전화	042-350-8305	휴대전화	*******	
	연락처 전자우편	daegyeong.kim@kaist.ac.kr	국가연구자번호	*******	

이 최종보고서에 기재된 내용이 사실임을 확인하며, 만약 사실이 아닌 경우 관련 법령 및 규정에 따라 제재처분 등의 불이익도 감수하겠습니다.

2023 년 5 월 30 일

연구책임자: 강 병 훈 (인)

주관연구개발기관의 장: 이 광 형 (직인)

공동연구개발기관의 장: 최 재 붕 (직인)

과학기술정보통신부장관 귀하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최종보고서

보안등급: 일반[●], 보안[ ]

중앙행정기관명	과학기술정보통신부	사업명	사업명	정보보호글로벌선도 기술개발 사업
전문기관명(해당 시 작성)	정보통신기획평가원		내역사업명 (해당 시 작성)	정보보호 국제공동연구
공고번호	제2019-0654호	총괄연구개발 식별번호 (해당 시 작성)		
		연구개발과제번호		2020-0-00666

기술분류							
국가과학기술 표준분류	공통보안기술 (EE0301)	50 %	Cloud Computing / Grid Computing (EE0110)	30 %	인공지능 (EE0108)	20 %
부처기술분류 (해당 시 작성)	시스템보안	50 %	클라우드보안	30 %	기타 ICT 보안	20 %

총괄연구개발명 (해당 시 작성)	국문	
	영문	
연구개발과제명	국문	개별분리 아키텍처 기반 5G 데이터 중심 클라우드 컴퓨팅 플랫폼 보안 연구
	영문	Research on security of 5G-data-intensive computing platforms based on disaggregated architecture

주관연구개발기관	기관명	한국과학기술원	사업자등록번호	314-82-01980
	주소	대전 유성구 대학로 291 (34141)	법인등록번호	
연구책임자	성명	강병훈	직위	교수
	연락처 직장전화	042-350-8322	휴대전화	*******
	연락처 전자우편	brentkang@kaist.ac.kr	국가연구자번호	*******

연구개발기간	전체 2020. 04. 01 – 2023. 03. 31 (3년)

연구개발비 (단위: 천원)		정부지원 연구개발비	기관부담 연구개발비		그 외 기관 등의 지원금				합계			연구개발비 외 지원금
					지방자치단체		기타( )					
		현금	현금	현물	현금	현물	현금	현물	현금	현물	합계	
총계		750,000		17,502					750,000	17,502	767,502	
1단계	1년차	250,000		17,502					250,000	17,502	267,502	
	2년차	250,000							250,000		250,000	
	3년차	250,000							250,000		250,000	

공동연구개발기관 등 (해당 시 작성)	기관명	책임자	직위	휴대전화	전자우편	비고 역할	비고 기관유형
공동연구개발기관	성균관대학교	이호준	조교수	*******	hojoon.lee@skku.edu		대학
연구개발기관 외 기관	CISPA	Sven Bugiel	정년 트랙 교수	*******	bugiel@cispa.de		연구소

연구개발담당자 실무담당자	성명	김대경	직위	박사과정	
	연락처 직장전화	042-350-8305	휴대전화	*******	
	연락처 전자우편	daegyeong.kim@kaist.ac.kr	국가연구자번호	*******	

이 최종보고서에 기재된 내용이 사실임을 확인하며, 만약 사실이 아닌 경우 관련 법령 및 규정에 따라 제재처분 등의 불이익도 감수하겠습니다.

2023 년 5 월 30 일

연구책임자: 강 병 훈

주관연구개발기관의 장: 이 광 형 (직인)

공동연구개발기관의 장: 최 재 붕 (직인)

과학기술정보통신부장관 귀하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 요 약 문 >

<table>
<tr><td colspan="2">사업명</td><td colspan="3">정보보호글로벌선도기술개발 사업</td><td colspan="2">총괄연구개발 식별번호
(해당 시 작성)</td><td></td></tr>
<tr><td colspan="2">내역사업명
(해당 시 작성)</td><td colspan="3">정보보호 국제공동연구</td><td colspan="2">연구개발과제번호</td><td>2020-0-00666</td></tr>
<tr><td rowspan="2">기술분류</td><td>국가과학기술 표준분류</td><td>공통보안기술 (EE0301)</td><td>50 %</td><td>Cloud Computing / Grid Computing (EE0110)</td><td>30 %</td><td>인공지능 (EE0108)</td><td>20%</td></tr>
<tr><td>부처기술분류
(해당 시 작성)</td><td>시스템보안</td><td>50 %</td><td>클라우드보안</td><td>30 %</td><td>기타 ICT 보안</td><td>20%</td></tr>
<tr><td colspan="2">총괄연구개발명
(해당 시 작성)</td><td colspan="6"></td></tr>
<tr><td colspan="2">연구개발과제명</td><td colspan="6">개별분리 아키텍처 기반 5G 데이터 중심 클라우드 컴퓨팅 플랫폼 보안 연구</td></tr>
<tr><td colspan="2">전체 연구개발기간</td><td colspan="6">2020. 04. 01 - 2023. 03. 31 (3년)</td></tr>
<tr><td colspan="2">총 연구개발비</td><td colspan="6">총 750,000 천원
(정부지원연구개발비: 750,000 천원, 기관부담연구개발비 : 천원,
지방자치단체: 천원, 그 외 지원금: 천원)</td></tr>
<tr><td colspan="2">연구개발단계</td><td colspan="3">기초[ ] 응용[○] 개발[ ]
기타(위 3가지에 해당되지 않는 경우)[ ]</td><td colspan="2">기술성숙도
(해당 시 기재)</td><td>착수시점 기준(2)
종료시점 목표(5)</td></tr>
<tr><td colspan="2">연구개발과제 유형
(해당 시 작성)</td><td colspan="6"></td></tr>
<tr><td colspan="2">연구개발과제 특성
(해당 시 작성)</td><td colspan="6"></td></tr>
<tr><td rowspan="6">연구개발
목표 및 내용</td><td colspan="2">최종 목표</td><td colspan="5">개별분리 아키텍쳐 클라우드를 위한 신뢰 실행기술 체계 개발
개별분리 시스템을 위한 Memory Access Control 모듈 개발
개별분리 시스템상의 Accelerator(GPU 등)를 위한 Secure Offloading 프레임워크 개발</td></tr>
<tr><td colspan="2">전체 내용</td><td colspan="5">1년 차에는 개별분리 시스템 시뮬레이터 개발 및 시스템 내 가속기 pool의 접근 제어 모델과 Capability을 시뮬레이터 상에 구현함. 2년 차에는 개별분리 시스템에 사용되는 안전한 프로토콜 설계 및 구현 및 메모리 접근 제어기 개발. 개별분리 시스템에 사용되는 가속 풀 컨트롤러를 개발하고 및 CPU TEE, 가속기 원격인증 및 키 교환 모델을 개발. 3년 차에는 최종 Prototype을 평가 지표에 따라 측정 및 평가한 후 구현하지 못한 다른 보안요소 구성 방법을 제안.</td></tr>
<tr><td rowspan="2">1단계
(해당 시 작성)</td><td>목표</td><td colspan="5"></td></tr>
<tr><td>내용</td><td colspan="5"></td></tr>
<tr><td rowspan="2">n단계
(해당 시 작성)</td><td>목표</td><td colspan="5"></td></tr>
<tr><td>내용</td><td colspan="5"></td></tr>
</table>

연구개발성과	본과제에서는 개발한 보안 시스템에 대해서 정량적 성과목표로 성능 저하, 메모리 접근 지연 시간 감소, 인터커넥션 대역폭 감소, 그리고 비인가 접근 차단율을 제시하고자 함. 먼저, 성능 저하는 벤치마크 소프트웨어들을 제안한 보안 시스템 없이 실행했을 때 소요되는 시간과 본 시스템을 적용한 뒤 실행했을 때 소요되는 시간 비율로 측정함. 그리고 메모리 접근 지연 시간 감소에 대해 특정 크기의 메모리 영역에 대해 데이터를 가져오는 시간 및 저장하는 시간을 보안 시스템 없이 실행했을 때와 실행했을 때 소요되는 시간 비율로 측정함. 또한, 인터커넥션 대역폭 감소 측면에 대해서는 동일한 메모리 중심의 어플리케이션을 실행했을 때, 인터커넥션 대역폭을 보안 시스템이 없을 때 와 있을 때에 각각 측정하여 얻은 값의 비율로 측정함. 마지막으로, 비인가 접근 차단율을 하드웨어 구성 요소 (CPU 및 GPU)에서 인가되지 않은 영역의 메인 메모리에 접근하는 악의적인 어플리케이션을 실행하여 다양한 방법의 비인가 접근들에 대해 보안 시스템이 차단에 성공하는 비율을 측정함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

연구개발성과 활용계획 및 기대 효과	대규모의 서버가 밀집된 기업(SKT, KT 등 클라우드 기업, 인터넷 포탈 기업)뿐만 아니라 정부 부처 데이터 센터 등 앞으로 서버를 활용하는 각계에서 개별분리 아키텍처 관련 기술의 수요가 있을 것으로 예상됨. 또한, 개별분리 아키텍처는 컴퓨터 architecture의 근간을 흔드는 새로운 패러다임이며, 수없이 응용될 수 있는 원천 기술임. 이러한 기술을 먼저 확보한다면 국내 업계가 필요로 하는 기술을 해외에서의 수입 없이 원활한 기술 이전이 가능할 것으로 기대됨. 최종적으로 기존의 서버 관련 기업 또는 신생기업에서 기술의 응용을 통해 사업 아이템을 구상할 수 있으며, 이러한 기술을 서버 시스템에 적용함으로써 타사와의 차별성을 둘 수 있음. 또한, 개별분리 아키텍처를 적용한 기업은 각 컴포넌트(CPU, 메모리, GPU 등)를 큰 어려움 없이 추가/제거할 수 있어 유지/보수 측면에서 생산성 향상 및 비용 절감 등 매우 큰 이점이 있음
연구개발성과의 비공개여부 및 사유	공개

연구개발성과의 등록·기탁 건수	논문	특허	보고서 원문	연구 시설·장비	기술 요약 정보	소프트웨어	표준	생명자원		화합물	신품종	
								생명 정보	생물 자원		정보	실물
	5	5				1						

연구시설·장비 종합정보시스템 등록 현황	구입 기관	연구시설·장비명	규격 (모델명)	수량	구입 연월일	구입가격 (천원)	구입처 (전화)	비고 (설치장소)	ZEUS 등록번호

국문핵심어 (5개 이내)	개별분리 시스템	접근제어 모델			
영문핵심어 (5개 이내)	Disaggregated System	Access Control Model			

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<목 차>

1. 연구개발과제의 개요 ····· 1

2. 연구개발과제의 수행 과정 및 수행 내용 ····· 5

3. 연구개발과제의 수행 결과 및 목표 달성 정도 ····· 51

4. 목표 미달 또는 미흡한 사항에 대한 원인분석 ····· 60

5. 연구개발성과의 관련 분야에 대한 기여 정도 ····· 63

6. 연구개발성과의 관리 및 활용 계획 ····· 64

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획 ····· 67

8. 연구개발비 사용실적 ····· 68

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1. 연구개발과제의 개요

1-1. 배경

- **5G 시대와 클라우드 컴퓨팅**

  5G 시대의 도래로 인해 대역폭의 증가뿐만 아니라, 그 초연결성으로 인해 IoT 등의 데이터를 생산하는 주체의 증가를 불러오고 있다. 또한, AI 기술의 발전과 그 경쟁적인 도입으로 인하여, 기존의 서비스 및 인프라에 많은 데이터를 생산하여 수집하도록 하는 업계의 경향도 데이터양의 증가의 한 원인이다. 데이터의 양과 함께 클라우드에서의 데이터의 연산 또한 폭발적으로 증가하고 있다. 이러한 흐름은 현재 IT의 큰 요소로 잡은 클라우드의 변화를 요구하고 있다.

- **데이터-중심 클라우드에서의 민감한 데이터 격리 연산 필요성**

  AI 기술의 활용이 늘어남에 따라 데이터에 포함된 민감한 개인정보보호에 대해 인식이 갈수록 커지고 있다. 또한, 기업의 입장에서도 오랜 시간 데이터의 축적과 연구를 통해 도출된 인공지능 모델은 회사의 중요 지적재산으로 인식되고 있다. 그럼에도 불구하고, 확장성과 비용 문제로 최근 대부분 회사와 기관들은 아마존 AWS 등의 클라우드 서비스나 자체 구축 클라우드를 이용하여 AI 연산을 하는 경우가 대부분이다. 클라우드는 기본적으로 다수의 사용자, 그리고 다수의 작업이 이루어지기 때문에 철저한 보안관리가 필수적이며, 민감한 외부 클라우드 서비스를 사용은 편리성에 대한 대가로 보안 측면에 있어서는 클라우드 서비스 제공자를 전적으로 신뢰해야 하는 문제점이 있다. 최근 이러한 클라우드에서의 안전한 연산에 대한 요구가 높아져 감에 따라, 클라우드 서비스 제공자들도 보안성을 서비스의 큰 경쟁력 중 하나로 인식하기 시작했다. 마이크로소프트 애저 기밀 컴퓨팅 (Azure Confidential Computing) 등은 인텔 SGX 등의 하드웨어 신뢰 실행환경 기술을 이용하여, 클라우드 내에서 해당 사용자를 제외한 누구도 연산 내용을 엿보지 못하도록 하는 서비스를 제공하고 있다.

- **고확장성(scalable) 클라우드의 필요성: 개별분리 클라우드**

  기존 클라우드의 아키텍처는 CPU, GPU, 메모리와 같은 컴포넌트들을 포함한 머신 단위로 구성되고 있다. 따라서, 클라우드에게 요구되는 연산 능력과 저장 공간을 충족하기 위해 각각의 컴포넌트들을 유연하게 가감하거나 알맞게 시스템을 증축하기에는 어려움이 있다. 이는 시스템 자원의 완전 활용에 대한 어려움을 뜻하고 결국은 클라우드 센터의 운영 효율의 감소를 초래한다. 일례로 구글이 이전 공개한 자료에 따르면 구글 데이터 센터의 한 클러스터가 약 50%에 육박하는 메모리를 활용하지 못한 채 운영되고 있다. 게다가 AI 기술활용의 확산에 따라 클라우드 내 GPU와 TPU(Tensor Processing Unit) 같은 가속기의 필요 연산량이 다른 컴포넌트들의 필요성에 비해 더욱 증가하는 추세이므로, 이러한 클라우드 내 자원의 낭비가 더욱 심각해질 것으로 예상된다. 또한, 기존의 분산처리 아키텍처에서는 각각 머신 (Machine) 간에 데이터를 분산하여 처리한 후 결과를 수집 및 병합하는 과정에서 하드웨어 능력의 상당 부분이 낭비된다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최근 클라우드 구성 아키텍처 추세인 개별분리는, 하나의 머신에 밀접하게 연결된 CPU, 가속기(Accelerator, 예: GPU) 등의 구성 요소들을 해체하여, 각 풀(Pool)로 묶어 배치하고 이를 초고속 패브릭 인터커넥트(Fabric Interconnect)로 연결하는 기술이다. 이러한 특성을 통해 CPU 중심 아키텍처에서 데이터를 각 컴포넌트 간 옮기는데 생기는 오버헤드를 최소화하고, 또한 성능 요구에 따른 확장성과 비용을 개선할 수 있다(그림 1 참고). 현재 Intel, Facebook, HP 등의 업체들이 각각 자체적인 아키텍처를 개발 중이며, 학계 또한 현재도 활발히 연구가 진행 중이다.

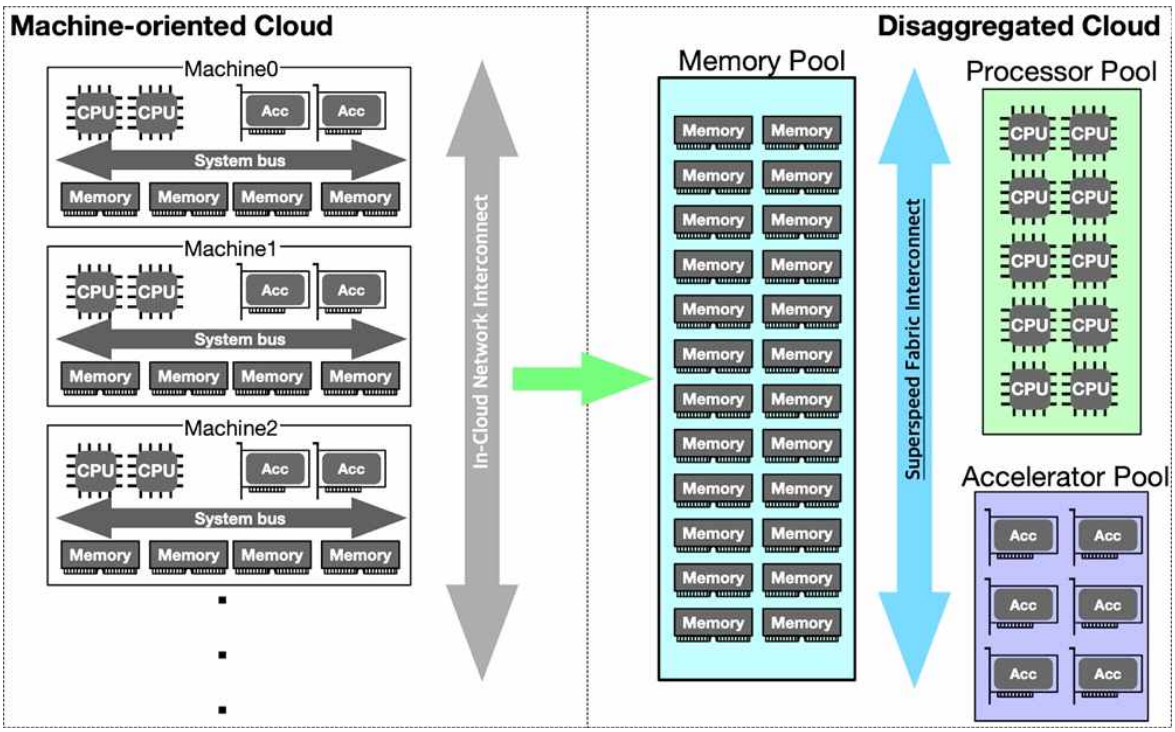


그림 2 개별분리 클라우드 개념도

1-2. 차세대 데이터-중심 클라우드 보안기술 제안

개별분리 아키텍처가 최근 많은 관심을 받는 것에 비해 보안 측면의 고려가 현재까지 매우 부족한 상황이다. 예를 들어, 현재 아키텍처에서는 악의적인 CPU가 다른 CPU에 할당된 메모리에 접근할 때 해당 메모리의 무결성을 보장할 수 없다. 또한, GPU의 메모리 접근 그리고 CPU의 GPU 접근 등과 같은, 그 외의 구성 요소 간 통신의 경우에도 이와 흡사한 보안 이슈가 발생한다. 그러므로 개별분리 아키텍처를 위한 새로운 보안 대책이 절실한 형편이다. 한편으로는 이러한 새로운 아키텍처의 특성이 오히려 기존 클라우드에서 풀 수 없었던 보안 문제들을 해결하는 데 도움이 될 것으로 예측된다. 기존의 클라우드 아키텍처에서는 CPU, GPU 등의 구성 요소들이 매우 밀접하게 결합하여 이들의 상호작용 방식을 바꾸기가 어려웠다. 하지만 개별분리 시스템은 이러한 구성 요소들이 더 이상 머신 내의 시스템 버스에 속해있지 않기 때문에, 중앙통제적인 접근 통제 및 컴포넌트 간의 안전한 채널 구성 지원을 구현할 수 있을 것으로 예상한다. 그러므로 본 연구에서는 차세대 개별분리 클라우드 시스템 자원들의 보안을 중앙에서 관리할 수 있는 중앙신뢰관리유닛(Central Trust Management Unit, CTMU)의 연구를 제안한다. 본 연구팀이 설계한

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

세부 구성 요소는 다음과 같다.

- **안전한 프로토콜**
  개별분리 아키텍처에서 사용되는 프로토콜은 다양한 공격시나리오에 대비하여 설계하여야 한다. 대표적인 예로 메시지 패킷 변조 및 위조 공격이 있으며, 그 외에도 내/외부에서 프로토콜의 취약점을 이용한 공격이 발생할 수 있다.

- **가속기 접근제어 모델**
  가속기들이 하나의 군을 이루어 다른 풀에서 가속기 풀에 속해있는 모든 가속기에 접근할 수 있다. 가속기도 자체 메모리를 가지고 있으므로, 악의적인 공격으로부터 메모리를 보호할 필요가 있다. 예를 들어, 개별분리 아키텍처에서는 GPU를 다른 사람과 공유하는 상황이 발생하므로 해당 GPU 메모리에 남아 있는 정보들을 도용하는 공격을 할 수 있다.

- **메모리 접근제어 모델**
  개별분리 시스템에서 다른 풀과 메모리 풀 간의 메모리 접근제어를 함으로써 악의적인 사용자가 다른 사용자의 데이터에 접근하는 것을 차단할 수 있다.

- **안전한 가속기 오프로딩**
  가속기와 CPU에서 실행될 코드와 데이터를 처리하고, 서로 데이터를 주고받으면 데이터의 무결성을 보장하기 위해 암호화된 채널을 통해 안전하게 오프로딩하는 프로토콜이 필요하다.

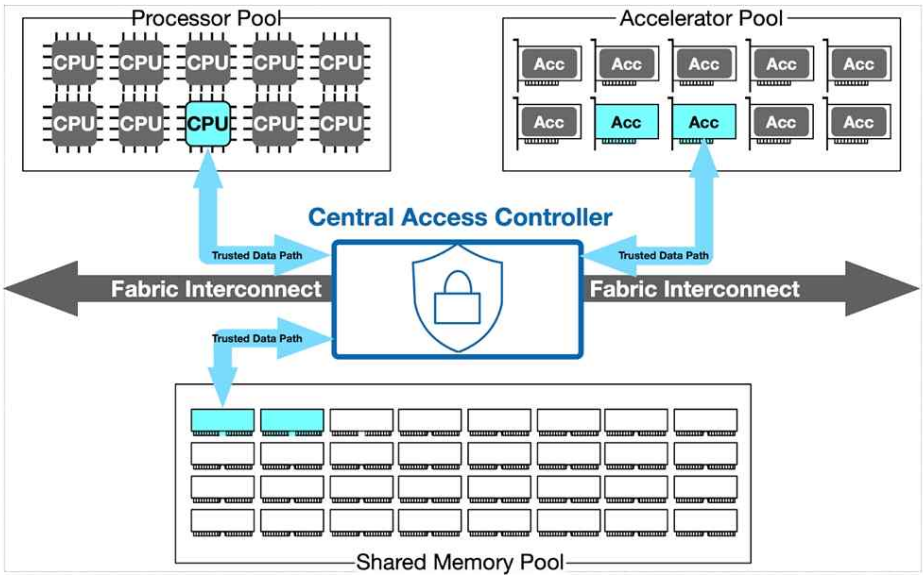


그림 3 본 과제에서 연구 및 개발한한 시스템의 기본개념도

1-3. 기술의 용도 및 적용 분야

- **개별분리 클라우드 컴퓨팅 기술의 도입 촉진**
  개별분리 아키텍처는 데이터-중심 클라우드 환경 구성에 있어 확장성과 성능적

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

이점이 분명하다. 아키텍처의 가장 큰 숙제 중 하나인 보안 문제를 해결함으로써 클라우드의 새로운 방향에 기여하고자 한다.

- **클라우드에서의 안전한 데이터 연산**
  최근 많은 요구가 있는 클라우드에서의 안전한 GPU 이용 인공지능 학습 및 추론 기능을 클라우드 서비스 제공자나 다른 사용자가 열람하지 못하도록 안전하게 격리하여 수행할 수 있다.

1-4. 경제/산업적 중요성

- **차세대 클라우드 핵심기술 확보**
  차세대 클라우드 구성 아키텍처인 개별분리 아키텍처의 보안 연구를 선행하여, 관련 특허 및 기술력 확보를 할 수 있을 것으로 기대된다.

- **개인정보보호 지원 AI 서비스 구축**
  클라우드와 AI를 결합한 서비스의 상당 부분은 개인정보보호가 가장 큰 이슈 중의 하나이다. 본 연구에서 제안하는 기술은 클라우드의 성능은 이용하되, 개인정보는 클라우드 관리자 또한 열람하지 못하도록 해, AI 서비스의 보안성을 강화할 것으로 기대된다.

- **차세대 클라우드 표준화**
  현재 데이터 중심 클라우드 보안기술은 표준화가 되지 않은 상태로써, 이를 논문/특허의 형태로 제시하여 차세대 클라우드 보안 관련 표준화에 기여할 것으로 기대된다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구개발과제의 수행 과정 및 수행 내용

2-1. 1년차(2020년)의 연구 범위

본 과제의 1년차 연구 범위와 관련 기관은 다음과 같다.

- **KAIST와 CISPA는 서로 협력하여 개별분리 시스템의 풀 시스템(full-system) 시뮬레이터를 연구 및 개발한다.**
  개별분리 시스템, 그리고 가속기 중심의 클라우드 연산이라는 새로운 아키텍처 패러다임에 관한 보안 연구를 위해서는 하드웨어/소프트웨어 공동설계가 필요하다. 이를 위한 초석으로써 시스템 전체를 각각 사이클마다(Cycle-accruate) 시뮬레이터 하는 풀 시스템 시뮬레이터의 개발이 반드시 선행되어야 한다. 구체적으로는, 기존의 시스템 메인 버스를 통한 CPU와 GPU 간의 통신이 개별분리되어 중앙 공유 버스를 이루어지는 새로운 모델에 대한 시뮬레이션의 구현이 될 수 있다. 1차연도에 이러한 새로운 모델을 풀 시스템 시뮬레이터를 통해 구현하여 2년차, 3년차 연구의 토대가 되도록 할 필요성이 있다.

- **성균관대학교와 CISPA는 서로 협력하여 개별분리 시스템의 가속기 모델을 연구 및 개발한다.**
  최근 데이터 센터 및 클라우드에서의 연산량의 많은 부분은 GPU나 TPU 등의 가속기에 오프로드되고 있다. 기존의 시스템에서는 가속기들이 시스템의 PCI 버스 등에 장착되어 메인 시스템과 밀접하게 결합이 되어 있지만, 개별분리 시스템 모델에서는 메인 시스템과 분리되어, 동종 혹은 이종(heterogeneous)의 가속기들과 하나의 군을 이루어 중앙 공유 버스(네트워크, 혹은 초고속 Fabric)에 연결된다. 본 연구에서는 개별분리 시스템의 다양한 특징을 고려하여 가속기 풀(pool)의 접근제어 모델을 시뮬레이터 상에 구현한다. 본 연구에서 성균관대학교는 CISPA와의 협력을 통하여 개별분리 메모리 접근 관리 기술에 대한 자문을 얻어 KAIST 측이 개발한 개별분리 시스템 시뮬레이터 상에서 동작하는 가속기 접근제어 모델을 구현한다.

2-1-1. 설계 개요

본 과제의 개별분리 아키텍처 시뮬레이터는 개별분리 시스템의 보안기술을 구현하고, 평가하는 데 목적이 있다. 따라서 개별분리 아키텍처 시뮬레이터의 설계는 시뮬레이터 자체의 성능보다는 안정적이고 정확하게 패킷을 전달하는 인터커넥트(interconnect) 본연의 기능을 수행할 수 있도록 하고, 2년차 연구 내용인 보안 기능을 적용하여 실행시키는 것에 주안점을 두었다.

1) 설계 가정 사항

- **캐시 일관성 도메인(Cache Coherence Domain)**
  현재의 인터커넥트 기술로는 캐시 일관성(cache coherency)을 위한 통신량을

감당하기에는 어려움이 있어 보이므로, 캐시 일관성을 지원하는 개별분리 인터커넥트는 고려하지 않는다. 그렇기에 여러 CPU 노드들을 통한 논리적 시스템 구성 및 원격 메모리(remote memory)의 공유 또한 고려하지 않는다.

- **소용량 로컬 메모리(Local memory)의 사용**

  메모리가 CPU에 종속적으로 배치되는 기존의 컴퓨터 아키텍처와 달리, 개별분리 아키텍처에서는 적은 용량의 로컬 메모리를 제외한 대부분 메모리는 원격에 위치한다. 적은 용량의 로컬 메모리는 현재 네트워크 기술에 따른 지연 등을 고려할 때, 기존 시스템과 유사한 수준의 성능을 보장하기 위해서 필수적이다.

- **영구적인 저장 장치(Persistent Storage)**

  개별분리 시스템의 메모리 구성 시 전통적인 DRAM뿐만 아니라 gem5 시뮬레이터에서 지원하는 NVMe(Non-Volatile Memory Express) 메모리를 같이 사용하였다. 개별분리 시스템 환경은 영구 메모리 (Persistent memory)를 사용하여 메모리를 구성하기 용이하며, NVMe은 비휘발성으로 지속해서 전원공급을 할 필요가 없으므로 메모리의 무결성을 보장하기 쉽다. 이러한 구성 방법은 개별분리 시스템의 주요 사용 사례 중 하나인 데이터 센터와 같은 환경에서 전력 소모량을 감소시킬 수 있다.

- **인터커넥트**

  노드 간의 통신을 담당하는 인터커넥트의 성능은 일정 기준 이상을 만족하여야 한다. 전통적인 아키텍처는 CPU와 메모리 간, CPU와 가속기 간 온 칩(on-chip) 또는 온 보드(on-board) 통신이나, 개별분리 시스템에서는 오프칩(off-chip) 또는 오프보드(off-board) 통신, 심지어 원격 통신이 될 수 있는 환경이기 때문이다. 본 과제에서는 인터커넥트는 개별분리 시스템에서 충분한 속도를 가지고 있음을 가정하고 있으며, 실제 개발된 솔루션으로는 InfiniBand 또는 100Gbps Ethernet, Intel Omni-Path 등이 사용될 수 있다.

- **가속기(Accelerator)**

  최근 클라우드에서의 가속기의 역할이 중요해짐에 따라, 전통적인 GPU부터 자체설계 NPU까지 가속기의 종류가 다양해지고 있다. 본 과제에서는 개별분리 아키텍처나 신뢰 실행을 고려하지 않고 설계된 PCI 기반의 범용성 가속기 (GPU)를 상정하고 개발을 진행하였다. 이러한 설계를 통하여, 기존의 가속기들을 본 과제에서 개발하는 개별분리 아키텍처 상에서 기존과 같은 구성으로 사용할 수 있도록 한다. 단, 현재 gem5 시뮬레이터에서는 GPU 시뮬레이션 지원이 미흡한 관계로 일반적인 PCI 장비를 시뮬레이터 상에서 추가하고 구현/실험하였지만, 실제 GPU들도 CPU 및 개별분리 아키텍처와의 연동 측면에서는 일반적인 PCI와 대동소이하다. 차년도에서는 현시점에도 개발이 진행되고 있는 gem5기반 GPU 시뮬레이션을 도입하여 GPU 연산 속도에 대한 성능 측정도 진행하고자 한다.

2) 전체 시스템 개요

컴퓨팅 노드는 수정되지 않은 프로세서 칩과 칩 무수정 및 인터커넥트와 연결을 지원하기 위한 인터커넥트 로직으로 구성된다. 프로세서는 시중에서 구할 수 있는 상용 프로세서를 탑재할 수 있도록 프로세서의 수정을 지양한다. 현재 프로세서는 전통적인 형태의 아키텍처만을 지원하고 개별분리 아키텍처를 고려하지 않게 설계되어 있어 프로세서의 수정을 피하기 위해서는 별도의 기능이 필요하다. 이에 프로세서와 인터커넥트를 연결하고, 그 인터페이스를 담당할 인터커넥트 로직이 필요하다.

인터커넥트는 컴퓨팅 노트, 메모리 노드 등 노드 간의 통신을 위해 오프칩(off-chip) 라우팅을 수행한다. 또한 다양한 토폴로지 지원, 패킷 손실 없는 신뢰 가능한 통신, 가상 채널 할당, 흐름 제어 등의 기능을 통해 노드 간 원격 접근으로 인한 성능 감소를 최소화한다.

메모리 풀 관리자는 메모리를 효율적으로 관리하기 위한 구성 요소다. 메모리의 형태는 DRAM, NVMe 등 여러 형태가 될 수가 있으며 용량도 다양하다. 이를 효율적으로 관리하기 위해서는 중앙에서 관리하고 이러한 메모리 모듈로의 통신을 지원하는 중앙집중적인 메모리 풀 관리자가 필요하다.

그림 3은 본 과제에서 수행한 개별분리 시스템 구현의 전체적인 개요를 나타낸다.

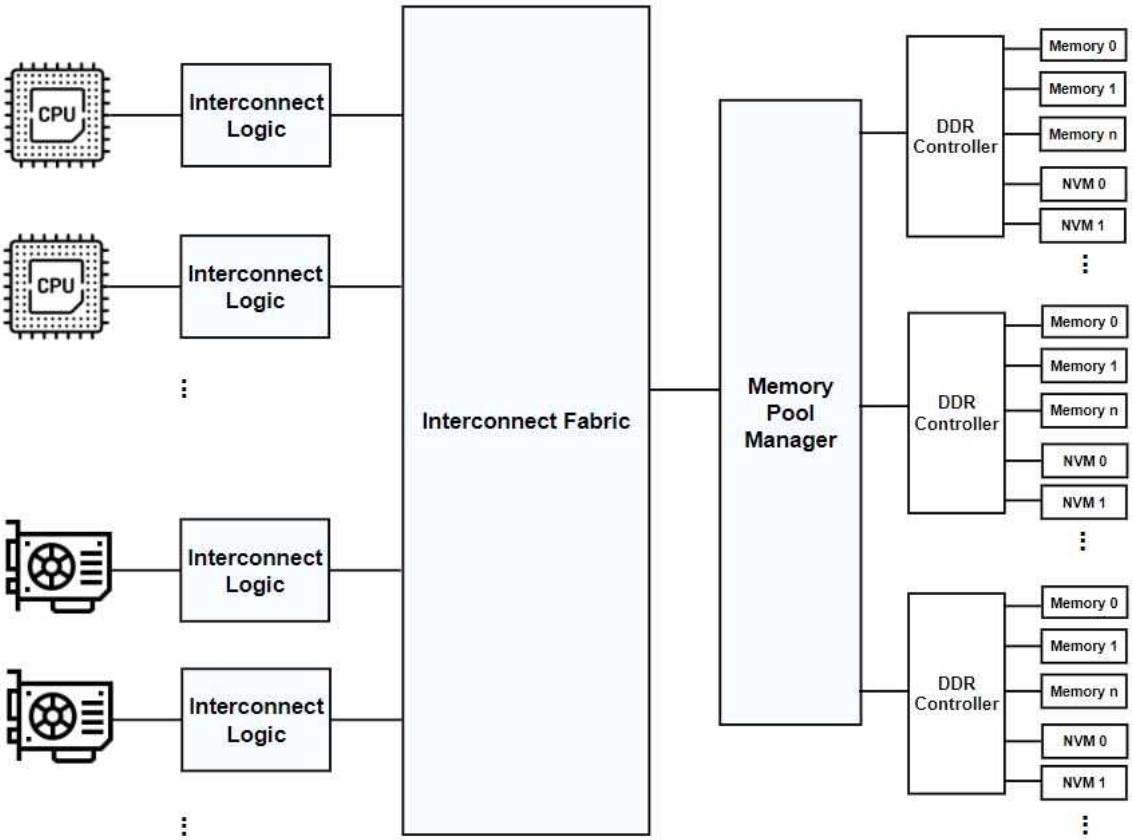


그림 4 전체 개별분리 시스템의 개요

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3) 컴퓨팅 노드(Computing node) 설계

(1) 기본 구성

컴퓨팅 노드를 구성하기 위해선, 그림 4와 같이, 일반적인 컴퓨팅을 위한 요소(e.g. CPU)와 이를 개별분리 인터커넥트와 이어주는 인터커넥트 로직 (Interconnect Logic)이 필요하다.

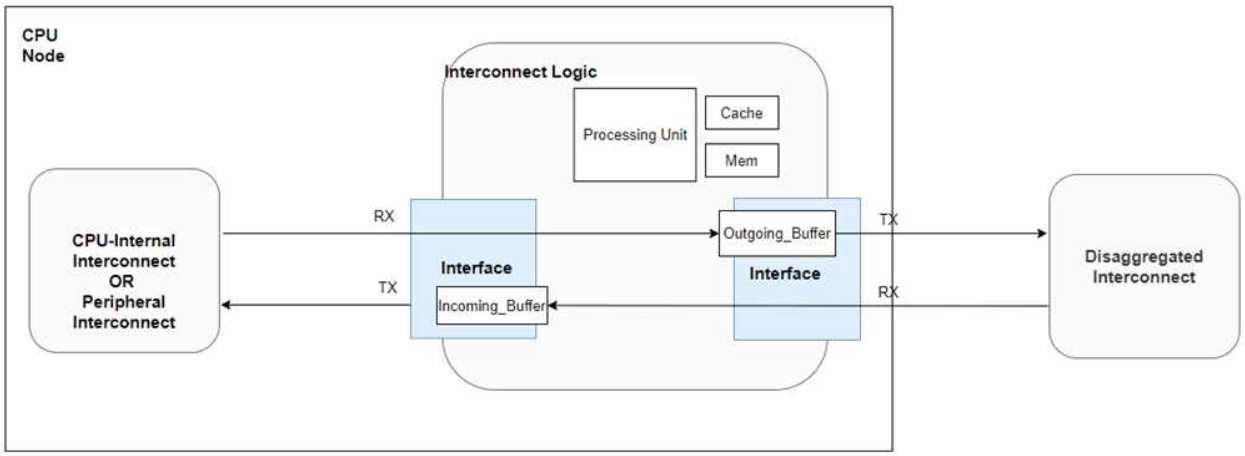


그림 5 컴퓨팅 노드(Computing Node)의 전체적인 구조

(2) 인터커넥트 로직

- **프로토콜 변환**

  가장 먼저 인터커넥트 로직과 CPU 등의 컴퓨팅 요소는 임의의 인터커넥트를 통해 연결돼야 한다. 하지만, 이러한 프로세서를 위한 인터커넥트는 개별분리 인터커넥트와는 다른 프로토콜을 사용하기 때문에, 인터커넥트 로직은 프로토콜 변환을 할 필요성이 있다.

- **주소 변환**

  컴퓨팅 요소 내부에서 사용하는 주소 공간(Physical Address Space)과 개별분리 인터커넥트에서 사용하는 주소 공간(Fabric Address Space)에는 차이가 있다. 그렇기에 트래픽들을 알맞은 목적지 노드 (예: 메모리 노드)에 보내기 위해서는, 노드 내부 주소 공간을 중심으로 이루어진 트래픽들을 개별분리 인터커넥트의 주소 공간에 맞게끔 변환하는 과정이 필요하다.

- **위치**

  그림 4에서 보여주듯, 인터커넥트 로직은 2가지의 위치에 존재할 수 있다. 첫 번째로, 기존 시스템에 주변장치를 연결하기 위해 사용하는, PCIe 인터커넥트에 부착이 될 수 있다. 즉, 이 방식을 사용한다면 인터커넥트 로직은 하나의 PCIe 카드의 형태가 된다. 하지만, 이러한 방식을 사용할 시, 컴퓨팅 요소들의 변형이 필요하지 않다는 장점이 있다. 하지만, 현존하는 CPU들은 PCIe로 전송되는 트래픽에 관한 캐시 일관성을 지원하지 않기 때문에 CPU에 존재하는 캐시들을 잘 활용하기 힘들다는 단점이 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다른 방법은, 컴퓨팅 요소의 변경을 통해 인터커넥트 로직을 내부에 직접 삽입할 수 있다. 이러한 경우 프로세서의 하드웨어적인 변경이 필요하지만, 기존 캐시 계층들을 전부 활용할 수 있다는 장점이 있다.

- **자원 할당 정보 제공**

  개별분리 시스템에서 원격 메모리 등의 자원을 사용하기 위해서는 자원에 대한 정보들을 소프트웨어가 알 수 있는 방법이 필요하다. 본 과제에서는 자원 할당 정보를 제공하기 위하여 인터커넥트 로직에서 자원에 대한 정보를 내부 메모리에 저장하고, 기존 주변장치 내부 메모리와 매핑하는 방법과 유사한 방법으로 노드 내부 주소 공간과 매핑 한 다음 MMIO를 통해 자원 할당 정보를 제공한다.

4) 인터커넥트 설계

인터커넥트는 구성 요소, 혹은 인터커넥트 로직과 연결되어 패킷을 목적지 구성 요소 혹은 인터커넥트 로직에 전달하는 역할을 한다. 이를 위하여 인터커넥트를 구성하는 라우터 간에 목적지에 이르는 경로에 따라 패킷을 전달(relay)한다. 인터커넥트는 패킷 등의 데이터를 손실 없이 목적지에 효율적으로 전송해야 한다. 인터커넥트의 토폴로지와 라우팅, 흐름 제어는 패킷을 효율적으로 전달하기 위한 방법을 나타내며, 구성 요소, 인터커넥트의 인터페이스는 신뢰 가능한 전송을 지원한다.

1년차의 인터커넥트 설계는 개별분리 시스템의 형태에 영향을 주는 인터커넥트 토폴로지, 구성 요소와 인터커넥트의 연결 및 신뢰 가능한 전송(reliable transmission)을 지원하는 인터페이스를 주요 요소로 두고 설계하였다. 인터커넥트의 라우팅과 흐름 제어 방법은 이 과제의 범위에서 벗어남으로, 설계의 범위에서 제외한다.

(1) 인터커넥트의 전체적인 동작 과정

개별분리 시스템의 각 구성 요소들은 인터커넥트에 연결된다. 그림 5와 같이 인터커넥트와 각 구성 요소들은 인터페이스와 링크로 연결되어, 패킷을 인터커넥트에 입력으로 주거나, 인터커넥트로부터 패킷을 받는다. 인터커넥트는 출발지 구성 요소로부터 받은 패킷을 목적지 구성 요소로 전송하기 위하여, 인터커넥트를 구성하는 하나 혹은 다수의 라우터로 사이의 적절한 경로를 계산한다. 패킷은 라우팅 테이블 혹은 계산된 경로에 따라 목적지 패킷에 전달된다. 따라서 각 구성 요소는 인터커넥트의 토폴로지를 정확히 알지 못하여도 라우터의 집합인 인터커넥트에 연결하는 것으로 목적지에 패킷을 보낼 수 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

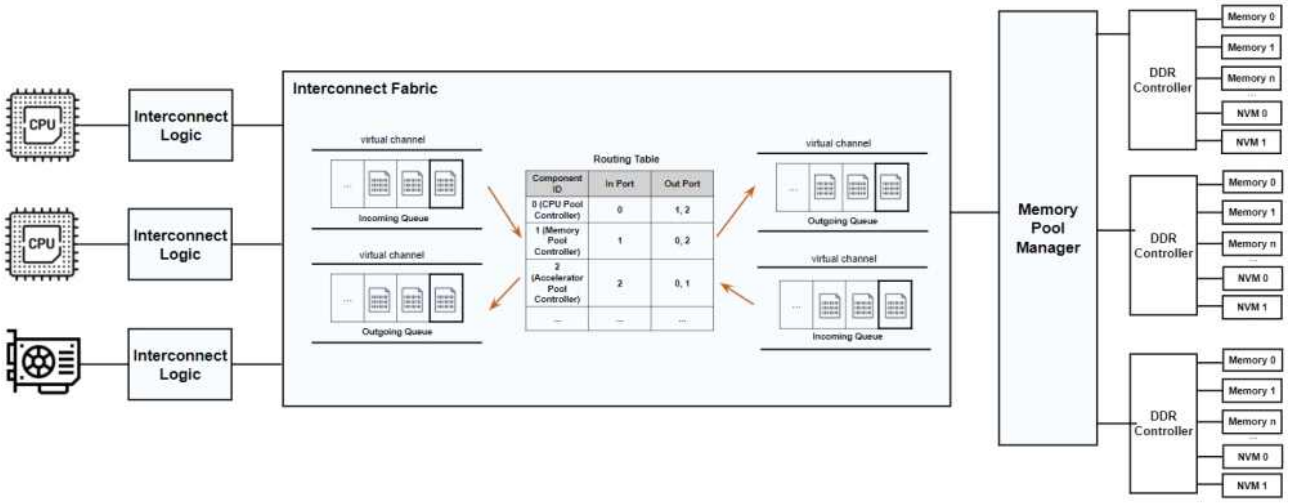


그림 6 인터커넥트의 구성

(2) 인터커넥트의 토폴로지

인터커넥트의 토폴로지는 2가지 형태로 설계하였다. 개별분리 시스템은 새롭게 개발 및 연구되고 있는 시스템 아키텍처로, 선제적으로 보안 위협에 대응할 수 있는 보안 기법을 연구하는 본 과제의 목표이다. 또한, 개별분리 시스템에 대한 효율적인 보안 기법은 개별분리 시스템의 구성 형태와 밀접한 관련이 있다. 1년차에서는 2년차에서 연구될 개별분리 시스템의 보안 기법의 여러 가지 가능성을 고려하고 수용할 수 있는 인터커넥트를 설계하기 위하여, 두 가지의 인터커넥트 토폴로지를 설계하였다. 앞서 제안서에서 제안하였던 풀 관리자의 개념을 구현하기 위한 토폴로지와 풀의 경계 없이 인터커넥트에 연결되는 토폴로지가 있다. 첫 번째 토폴로지는 각 구성 요소 풀을 관리하는 풀 관리자의 도입을 고려한 디자인으로, 같은 유형의 구성 요소들을 통합적으로 관리 혹은 보호할 수 있다. 두 번째 토폴로지는 풀 관리자를 포함하지 않는 인터커넥트 형태로, 구성 요소의 특성에 따라 관리되어야 할 기능 혹은 보호 기술이 없는 경우, 인터커넥트의 토폴로지 형태와 무관하게 각 구성 요소를 인터커넥트에 연결할 수 있다. 본 과제에서는 2년 차에 연구될 개별분리 시스템의 보안 기법은 풀 관리자 혹은 그 외의 모듈을 중심으로 하여 구성 요소를 보호하는 형태가 될 수 있다. 따라서 2년 차에서는 첫 번째 혹은 두 번째 인터커넥트 토폴로지 중 연구된 보안 기법에 적합한 토폴로지를 선택하여 사용할 예정이다.

- **풀 관리자를 포함한 인터커넥트**

  첫 번째 인터커넥트 토폴로지 형태는 팻트리(fat-tree)를 이용한 계층적인 라우터의 구성이다. 팻트리의 각각 단말 노드는 한 종류의 구성 요소(예: 메모리, CPU, GPU)를 모아 연결한다. 각각의 단말 라우터는 루트 라우터에 연결된다. 설계된 인터커넥트 토폴로지는 기본적으로 2단계로 구성된다. 따라서 단말 라우터와 연결된 루트 라우터는 각 단말 라우터에 연결된 구성 요소들의 트래픽이 집중되어 병목 현상이 일어날 가능성이 있다. 따라서 팻트리의 특성을 이용하여 각 단말 라우터와 루트 라우터를 연결하는 링크의 대역폭을 크게 설정하는 것으로 이러한 병목 현상을 줄일 수 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

팻트리를 이용한 첫 번째 인터커넥트 토폴로지는 풀 단위로 보안 기법 혹은 관리가 효율적일 때 사용될 수 있다. 예를 들어, 전체적인 시스템에 초기화가 필요할 경우, 구성 요소 별로 수행되어야 하는 공통적인 명령이 존재할 수 있다. 이런 경우, 구성 요소에 특성에 따라 초기화에 필요한 명령을 일괄적으로 실행할 수 있도록 설정할 수 있다. 풀 관리자로 사용되는 단말 라우터는 초기화 패킷을 받았을 때, 풀을 구성하는 각각의 구성 요소들에 일괄적으로 특성에 맞는 명령을 내릴 수 있다.

- **풀 관리자를 포함하지 않는 인터커넥트**

  두 번째 인터커넥트 토폴로지 형태는 구성 요소 풀 없이, 각각의 구성 요소가 인터커넥트에 연결된다. 이 토폴로지 형태는 유사한 구성 요소 간의 통합된 관리 요구사항이 없을 경우 사용될 수 있다. 이때 인터커넥트 토폴로지 형태는 단일한 라우터로 구성되거나, 그 외에 여러 가지 유형의 토폴로지가 될 수 있다.

  풀 관리자를 포함하지 않는 인터커넥트의 장점은 토폴로지에 제한이 없다는 점이다. 단말 라우터가 풀 관리자의 기능을 수행하지 않기 때문에, 인터커넥트의 토폴로지는 계층을 가질 필요성이 없어지며, 메시(Mesh), 크로스바(Crossbar, 그림6 참조), pt2pt(Point to point) 등의 다양한 토폴로지로 구성할 수 있다.

  풀 관리자를 포함한 인터커넥트와 포함하지 않는 인터커넥트 토폴로지의 차이점은 같은 종류의 구성 요소가 풀을 구성하여 단일한 라우터에 연결되어 있는지, 풀과 연결된 라우터가 풀을 위하여 특수한 기능을 포함하는 지로 볼 수 있다.

(3) 인터페이스(Interface)

개별분리 시스템에서 구성 요소와 인터커넥트, 정확하게는 인터커넥트를 구성하는 라우터에 연결하기 위해서는 인터페이스에 링크를 접속하여야 한다. 인터페이스는 구성 요소와 라우터에 존재하며, 패킷을 전송하거나 받는다. 각각의 인터페이스에는 구성 요소 혹은 라우터의 메시지를 알맞은 포맷의 패킷으로 변환한다. 개별분리 시스템의 인터페이스는 각각의 구성 요소에서 발생하는 메시지를 라우팅 가능한 패킷으로 변환시키고, 안정적인 전송을 위하여 패킷의 재전송, 오류를 확인하는 기능을 수행한다.

(4) 메시지 변환

인터페이스는 인터커넥트 로직 혹은 컴포넌트로의 인터페이스는 패킷을 라우팅을 위한 패킷 포맷으로 변환한다. 라우팅을 위한 패킷 포맷으로 변환하기 위하여, 라우팅을 위한 정보가 저장된 헤더를 씌워, 캡슐화(encapsulation)한다. 캡슐화된 패킷은 목적지 인터커넥트 로직 혹은 컴포넌트의 인터페이스에서 다시 원래의 구성 요소 패킷으로 변환된다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

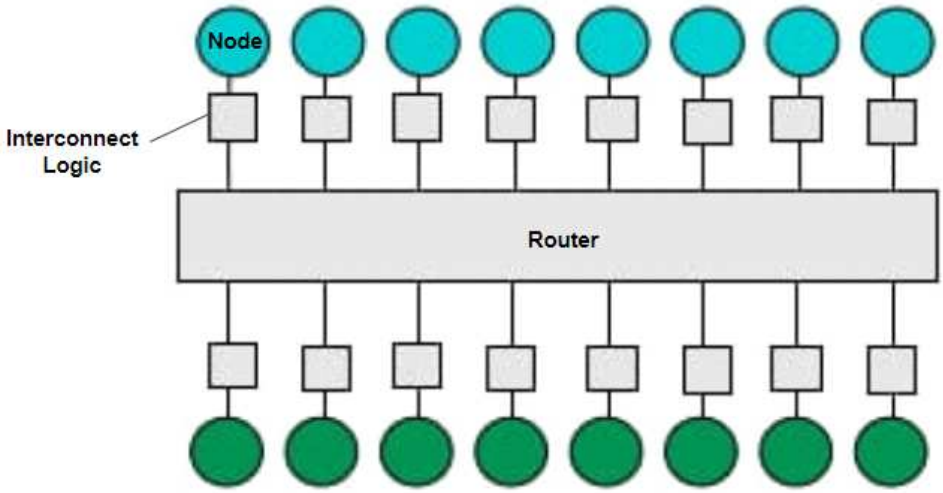


그림 7 풀 관리자를 포함하지 않는 인터커넥트의 토폴로지 예시(Crossbar)

(5) 신뢰 가능한 전송

개별분리 시스템은 수많은 CPU와 메모리, 그리고 가속기 등이 서로 메시지를 주고받으며 동작하는 거대한 시스템이다. 패킷이 손실되거나 손상된다면 시스템 전체가 불안정해질 수 있으므로 신뢰 가능한 통신은 매우 중요하다. 본 시스템에서 인터커넥트는 PCIe 통신 스택 중 신뢰 가능한 통신을 도입하여 노드 간 신뢰 가능한 통신을 지원한다. 신뢰 가능함은 아래 세 가지로 이루어진다. 첫 번째, ack/nak 기능을 통하여 노드 간 통신이 손실 없이 완전하게 이루어졌는지 상호 확인한다. 두 번째, sequence number를 통하여 통신이 의도한 순서대로 이루어지고 중복이 되지 않도록 한다. 세 번째, CRC/체크섬 확인을 통해 패킷 손상을 확인한다.

5) 메모리 노드 설계

(1) 메모리 풀 관리자(Memory Pool Manager)

메모리가 CPU에 종속적으로 배치되는 기존의 컴퓨터 아키텍처와 달리, 개별분리 아키텍처에서는 적은 용량의 로컬 메모리를 제외한 메모리 대부분은 원격에 위치한다. 이러한 대규모의 원격 메모리 집합을 효율적으로 관리하기 위해서 메모리 풀 관리자(Memory pool manager)의 개념을 도입할 수 있다. 메모리 풀 관리자는 각각의 메모리 미디어(DRAM, NVM(비휘발성 메모리, Non-Volatile Memory))의 통신을 제어하는 메모리 컨트롤러(Memory controller)들의 모임인 메모리 풀(Memory pool)을 총괄하는 시스템 구성 요소이다.

메모리 풀 관리자는 다음과 같은 기능을 가진다.

- 개별분리 인터커넥트를 통해 컴퓨팅 노드와 연결. 인터커넥트와 연결하기 위한 요소인 인터커넥트 로직을 가짐
- 자신의 포트를 통해 메모리 컨트롤러들과 연결
- 컴퓨팅 노드에서 수신된 패킷을 구성 요소 식별자(CID: Component IDentifier)와 같은 라우팅 정보를 참고하여 적절한 메모리 컨트롤러로 전달
- 메모리 컨트롤러에서 CPU, GPU 노드로 응답하는 패킷을 라우팅 정보를 참고하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

여 올바르게 전달

(2) 메모리 컨트롤러

메모리 컨트롤러는 메모리 풀 관리자를 통해 전달받은 패킷의 요청을 저장 미디어(DRAM 혹은 NVM)가 수행해야 할 저수준의 명령으로 변환하는 역할을 수행한다. 여러 개의 요청 처리에 걸리는 시간을 최소화하기 위해 컨트롤러 내 요청 및 응답 큐가 존재하여, (실제로는 미디어가 저속으로 동작할지라도) 컴퓨팅 노드에서 겪게 되는 지연을 감춘다. 구체적인 예시로는 요청 수신 시 실제 미디어가 요청을 서비스한 후 응답하지 않아도 되는 경우(메모리 쓰기 요청)가 있다. 이런 경우는 요청 큐에 요청을 저장해 두지만 바로 미디어에서 요청이 서비스되지는 않는다. 컴퓨팅 노드는 응답을 기다릴 필요가 없으므로 다른 작업을 바로 진행할 수 있으며, 메모리 컨트롤러는 추후 적절한 시기에 해당 요청을 처리할 수 있다. 또한, 메모리 컨트롤러는 자신의 큐가 처리할 수 있는 패킷의 개수를 초과하여 요청을 받는 경우를 방지하기 위해 흐름 제어 기능을 수행한다. 컴퓨팅 노드가 송신하여 메모리 풀 관리자를 거쳐 수신된 패킷의 오류 검출 및 처리 기능을 수행한다.

(3) 패킷 다중화

개별분리 아키텍처에서는 인터커넥트를 통해 여러 개의 CPU 및 GPU 노드가 메모리 풀에 읽기/쓰기 요청을 전달할 수 있다. 이러한 요청은 패킷(Packet)의 형태로 메모리 풀 관리자에 도착한다. 요청을 올바르게 수행하기 위해서는 패킷을 적절한 메모리 컨트롤러로 전달하는 과정이 필수적이다. 패킷의 헤더(Header) 부분에는 이러한 목적에 활용 가능한 정보가 포함되어 있다.

이 정보들은 컴퓨팅 노드의 인터커넥트 로직에 의해 패킷 헤더에 기록된다. 구체적으로, 인터커넥트 로직은 주소 변환 과정을 통해, 개별 컴퓨팅 노드에 한정된 물리 주소 공간을 인터커넥트 상의 패킷 라우팅에 사용할 수 있는 패브릭 주소 공간으로 변환한다. 패브릭 주소 공간에는 구성 요소 식별자 등의 정보가 포함된다. 주소 변환된 결과는 패킷 헤더에 포함된다. 패킷은 컴퓨팅 노드로부터 인터커넥트를 통해 메모리 풀 관리자로 전달되며, 메모리 풀 관리자는 패킷 헤더를 참조해 패킷을 적절한 메모리 컨트롤러로 전달한다.

반대로, 메모리 컨트롤러에서 컴퓨팅 노드로 요청에 대한 응답 패킷을 전송할 때도 마찬가지로 패킷 헤더에 적절한 컴퓨팅 노드를 가리키는 정보를 기록해야 한다. 이러한 과정은 메모리 풀 관리자의 인터커넥트 로직을 통해 수행된다.

(4) 다양한 저장 미디어 지원

개별분리 시스템에서는 전통적인 휘발성 저장 미디어인 DRAM뿐만 아니라, 비휘발성 메모리(NVM) 또한 사용할 수 있다. 비휘발성 메모리는 단순히 기존의 보조기억장치를 대체하는 용도로 사용 가능할 뿐만 아니라, 인-메모리 데이터베이스(In-Memory Database)와 같은 특수한 목적의 응용 프로그램의 성능을 향상하기 위해 쓰일 수도 있다.

메모리 컨트롤러는 기존 DRAM과 더불어 NVM도 제어할 수 있는 기능을 가진다.

여러 개의 미디어가 하나의 메모리 컨트롤러에 연결되어 있을 때, 각 미디어의 물리적 용량을 합쳐 컨트롤러당 하나의 병합된 주소 범위를 구성할 수 있다.
새로운 저장 미디어를 도입함에 따라 발생할 수 있는 다른 시스템 구성 요소의 소프트웨어적/하드웨어적 변경을 최소화하기 위해, 메모리 컨트롤러가 노출하는 인터페이스는 기존과 같이 유지된다.

6) 가속기(GPU 등) 설계

(1) 인터커넥트 로직

GPU 등의 가속기 노드는 CPU와 같은 컴퓨팅 노드로 분류되며, CPU 노드와 마찬가지로 인터커넥트 로직을 통하여 개별분리 인터커넥트 패브릭에 접속한다. 인터커넥트 로직을 통해 개별분리 시스템 지원이 내재되어 있지 않은 가속기와 패브릭 사이의 통신 규약을 전환(Translate) 하는 방식으로 동작한다.

(2) CPU → 가속기 연산 오프로딩

개별분리 시스템에서의 가속기(예: GPU)들은 CPU와 같은 컴퓨팅 노드이지만, CPU로부터 연산 작업을 오프로딩 받는 CPU 종속성을 가진다. 따라서 가속기 노드는 CPU 노드 및 메모리 풀에서 할당받은 메모리와 워크그룹(workgroup)을 형성하고 CPU가 주도하는 연산에서 일부를 전달받아 가속하는 역할을 수행한다.

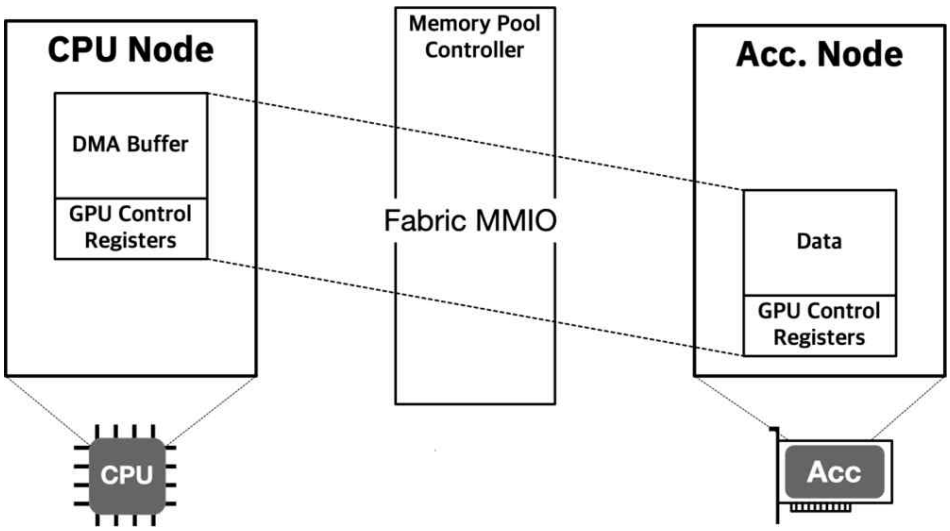


그림 8 Fabric MMIO를 통한 CPU 노드와 가속기 노드 간의 통신

(3) CPU 노드-가속기 노드 간 통신

개별분리 시스템에서도 기존 클라우드 아키텍처와 마찬가지로 CPU와 GPU는 메모리 매핑 기반 통신 (Memory-Mapped IO, MMIO)을 통하여 통신한다. 본 과제에서 개발하는 개별분리 시스템에서는 메모리 풀 관리자에 MMIO 구성을 통해서 CPU와 GPU 간의 통신 채널을 수립한다. 메모리 풀 관리자는 CPU와 가속기 노드 간의 통신에 사용할 패브릭 주소를 할당하고, 인터커넥트 로직에 주소 변환 정보를 탑재시

켜 CPU 내부 버스 주소, 패브릭 주소, GPU 내부 주소 간의 주소를 변환해 주는 역할을 하여 패브릭을 통하여 CPU와 GPU가 로컬 머신에서와 같이 통신할 수 있도록 설정해 준다. CPU 노드와 가속기 노드는 로컬 머신에 배치되어있을 때와 마찬가지로 작동하지만, 각각 노드에서 송출되는 패킷은 인터커넥트 로직과 풀 관리자에 의해 주소 전환 및 라우팅 과정이 이루어진다.

(4) CPU 노드 및 가속기 노드 간의 메모리 접근제어

CPU와 가속기 노드 서로 간의 메모리 접근제어는 인터커넥트 로직에 의해서 이루어진다. 인터커넥트 로직에서는 각 노드 간의 주소를 변환할 수 있는 주소 전환정보를 통해 다른 노드가 사용 중인 메모리를 확인하거나, 각 노드 간 접근해야 하는 주소를 확인하게 된다. 이를 통해 사용할 수 없는 메모리의 접근을 제한하여 메모리를 안전하게 보호할 수 있도록 한다.

2-1-2. 구현 개요

1) 개별분리 구조 구현

Gem5에는 기본적으로 프로그램이 한 머신에서 실행되는 것을 가정하도록 디자인이 되어 있다. 하지만, 본 연구를 위해서는 이러한 구조를 바꾸어서 CPU와 메모리를 다른 노드 분리하고 여러 노드가 같은 개별분리 인터커넥트를 사용하도록 해야 한다. 이를 위해서, 주요 요소들(예: CPU, Memory)을 각각 한 머신 (System 클래스)에 부착된 것처럼 구현하였다. 그리고, Gem5의 통곗값 등 중앙적 관리가 필요한 부분들이 있으므로 이를 위해 중앙 관리를 위한 DisaggrgeatedSystem이라는 새로운 클래스를 구현하여서 해결하였다.

2) 컴퓨팅 노드 구현

(1) 인터커넥트 로직

현재 인터커넥트 로직은 CPU 내부에 하드웨어 변경을 통해 포함된 형식으로 구현되어 있다. 하지만 간단한 설정 파일 변경으로 외부에도 위치할 수 있게 구현이 되어 있다. 인터커넥트 로직의 주요 역할은 주소 변환 기능이다. 이 기능은 기존의 페이지 테이블 워킹 코드를 활용하여 패킷의 주소 변환을 간단한 설정 파일 변경으로 조정할 수 있게 구현되어 있다. 또한, 인터커넥트 로직은 해당 CPU로부터 전송되는 패킷이 어떤 컴포넌트를 목적지로 하는지 알려주는 CID를 로직이 가진 맵핑 정보를 통해 패킷에 입력하도록 구현되어 있다. 마지막으로 기존 시뮬레이터에 없던 인터커넥트 로직의 구성 요소들에 대한 세부 지연시간을 적용하여 시뮬레이터를 활용한 실험에서 정확한 지연시간을 얻을 수 있도록 구현되어 있다.

(2) CPU

실존하는 대부분의 CPU가 3계층의 캐시(코어별 L1/L2 캐시 + 공유 L3 캐시)를 가지고 있지만, Gem5에서 기본적으로 제공하는 캐시 계층은 2계층 (코어별 L1 캐시 + 공유 L3 캐시)으로 한 계층이 부족하다. 그렇기에 추가적인 구현으로 캐시 1계층을 더하였다. 또한, Gem5에는 여러 CPU 관련 설정값이 존재한다 (e.g., CPU 클록,

캐시 크기). 이러한 설정값을 자세히 할수록 더 현실적인 시뮬레이션이 가능하게 된다. 따라서, 그러한 설정값에 대한 조사와 구현을 진행하고 있다.

3) 인터커넥트 구현

개별분리 시스템 시뮬레이터의 인터커넥트는 Gem5의 구성 요소 중 하나인 가넷(Garnet, 현재 최신버전 3.0 사용)을 수정 및 기능을 추가하여 구현하였다. 가넷은 Gem5 내부의 인터커넥트 네트워크 모델을 시뮬레이팅 한다. 가넷은 Gem5에서 서브 메모리 시스템의 자세한 구현을 나타내는 루비 시스템 모델의 토폴로지, 라우팅 기능을 이용하여 동작한다. 가넷은 프로그램 인자를 통하여 여러 가지 네트워크 설정을 손쉽게 조정할 수 있다. 인터커넥트 시뮬레이터를 구현하기 위하여 가넷의 코드를 수정 및 추가하였다.

(1) 개별분리 시스템 구성 요소의 패킷 구현

개별분리 시스템을 위한 패킷을 위하여 Gem5의 패킷 클래스를 수정하였다. 먼저 출발지와 목적지의 구성 요소 ID를 추가하였다. 설계한 개별분리 시스템은 각 구성 요소에 구성 요소 ID를 부여하고, 패킷에 목적지를 표기할 때 해당하는 구성 요소의 구성 요소 ID를 명시한다. 또한 목적지 구성 요소에서 응답을 보내기 위해서 출발지 구성 요소 ID를 알 필요가 있다. 따라서 출발지와 목적지의 구성 요소 ID를 Gem5의 패킷 자료구조에 추가하여 이를 해결했다. 두 번째는 메모리의 서브 CID 필드를 추가해, 일단 메모리 풀 관리자에 도착한 패킷은 더는 인터커넥트의 간섭 없이 자체적으로 패킷 헤더에 기록된 주소에 대응되는 메모리 컨트롤러로 패킷을 전달할 수 있도록 하였다. 메모리 서브 CID 필드는 인터커넥트가 메모리 풀 내부의 모든 구성 요소에 따른 라우팅 방법을 계산하지 않도록 하여 라우팅 복잡성을 해소한다. 메모리 서브 CID 필드를 통하여 인터커넥트는 단순히 최상위 요소인 메모리 풀 관리자에 패킷을 전달하는 것으로 개별 메모리 구성 요소에 패킷을 전달 할 수 있다. 세 번째로 패킷의 에러를 검사하기 위하여 CRC 필드를 추가하였다. CRC 변수는 안정적인 전송을 위해서 전송된 데이터의 손실 여부를 검사하는 검사할 수 있다. 시뮬레이터 환경에서는 패킷 전송 때에 데이터 손실에 따른 오류가 발생하지 않으므로 이 값은 항상 0으로 설정되고 CRC 값 계산과 검사가 생략된다. 마지막으로 순서 필드가 추가되었다. 한 동작을 위하여 여러 패킷이 필요할 경우, 패킷이 전송 혹은 생성된 순서대로 목적지 구성 요소에 도착하지 않더라도 순서 필드를 참조하여 데이터를 순서대로 조합할 수 있다. 또한 순서 필드를 이용하여 예측되는 순서의 패킷을 전송하도록 보장하는 것으로 안정적인 전송을 할 수 있다.

(2) 토폴로지

풀 관리자가 있는 토폴로지를 구현하기 위하여, 루트 라우터와 3개의 단말 노드, 즉 풀 관리자를 가지는 2-레벨로 구성된 팻트리 토폴로지를 구현하였다. 가넷은 기본적으로 여러 형태의 토폴로지를 지원하지만, 팻트리는 지원하지 않는다. 팻트리 토폴로지를 구현하기 위하여 가넷에 기능을 추가하는 방식을 사용하였다. 가넷은 파이썬 언어로 간단하게 토폴로지를 구성할 수 있도록 지원한다. 이러한 가넷의 기

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	패킷 내용

메모리 읽기 요청	typedef struct _PACKET_READ_REQUEST{
	packet_type _packet_type:4;
	CID src_cid:32;
	CID dst_cid:32;
	CID mem_sub_cid:32; // -1 if dst_cid != mem_pool manager
	uint32 seq_num:32;
	Addr phy_addr:64;
	uint32 crc:32;
	} PACKET_RED_REQUEST;

메모리 읽기 응답	typedef struct _PACKET_READ_RESPONSE{
	packet_type _packet_type; // request or response
	CID src_cid;
	CID dst_cid;
	CID mem_sub_cid; // -1 if dst_cid != mem_pool manager
	uint32 seq_num:32;
	Addr phy_addr;
	uint32 crc:32;
	uint32 data_length;
	unsigned char *data;
	} PACKET_RESPONSE;

메모리 쓰기 요청	typedef struct _PACKET_WRITE_REQUEST{
	packet_type _packet_type;
	CID src_cid:32;
	CID dst_cid:32;
	CID mem_sub_cid:32; // -1 if dst_cid != mem_pool manager
	uint32 seq_num:32;
	Addr phyAddr;
	uint32 data_length;
	unsigned char* data;
	} PACKET_WRITE_REQUEST;

메모리 쓰기 응답	typedef struct _PACKET_WRITE_RESPONSE{
	packet_type _packet_type; // request or response
	CID src_cid;
	CID dst_cid;
	CID mem_sub_cid; // -1 if dst_cid != mem_pool manager
	uint32 seq_num:32;
	uint8 ret:8; // 32 response codes
	uint32 crc:32;
	} PACKET_RESPONSE;

표 1 패킷 클래스의 추가된 코드

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

능을 이용하여 팻트리 토폴로지를 추가하였다. 그림 8은 풀 관리자를 포함한 인터커넥트를 설명한다. 토폴로지는 루트 라우터와 다수의 리프 라우터로 구성된 2-레벨의 팻트리로 구성된다. 리프 노드의 라우터는 클러스터 형식으로 이뤄진 구성 요소의 풀과 연결된다.

풀 관리자가 없는 토폴로지를 사용하기 위해서는 단순히 가넷이 지원하는 토폴로지 중 하나를 선택하거나, 새로운 토폴로지를 생성하여 사용하도록 설정한다.

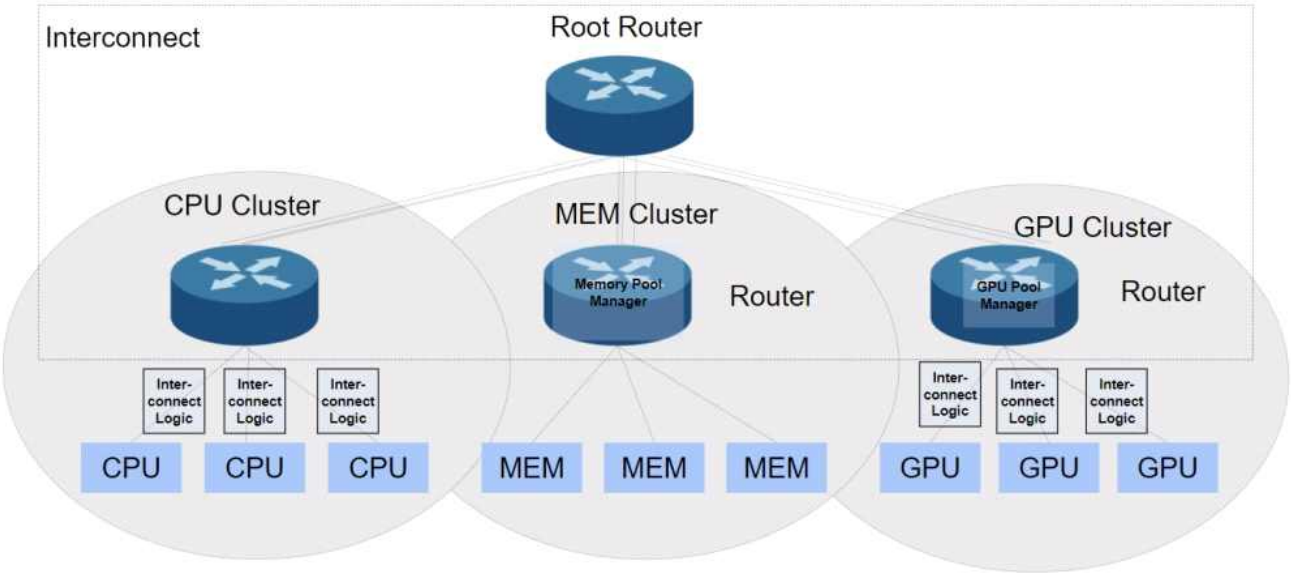


그림 9 우리 시스템을 위한 2-레벨 팻트리 토폴로지 구성

(3) 인터페이스

인터커넥트 시뮬레이터에서 구성 요소로부터 받은 패킷을 라우팅을 위한 패킷으로 변환하기 위한 기능을 구현하였다. 가넷과 구성 요소는 루비 시스템의 캐시 컨트롤러에 의해 연결된다. 가넷의 스탠드얼론(standalone)에서 캐시 컨트롤러는 더미 프로토콜을 실행하여 구성 요소로 패킷을 받아 가넷으로 전달하는 역할을 한다. 따라서 캐시 컨트롤러는 개별분리 시스템의 구성 요소와 인터커넥트를 연결하는 인터페이스 역할을 하는 것을 알 수 있다. 캐시 컨트롤러는 캐시 일관성 프로토콜에 의해 수행할 작업이 결정된다. Gem5는 이 캐시 컨트롤러의 캐시 일관성 프로토콜을 이용하여 패킷을 라우팅이 가능한 패킷의 형태인 메시지(Message)로 변환한다. 이 점을 이용하여 캐시에 대한 작업을 하지 않는 더미 캐시 일관성 프로토콜을 작성한 후, 패킷을 메시지로 변환하는 기능을 추가하였다.

인터페이스에서 신뢰 가능한 전송을 지원하기 위하여, 인터페이스 클래스를 수정하였다. 구성 요소의 각 인터페이스 클래스에 패킷을 전송한 후, 이를 저장하는 재전송 버퍼(replay buffer)와 타이머를 구현하였다. 또한 인터페이스가 설정한 타이머가 타임아웃이 될 때까지 ACK 패킷을 받지 못하거나, NACK 패킷을 받았을 경우, 재전송 버퍼에 저장되어있는 패킷을 재전송하도록 구현하였다. 타이머가 타임아웃되기 전에 ACK 패킷을 받은 경우는 재전송 버퍼에서 ACK 패킷이 가리키는 패킷을 제거한다.

데이터 손실에 검사에 따른 재전송 기능은 시뮬레이션 환경에서는 필수가 아니므로 구현에서 생략되었다. 구현된 시뮬레이터에서는 패킷의 CRC 필드의 값은 항상 0이며, 전송받은 패킷에 대하여 CRC 값을 계산하여 CRC 필드와 비교하는 기능 또한 생략하였다.

4) 메모리 구현

(1) 메모리 풀 관리자와 패킷 다중화

위 설계에서 언급한 메모리 풀 관리자의 기능을 구현하기 위해, Gem5 시뮬레이터의 크로스바 (Crossbar) 클래스를 수정했다. 기존 크로스바 클래스는 단순히 여러 포트를 통해 Gem5 구성 요소를 연결하는 기능만 가지고 있으며, 개별분리 시스템에 해당하는 패킷 라우팅 및 헤더 처리 기능은 갖추고 있지 않았다. 이를 수정하여, 상기 설계에서 언급한 기능을 추가했다.

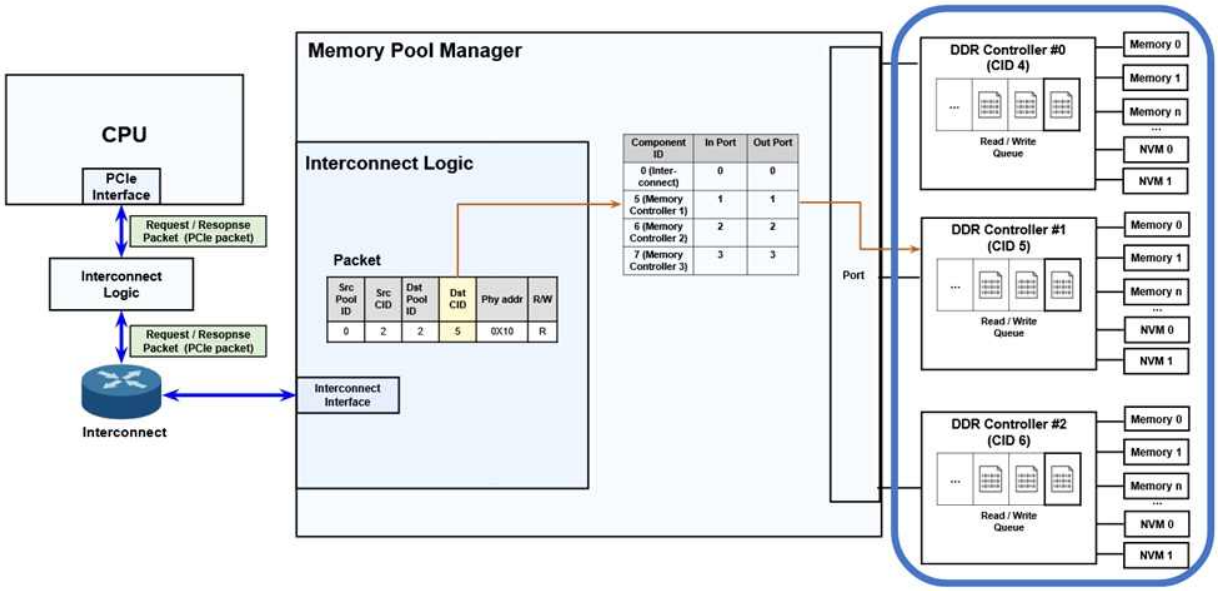


그림 10 메모리 풀 관리자의 구조

- 인터커넥트 로직을 통해 인터커넥트와 연결 (그림 9에서 메모리 풀 관리자의 인터커넥트 로직 내 인터커넥트 인터페이스 부분)
- 여러 개의 메모리 컨트롤러와 연결 (그림 9에서 오른쪽 둥근 사각형 부분)
- 라우팅 정보를 참고하여 패킷을 올바르게 전달 (상기 설계의 패킷 다중화 항목, 위 그림 9에서 Packet의 목적지 CID 정보를 참고해 올바른 메모리 컨트롤러로 패킷을 전달하는 것을 나타낸 주황색 화살표 흐름)

상기 설계에서는 오류 제어 기능을 포함했으나, 시뮬레이터 상에서 이러한 기능을 실제로 테스트하는 방법은 의도적으로 에러를 일으키는 방법뿐이므로, 이 기능들은 더미 함수로만 구현했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 메모리 컨트롤러와 다양한 미디어 지원

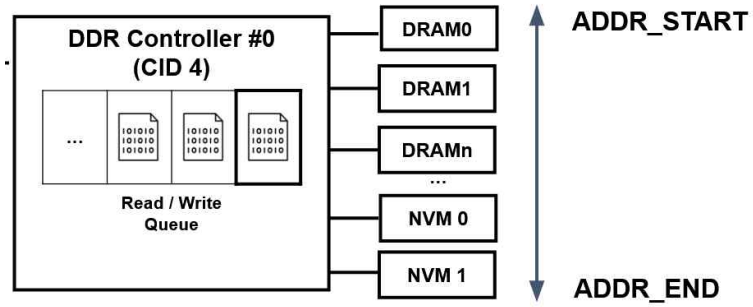


그림 11 하나의 메모리 컨트롤러에 다수의 DRAM과 NVM이 연결된 모습

위 설계에서 언급한 메모리 풀 관리자의 기능을 구현하기 위해, Gem5 시뮬레이터의 MemCtrl 클래스를 수정했다. 기존 MemCtrl 클래스는 하나의 DRAM 미디어와 하나의 NVM 미디어만을 연결할 수 있었다. 개별분리 시스템에서는 여러 개의 저장 미디어를 유연하게 관리할 수 있어야 하므로, 이러한 제한을 해결하기 위해 다음과 같은 사항을 새로 구현했다.

- 메모리 미디어를 설정 스크립트에 따라 임의의 개수만큼 컨트롤러에 연결할 수 있도록 수정 (위 그림 10에서 오른쪽 DRAM0~n과 NVM0~1)
- 여러 개의 미디어가 가지는 용량을 병합해 컨트롤러당 하나의 병합된 미디어 주소 공간을 가짐 (위 그림 10에서 주소 공간의 시작과 끝 범위를 나타내는 오른쪽 화살표)
- 요청 패킷의 주소 필드에 따라 여러 개의 미디어 중 적절한 미디어로부터 요청을 서비스하도록 패킷 처리 알고리즘을 수정

5) 가속기(GPU 등) 구현

현재 Gem5 내부의 DMADevice를 상속하는 PCIDevice를 GPU로 상정하고 인터커넥트 로직과 Gem5 포트와 PCI Bridge 형태로 연결하여 구성하였다. 또한 본 과제에서 개발된 Gem5 기반의 개별분리 시스템과 병합하여 메모리 풀 관리자가 CPU 노드와 MMIO 채널을 수립하고 인터커넥트 로직을 통해 패브릭 주소 전환 및 라우팅을 거쳐 통신하는 전 과정을 구현하고 테스트하였다. 차년도에는 이를 확장하여 Gem5-gpu 등의 cycle-accurate 가속기 시뮬레이션을 사용하여 실제 상황에 가까운 형태로 확장할 예정이다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-2. 2년차(2021년)의 연구 범위

본 과제의 2년차 연구 범위와 관련 기관은 다음과 같다.

2-2-1 개발 목표

- **주관기관(KAIST), 공동연구협력기관 1(CISPA)**
  개별분리 시스템 시뮬레이터를 활용한 메모리 접근 제어기 설계 및 개발, 개별분리 시스템 시뮬레이터에 적용되는 프로토콜 설계 및 개발.

- **참여기관 1(성균관대학교)**
  개별분리 시스템상에서의 CPU와 가속 기간의 안전한 오프로딩 아키텍처 설계.

2-2-2. 개발 내용 및 범위

- **주관기관(KAIST), 공동연구협력기관 1(CISPA)**
  개별분리 시스템 시뮬레이터를 활용한 메모리 접근 제어기 개발 1차 연도에 개발한 개별분리 시스템 시뮬레이터를 활용하여 현재 기존 개별분리 시스템에서는 제공하지 않는 메모리 접근 컨트롤러를 개발하거나 새로운 메모리 접근 컨트롤러가 개발되었다면 더 좋은 성능을 보이는 컨트롤러를 개발하고자 한다. 해당 메모리 접근 컨트롤러는 기존의 CPU의 메모리 관리 유닛에서 제공하는 메모리 접근제어와 호환이 되고, 개별분리 시스템 레벨에서 각 하드웨어 풀 사이의 접근을 제어하여 큰 규모의 클라우드 시스템에서 각 사용자가 사용하는 하드웨어 사이에 효율적인 접근제어 메커니즘을 제공하는 것을 목표로 한다.

  기존의 개별분리 시스템에서 제공하는 기본적인 메모리 접근 통제 방식은 여러 프로세서와 메모리들은 각자 고유의 키를 가지고 있고, 스위치는 각 하드웨어의 키들을 연결하는 토폴로지를 가지고 있다. 그리고 해당 토폴로지를 통해 각 하드웨어 구성 요소 사이의 분리를 제어한다. 이러한 키 기반의 토폴로지를 통한 접근제어 방식은 시스템에 존재하는 각 하드웨어 풀 사이의 연결이 고정되어 있을 때는 상대적으로 성능 저하에 적은 영향을 미칠 수 있다. 하지만, 고성능 대역폭을 지원하고 토폴로지 설정이 시스템 실행 중에 동적으로 바뀌어야 하는 서버 컴퓨터상의 개별분리 시스템에서는 새로운 토폴로지 설정 및 많은 패킷 트래픽으로 인해 성능에 큰 영향을 미칠 수 있는 여지가 남아 있다.

  따라서 본 연구에서는 기존의 개별분리 시스템에서 제안한 키 기반의 접근제어 시스템에서 동적 설정으로 인해 발생할 수 있는 성능 문제 및 보안상의 문제점을 파악하고 해결해보고자 한다. 이를 위해 CISPA와의 협력을 통하여 개별분리 시스템을 테스트하기 위한 환경구축에 대한 자문을 얻고, 개별분리 시스템 보안 평가를 위한 테스트베드를 구축한다.

- **참여기관 1(성균관대학교), 공동연구협력기관 1(CISPA)**
  1) 가속기 풀 컨트롤러 (Accelerator Pool Controller)의 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

가속기 풀 컨트롤러는 풀에 존재하는 여러 가속기마다 각각 고유한 숫자(UUID)를 부여하여 각각을 식별하고 관리할 수 있도록 한다. 또한, 다수의 가속기가 묶인 풀에서 각각의 가속기마다 고유의 비대칭 암호화키를 부여하여 CPU TEE(Trusted Execution Environment, 신뢰 실행환경)와의 원격 인증 및 암호화 채널 구성을 가능하게 하는 역할을 하며, 성립된 암호화 채널들을 관리하는 역할을 수행한다. 본 과제에서는 이러한 역할을 하는 가속기 풀 컨트롤러를 개발할 예정이다. 성균관대는 CISPA의 보안 아키텍처 개발을 위한 지식과 경험을 활용하여 개별분리 시스템의 가속기 접근제어 모델 연구 및 개발에서 시뮬레이터 중 가속기 pool 컨트롤러를 설계할 예정이다.

2) 개별분리 CPU TEE, 가속기 원격 인증 및 키 교환 모델 개발

KAIST 측에서 개발할 중앙보안컨트롤러 (Central Trust Management Unit)이 제공하는 인터페이스를 이용하여 CPU TEE와 가속기 간의 통신에서 서로의 정체성(Identity)과 무결성(Integrity)을 검증하고 안전한 채널을 성립하는 프로토콜의 개발이 필요하다. 본 연구에서는 개별분리 시스템상에서 다른 Node가 끼어들지(man-in-the-middle attack) 못하도록 중앙보안컨트롤러를 통한 인증서 검증, 그리고 키 교환을 통해 CPU와 가속기 간의 암호화된 통신 채널을 성립하는 모델을 개발한다.

2-2-3. 설계 개요

기존에 사용하는 클라우드 아키텍처에서는 CPU, GPU 등의 컴포넌트들이 매우 밀접하게 결합되어 이들의 상호작용 방식을 바꾸기가 어려웠다. 하지만 개별분리 시스템은 이러한 컴포넌트들이 더 이상 같은 Machine 내의 시스템 버스에 속하지 않았기 때문에, 컴포넌트 간의 안전한 채널 및 접근 통제를 지원할 수 있을 것으로 예상한다. 본 연구에서는 차세대 개별분리 클라우드 시스템의 자원들의 보안을 중앙에서 관리할 수 있는 중앙 관리 유닛(Central Management Unit)을 제안한다.

본 과제에서 설계하고자 하는 세부 컴포넌트는 다음과 같다.

- **메모리와 가속기 접근제어 모델**
  개별분리 시스템에서 다른 pool 간의 메모리 접근제어를 함으로써 악의적인 사용자가 다른 사용자의 데이터에 접근하는 것을 차단할 수 있다.

- **안전한 가속기 오프로딩**
  가속기와 CPU에서 실행될 코드와 데이터를 처리하고, 서로 데이터를 주고받으면 데이터의 무결성을 보장하기 위해 암호화된 채널을 통해 안전하게 오프로딩하는 프로토콜이 필요하다.

- **안전한 프로토콜**
  개별분리 아키텍처에서 사용되는 프로토콜은 다양한 공격시나리오에 대비하여

설계하여야 한다. 대표적인 예로 메시지 패킷 변조 및 위조 공격이 있으며, 그 외에도 내/외부에서 프로토콜의 취약점을 이용한 공격이 발생할 수 있다.

1) 설계 가정 사항

본 과제에서 우리 시스템을 다음과 같은 설계상의 가정을 기반으로 설계한다.

- **완전 개별분리 시스템과 부분 개별분리 시스템**

  우리 시스템은 각 컴퓨트 노드가 별도의 로컬 메모리를 가지고 있지 않은 완전 개별분리 시스템(Fully disaggregated system)으로 가정한다. 이와 반대되는 개념으로 부분 개별분리 시스템(Partially disaggregated system)이 있다. 원격 메모리에의 접근이 충분히 빠르지 않은 현시점에서 부분 개별분리 시스템의 compute node의 내부에 로컬 메모리를 둠으로써 일종의 캐시로 사용할 수 있다. 그러나 이러한 부분 개별분리 시스템은 컴퓨트 노드의 설계가 복잡해지고, 메모리가 원격 메모리와 로컬 메모리로 이중으로 나뉘어 비용이 증가하게 된다. 그뿐만 아니라 메모리 계층이 한 단계 증가함으로써 로컬 메모리를 위한 캐시 알고리즘에 대한 비용이 역시 증가한다. 따라서 부분 개별분리 시스템은 완전 개별분리 시스템으로의 과도기적 대안일 뿐 궁극적으로는 완전 개별분리 시스템을 지향해야 한다. 본 과제에서는 이러한 점으로 미루어 우리 시스템을 완전 개별분리 시스템으로 설계 및 구현한다.

- **신뢰 모델**

  우리 시스템은 중앙 관리 유닛(Central Management Unit)을 신뢰한다. 따라서 중앙 관리 유닛이 생성하고 파괴하는 키(key) 역시 신뢰할 수 있다. Capability 토큰을 보관하고 검증하는 메모리 풀 매니저와 인터커넥트 로직을 신뢰한다. 반대로, 우리 시스템은 인터커넥트 로직과 메모리 풀 매니저 사이의 인터커넥트는 신뢰하지 않으며, 공격자는 네트워크상에서 중간에 임의로 조작(중간자 공격)을 시도할 수 있다.

- **연구 범위**

  본 과제에서는 공격자가 시스템을 물리적으로 접근할 수 없으며, 시스템이 물리적으로 안전하다고 가정한다. 따라서 시스템에 물리적으로 침해하여 시스템을 조작하거나 파괴하는 행위, 시스템에 추가적인 장치를 설치하여 조작 또는 엿듣는(eavesdropping), 메모리를 물리적으로 탈취하여 내부 데이터를 복원하는 행위(cold boot), 전력 소모량 등으로 알고리즘을 분석하는 부채널 공격(side channel attack) 등의 공격은 범위에 두지 않는다. 그뿐만 아니라 패킷 손실, 하드웨어 장치 장애 등의 서비스 거부 공격에 대한 내용, CPU 코어 간의 공격 및 CPU 자체의 취약점 등 노드 내부에서의 공격과 방어에 관한 내용은 다루지 않는다. 이러한 공격은 본 연구와는 별개의 주제이다.

2) 전체 시스템 개요

- Capability 기반의 접근제어

Capability 기반의 접근제어 (Capability-based Access Control) 는 신뢰할 수 없는 각 컴퓨트 노드가 원격 메모리에 접근하는 것을 통제하기 위해 사용된다. 우리의 시스템에서, 각 컴퓨트 노드는 원격 메모리를 메모리 풀 매니저를 통해 할당받는다. 할당받은 원격 메모리는 인터커넥트 로직을 통해 접근할 수 있다. 그러나 이때, 다음과 같은 보안 문제가 발생할 수 있다.

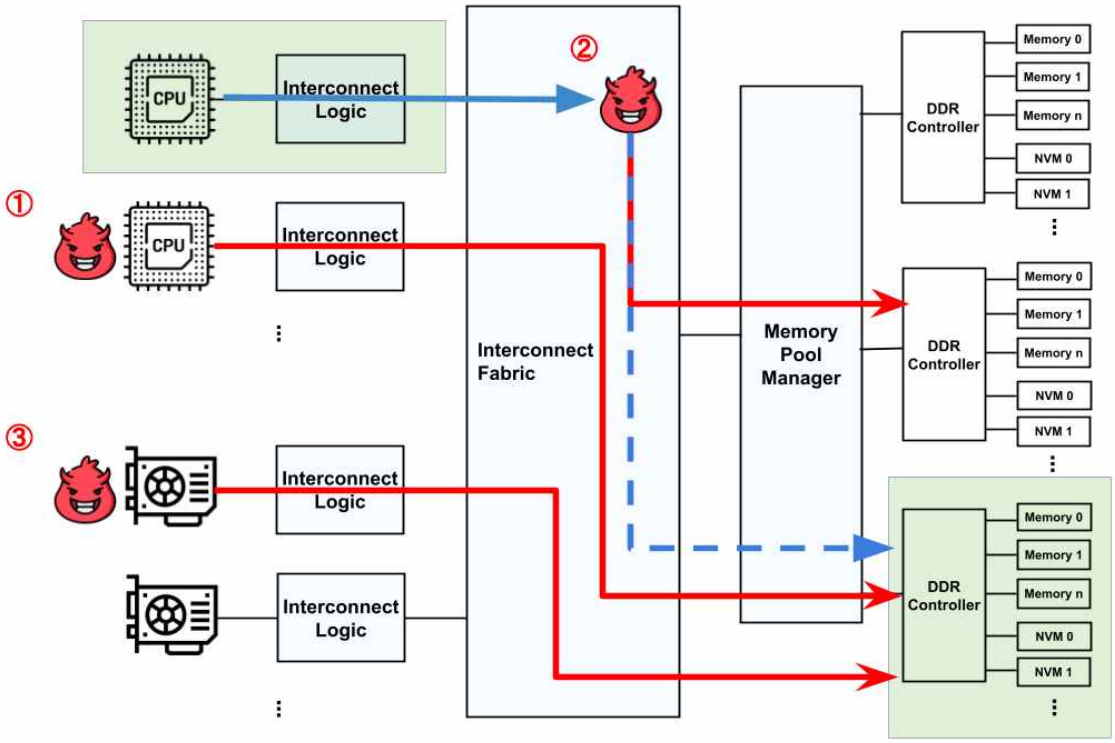


그림 12 개별분리 시스템의 보안 위협

녹색 배경의 컴퓨트 노드에 녹색 배경의 원격 메모리 풀이 할당되었다. 정상적인 접근 경로는 파란색 화살표이다. 이 때 발생 가능한 보안 위협은 붉은색 화살표로 표시되었다. 파란색 점선은 인터커넥트가 신뢰 가능할 때 정상적으로 메모리 접근 라우팅되는 경로이다.

1) 어느 한 컴퓨트 노드가 자신이 할당받은 범위를 벗어난 원격 메모리에 접근을 시도할 수 있다. 예를 들어, 컴퓨트 노드 상에서 동작하는 운영체제가 페이지 테이블 등을 임의로 조작함으로써, 자신이 할당받지 않은 원격 메모리의 물리 주소에 접근하려고 시도할 수 있다.

2) 인터커넥트를 신뢰할 수 없으므로, 컴퓨트 노드와 원격 메모리 사이에서 전송되는 데이터가 변조되거나 도청될 수 있다. 예를 들어, 인터커넥트가 컴퓨트 노드에 잘못된 데이터를 전달하기 위해서 올바르지 않은 원격 메모리 주소로 전송되는 데이터를 라우팅할 수 있다.

3) 어느 가속기가 자신과 연계되지 않은 컴퓨트 노드의 데이터에 접근할 수 있다. 예를 들어, 가속기를 사용하지 않는 애플리케이션을 실행하는 컴퓨트 노드

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

A가 원격 메모리에 데이터를 저장했다고 하자. 악의적인 가속기 B는 원격 메모리에 자신이 A와 연계된 것처럼 위장해 데이터를 훔치거나 조작하려 할 수 있다.

이러한 문제를 해결하기 위해서는 메모리를 읽거나 쓸 때 접근제어가 필수적이다. 그러나, 컴퓨트 노드와 가속기를 사용하는 임차인(tenant)이 시스템 레벨의 소프트웨어(예를 들어, 드라이버)를 포함한 임의의 코드를 실행할 수 있는 것을 고려한다면 접근제어를 위한 메커니즘은 그 외부에 구현되어야 한다.
이는 기존 시스템에서 주로 사용되었던 접근 제어 방식에 대한 전면적인 재수정을 해야 한다. 예를 들어, 기존 프로세서의 메모리 관리 유닛 (Memory Management Unit, MMU)를 이용한 접근제어의 경우 운영체제에서 악의적인 메모리 매핑 변경을 시도할 수 있으므로 사용할 수 없다.
또한, 개별분리 시스템에 새로 도입되는 구성 요소 중에서도 신뢰할 수 있는 최소한의 요소만을 이용해서 접근제어를 수행하여야만 한다. 예를 들어, 인터커넥트를 신뢰할 수 없으므로 전송 중에 접근제어를 수행해서는 안 된다.
따라서, 접근제어는 우리 시스템에서 신뢰할 수 있는 요소인 중앙 관리 유닛 (Central Management Unit), 메모리 풀 매니저, 인터커넥트 로직을 이용해 구현되어야 한다. 구현된 접근제어 메커니즘은 위에서 언급된 보안 문제를 모두 방지할 수 있어야 하며, 접근제어에 따른 성능 저하를 최소화할 수 있도록 각 인터커넥트 로직은 capability 일부를 캐싱하는 등의 최적화를 적용할 수 있다.

- **안전한 오프로딩(Secure Offloading)**

  안전한 오프로딩은 가속기, 사용자 및 메모리 풀 매니저 간의 신뢰를 필요로 한다. 이를 위해 가속기 자체에서 안전한 컴퓨팅을 지원해야 한다. 가속기가 패브릭에 연결을 시도하면, 메모리 풀 매니저는 가속기를 검증하는 원격인증(remote attestation)을 먼저 시행한다. 이는 안전한 장치로 위장하는 중간자 공격(Man-in-the-middle attack)을 수행하려는 악의적인 집단을 막기 위함이다. 그 다음, 메모리 풀 매니저와 안전한 가속기 간의 키 교환(key exchange)이 일어난다. 키 교환 후, 메모리 풀 매니저와 가속기는 공유한 세션 키(session key)를 서로 간의 통신 메세지 암호화에 사용한다. 사용자는 메모리 풀 매니저에게 요청하여 안전한 가속기를 다룰 수 있다. 그리고 사용자는 안전한 가속기에서 실행할 로직을 가진 가속 커널을 암호화하여 메모리 풀 매니저에게 보낸다. 메모리 풀 매니저는 커널을 암호화해 가속기에게 보내고, 커널이 올바르게 로드되었는지 (장치 메모리에 로드된 커널의 해시 체크섬을 통해) 검증하는 장치 측정(device measurement)을 수행한다. 그 후, 사용자는 메모리 풀 매니저를 통해 가속기에게 메모리 capability(capability transference key)를 위임한다. 가속기는 capability를 통해 사용자 정보를 담고있는 메모리에 접근할 수 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3) Capability-based Access Control의 설계

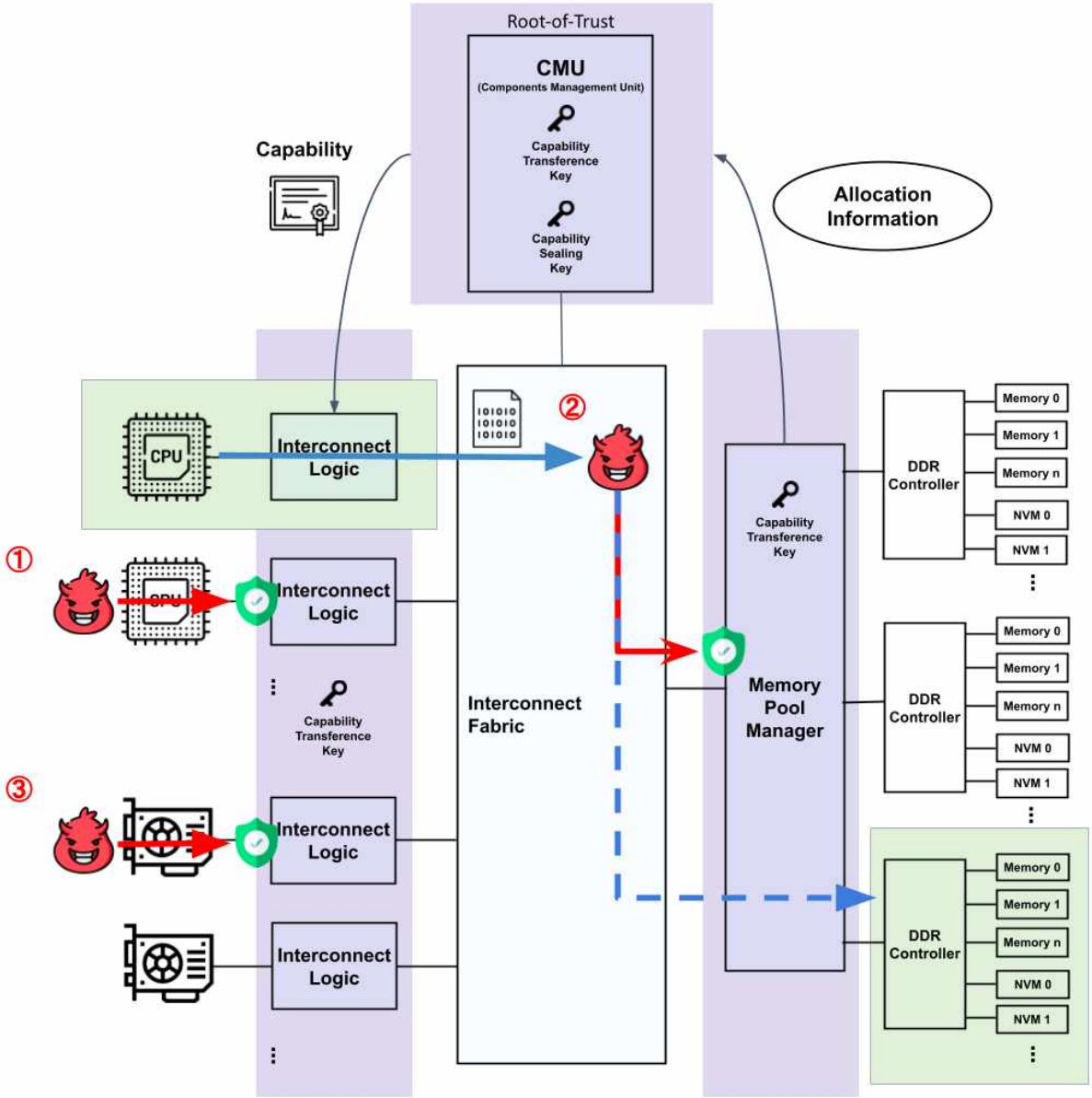


그림 13 Capability 기반 접근 제어

신뢰 가능한 요소는 보라색 배경으로 나타낸 중앙 관리 유닛, 메모리 풀 매니저, 인터커넥트 로직이다. 각 요소가 접근 가능한 키가 같이 표시되어 있다. 방패 표시는 세 요소들에 의해 위협이 차단되는 지점을 나타낸다.

우리 시스템에서 설계된 capability 기반의 접근제어는 중앙 관리 유닛, 메모리 풀 매니저, 인터커넥트 로직에서 수행된다. 각 컴퓨트 노드 및 가속기는 초기화 과정에서 자신이 사용할 수 있는 원격 메모리 영역을 할당받는다. 이때, 신뢰할 수 있는 세 가지 요소에 의해 할당받은 영역에 대한 capability가 생성되고, 이후 메모리 접근 과정에서 접근을 시도하는 주소와 capability의 권한이 비교된다.
중앙 관리 유닛은 메모리 할당 과정에서 capability를 생성하고 각 컴퓨트 노드 또는 가속기와 연결된 인터커넥트 로직에 해당 capability를 전달한다. 인터커넥트 로직은 전달받은 capability를 참고해 컴퓨트 노드 혹은 가속기가 메모리에 접근을 시도할 때마다 이를 제어한다. 메모리 풀 매니저는 위 과정에서 할당 관리 목적의 메

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

타데이터를 중앙 관리 유닛에 전달한다.
Capability 생성 과정에서 신뢰할 수 없는 요소에 의한 위조/변조 등을 방지해야 할 필요가 생긴다. 이를 위해, 생성 과정에 관여하는 모든 신뢰 가능한 요소는 capability transference 키를 공유한다. 이 키는 다른 신뢰할 수 없는 요소에 노출되지 않도록 분리된 영역에 존재한다.

(1) 중앙 관리 유닛

가) 접근 권한 부여

중앙 관리 유닛은 capability 기반 접근제어를 수행하는 과정에서 중추적인 역할을 한다. 접근제어 과정에서 취약점이 발생할 수 있는 경우를 최소화하기 위해서, 모든 접근 권한의 부여는 중앙 관리 유닛에 의해서 수행된다. 접근 권한의 부여는 각 컴퓨트 노드 또는 가속기의 초기화 과정에서 일회성으로 수행되므로, 중앙 관리 유닛에 의한 성능 병목현상은 일어나지 않는다.

나) 신뢰 근원 역할

중앙 관리 유닛은 Capability 생성 및 사용 과정에서 신뢰 근원 (Root of trust)의 역할을 한다. 메모리 영역 할당시, 중앙 관리 유닛은 각 컴퓨트 노드 또는 가속기가 접근할 수 있는 영역 및 다른 메타데이터를 담은 capability 데이터를 생성하고, 자신이 가진 capability sealing 키로 서명한다. Capability sealing 키는 중앙 관리 유닛이 외부 요소와 격리된 공간에 보관한다. 이는 신뢰할 수 없는 다른 요소가 capability를 위조하는 공격을 방지한다. 또한, 올바르게 서명된 capability는 오직 중앙 관리 유닛에 의해서만 생성할 수 있으므로 시스템의 다른 구성 요소는 서명을 검증하는 것으로 capability가 중앙 관리 유닛에 의해 생성되었음을 신뢰할 수 있다.

다) Capability 구성

중앙 관리 유닛은 자신이 초기화 중인 컴퓨트 노드 또는 가속기에 할당할 영역이 올바른 영역인지 (예를 들어, 다른 컴퓨트 노드에 중복으로 할당되지 않았는지)를 검사하기 위해 메모리 풀 매니저가 관리 중인 정보를 사용한다. 해당 정보는 capability transference 키로 서명되어 전달된다. 전달받은 할당 정보는 다른 메타데이터와 함께 capability 구조 안에 포함된 이후 중앙 관리 유닛에 의해 capability sealing 키로 서명되어 최종적으로 생성된다.
위 과정에서 메모리 풀 매니저와 중앙 관리 유닛 사이의 채널이 신뢰할 수 있는 경우 (예를 들어, 하드웨어적으로 결합된 경우)에는 할당 정보를 전송하는 과정에서 capability transference 키로 하는 서명을 생략할 수 있다. 하지만 그렇지 않은 경우(예를 들어, 신뢰할 수 없는 인터커넥트를 채널로 사용하는 경우)에는 할당 정보의 변조를 막기 위해 서명이 필요하다.

라) 인터커넥트 로직에 생성된 capability를 전달

생성된 capability는 초기화 중인 컴퓨트 노드 또는 가속기와 연결된 인터커넥트

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

로직에 전달된다. Capability는 Capability sealing 키로 서명이 완료되었기 때문에, 추가적인 전송 채널 보호는 생략할 수 있다.

(2) 인터커넥트 로직

가) 전달된 capability의 검증

메모리 할당 과정에서 중앙 관리 유닛이 만든 capability를 전송받은 인터커넥트 로직은 중앙 관리 유닛의 capability sealing 키로 capability가 서명되었는지 검증한다. 검증이 성공하기 이전까지 인터커넥트 로직은 자신이 연결된 컴퓨트 노드 또는 가속기의 메모리 접근을 허가하지 않는다. Capability 검증 과정은 컴퓨트 노드 또는 가속기의 초기화 과정에서 일회성으로 일어나므로, 성능 병목 현상이 일어나지 않는다.

나) Capability 기반의 접근제어

초기화 과정에서 capability의 검증이 성공적으로 수행되었다면, 인터커넥트 로직은 이후 연결된 컴퓨트 노드 또는 가속기가 수행하는 모든 메모리 접근을 해당 capability가 지정한 영역 정보 및 메타데이터와 비교한다. 이 과정은 컴퓨트 노드 또는 가속기가 재 초기화 되어 다른 원격 메모리 영역을 할당받을 때까지 유지된다.

만약 capability에서 허가되지 않은 메모리 접근이 수행된다면, 인터커넥트 로직은 해당 메모리 접근을 차단한다. 이때 추가로 외부 인터럽트 등을 통해 연결된 컴퓨트 노드나 가속기에 문제가 발생했음을 알릴 수 있다. 만약 범위를 벗어난 접근이 의도하지 않은 버그일 경우, 이를 통해 잘못된 매핑을 수정할 수 있다.

다) 접근제어 성능 향상

Capability 기반의 접근제어는 모든 메모리 접근에 대해서 수행되므로, 성능 병목현상에 영향을 미칠 수 있다. 인터커넥트 로직은 각 컴퓨트 노드 또는 가속기마다 존재하므로, 다른 구성 요소에 의해 성능이 영향을 받지 않으며 독립적으로 최적화할 수 있다.

Capability가 표현하는 메모리 영역은 경우에 따라 파편화되어 있을 수 있다. 파편화가 심한 경우, 비교할 메모리 범위를 가진 capability의 크기가 한 번에 처리하지 못할 정도로 커질 수 있다. 이런 경우, 각 메모리 접근을 제어하는 중 메모리 주소와 capability 범위 비교를 여러 번에 걸쳐 수행해야 해 과도한 오버헤드가 발생할 가능성이 있다. 따라서 인터커넥트 로직은 자주 사용되는 영역에 대한 규칙 일부를 자체적으로 캐싱(caching)할 수 있다. 이런 방법은 일반적으로 메모리 접근이 시간적/공간적인 지역성(temporal/spatial locality)을 가지고 있으므로, 오버헤드를 감소시킬 수 있다.

라) 접근제어 보장

인터커넥트 로직은 신뢰할 수 있는 요소이므로, capability가 올바르게 설정된다

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

면 컴퓨트 노드 또는 가속기에서 수행되는 코드의 종류와 관계없이 접근제어를 수행할 수 있다. 또한, 신뢰할 수 없는 요소가 인터커넥트 로직에 위조된 capability를 임의로 설정할 수 없도록 새로운 capability가 전달될 때마다 중앙 관리 유닛의 서명을 항상 검증한다.

마) 라우팅 무결성 보장

인터커넥트 로직에서 올바르게 접근제어가 수행된다고 하더라도, 인터커넥트는 자신이 메모리 접근을 라우팅하는 과정에서 헤더 정보 등을 변조하는 방법으로 올바르지 않은 메모리 영역으로 접근을 유도할 가능성이 있다. 따라서, 원격 메모리로 전달되는 데이터는 모두 capability transference 키로 서명된 이후 전송된다. 이 과정은 모든 메모리 접근에 필요하므로 병목현상을 일으킬 수 있으나, 암호 알고리즘 연산에 특화된 전용 하드웨어 등을 사용하여 최적화하기 쉽다 (아래 5) 암호 알고리즘 요구 사항 참고.)

(3) 메모리 풀 매니저

가) 할당 정보 관리

메모리 풀 매니저는 각 컴퓨트 노드 또는 가속기에 할당된 영역에 대한 정보를 관리한다. 각 컴퓨트 노드 또는 가속기는 자신에게 할당된 메모리 영역을 독점적으로 사용할 수 있으며, 메모리 풀 매니저는 원격 메모리가 중복으로 할당되는 경우를 방지한다.

나) 초기화 과정 중 중앙 관리 유닛에 정보 제공

중앙 관리 유닛이 메모리 할당 및 capability를 생성하는 과정에서, 메모리 풀 매니저는 현재 사용되지 않고 있는 영역에 대한 정보를 제공한다. 중앙 관리 유닛은 전달받은 정보를 다른 메타데이터와 함께 서명해 capability를 만들 수 있다. 1) 중앙 관리 유닛 - (3) Capability 구성에서 설명했듯이, 전송 채널의 신뢰 여부에 따라 이 정보는 capability transference 키로 보호되어야 한다.

다) 병목현상 방지를 위한 설계

메모리 풀 매니저는 원격 메모리에 전달되는 모든 패킷을 올바른 메모리 컨트롤러로 라우팅하는 역할도 담당하고 있다. 이 과정은 전체 시스템에 병목현상을 일으킬 가능성이 있으므로, 메모리 풀 매니저가 수행하는 작업은 되도록 최소화되어야 한다.

따라서, 우리 시스템에서는 capability의 검사를 메모리 풀 매니저가 아닌 인터커넥트 로직에서 수행한다. 이 방법은 각 인터커넥트 로직이 독립적으로 동작하므로 서로 성능에 영향을 미치지 않으며, 올바르지 않은 접근을 최대한 이른 시점에 차단할 수 있다는 장점이 있다.

라) 라우팅 무결성 보장

2) 인터커넥트 로직 - (5) 라우팅 무결성 보장에서 설명했듯이, 인터커넥트가 전

송 도중 데이터를 변조하는 위협이 존재할 수 있다. 따라서 메모리 풀 매니저는 송수신하는 데이터가 올바르게 전송되었는지 capability transference 키를 사용해 검증한다.

(4) 접근제어 보안 분석

전체 시스템 개요에서 설명했던 보안 위협 중 비인가 컴퓨트 노드/가속기의 원격 메모리 접근은 위의 capability 기반 접근제어에 의해 해결될 수 있다.

가) Capability 위조를 통한 비인가 접근 시도

인터커넥트 로직은 모든 원격 메모리 접근을 capability에 기반해 제어한다. 이 capability는 컴퓨트 노드/가속기의 외부인 인터커넥트 로직에 존재하므로 컴퓨트 노드/가속기가 직접 수정할 수 없다. 또한, 컴퓨트 노드/가속기는 중앙 관리 유닛의 capability sealing 키 없이는 capability를 위조해 인터커넥트 로직에 전달하더라도 검증을 통과할 수 없다. 그러나 capability sealing 키는 중앙 관리 유닛만이 격리된 영역에 가지고 있고, 다른 시스템 요소에 전달되지 않으므로 capability 위조를 통한 접근 제어 위반은 불가능하다.

나) 인터커넥트 로직을 경유하지 않는 원격 메모리 접속 경로

컴퓨트 노드/가속기는 반드시 인터커넥트 로직을 통해서만 원격 메모리에 접근할 수 있으며, 가정에 따라 물리적인 하드웨어 연결은 침해되지 않는다 (예를 들어, 인터커넥트 로직 없이 직접 인터커넥트에 접속되지 않는다). 따라서, capability 기반 접근 제어는 우회 불가능하다.

(5) 암호 알고리즘 요구 사항

Capability의 생성과 사용 과정에서 암호 알고리즘을 이용해 전송되는 데이터의 무결성을 보장해야 한다. 이 과정에서 사용되는 키는 두 가지가 있으며, 사용 목적에 따라 다른 종류의 키를 사용한다.

가) Capability transference key

Capability를 생성하거나 사용할 때, 신뢰할 수 있는 요소 간의 통신이 이루어진다. 그러나 이때 통신 채널은 신뢰할 수 없는 요소인 인터커넥트를 거치기 때문에, 종단 간 요소에서 전송되는 데이터의 무결성을 보장하는 방법이 필요하다.

신뢰할 수 있는 요소 간의 통신은 매우 빈번하게 이루어지기 때문에 (예를 들어, 인터커넥트 로직과 메모리 풀 매니저 간의 모든 통신) 고속으로 무결성을 검증하는 방법이 필요하다. 이를 위해서 capability transference key는 대칭키 기반의 메시지 인증 코드 (Message Authentication Code)를 사용한다. 이 알고리즘은 별도의 하드웨어를 통해 병목현상이 일어나지 않도록 연산을 가속할 수 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

나) Capability sealing key

Capability를 생성하는 주체는 신뢰 근원인 중앙 관리 유닛뿐이며, 다른 시스템 구성 요소는 중앙 관리 유닛이 capability를 생성했는지를 검증할 수 있어야 한다. 반면, capability의 생성은 각 컴퓨트 노드나 가속기가 초기화되는 시점에만 수행되므로 자주 일어나지 않는다. 따라서 중앙 관리 유닛은 비대칭키 기반의 capability sealing key를 사용해서 자신이 생성한 capability에 서명하며, 해당되는 비밀키는 외부 시스템 요소와 격리된 곳에 유지한다. 다른 시스템 구성 요소는 비밀키에 대응되는 공개키로 capability가 중앙 관리 유닛에 의해 서명되었는지를 검증할 수 있다.

(6) 프로토콜 안전성 분석

컴퓨트 노드나 가속기의 접근제어가 성공적으로 이루어지더라도, capability 전달과 메모리 접근은 신뢰할 수 없는 인터커넥트를 통해 이루어진다. 따라서, 요소 간의 통신을 포함하는 과정은 전송되는 데이터의 무결성을 보장할 수 있는 방식으로 수행되어야 한다. 그림 13은 인터커넥트에 의해 발생할 수 있는 위협과, 위협을 방지하기 위해 전송 도중에 HMAC/서명값을 포함하는 것을 나타낸다.

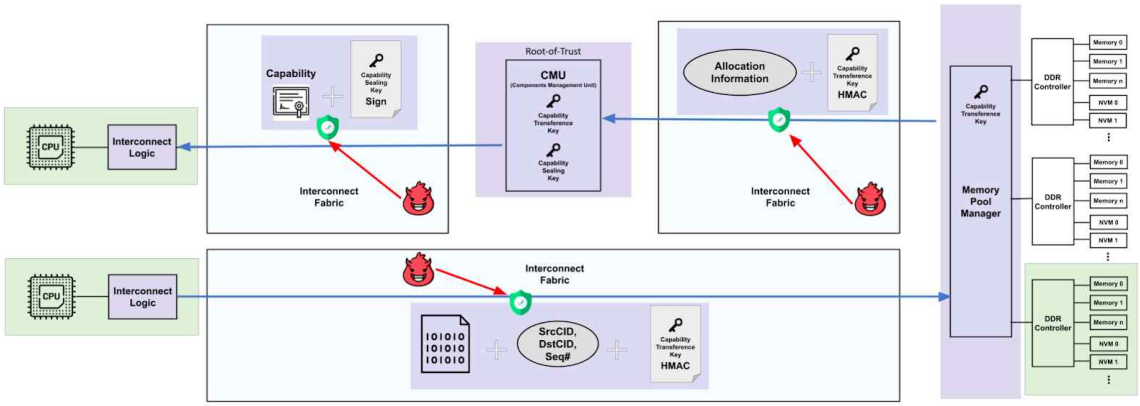


그림 14 Capability 프로토콜

파란색 화살표는 시스템 초기화와 작동 과정에서 요소 간 할당 정보, capability, 메모리 요청 전달을 나타낸다. 전송 과정 중 인터커넥트에 의한 침해를 방지하기 위해 서명 또는 HMAC 이 해당되는 키로 생성되어 같이 전달된다.

가) 중앙 관리 유닛의 capability 생성과 전달 과정 보호

중앙 관리 유닛이 capability를 초기화하기 위해서는 메모리 풀 관리자로부터 할당 관리 정보를 수신해야 한다. 이 정보는 무결성 보장을 위해 capability transference 키를 사용해 계산한 HMAC 값과 함께 전달된다.

최종 완성된 capability를 인터커넥트 로직에 전송할 때 신뢰할 수 없는 인터커넥트가 관여한다. 하지만 생성된 capability는 중앙 관리 유닛이 소유한 capability sealing 키로 서명되기 때문에 인터커넥트가 중간에 이를 변조하는 행위는 불가능하다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

또한, 부채널 공격은 고려하지 않기 때문에 할당 정보에 대한 무결성을 보장하되 기밀성을 보장할 필요는 없다. 따라서 실제 저장된 데이터가 아닌 메모리 사용량에 관한 정보만 가지고 있는 할당 관리 정보 및 capability는 암호화 없는 HMAC/서명만으로 보호된다.

나) 메모리 접근시 라우팅 무결성 보호

인터커넥트가 전송되는 데이터를 올바르지 않은 원격 메모리로 라우팅하려는 공격이 가능하다. 인터커넥트 로직의 검사를 통과하더라도 악의적인 인터커넥트는 전송 중인 데이터의 라우팅 정보를 수정함으로써 컴퓨트 노드/가속기의 오동작을 유도할 수 있다 (예를 들어, 컴퓨트 노드 A의 메모리 접근을 컴퓨트 노드 B에 할당된 원격 메모리로 라우팅할 수 있다).

그러나 인터커넥트 로직은 접근제어를 통과한 요청에 capability transference 키로 계산된 HMAC 값을 추가한다. 따라서, 해당 키가 없는 인터커넥트가 라우팅 정보를 수정할 경우 메모리 풀 매니저에 데이터가 도착하는 시점에 무결성이 침해되었음을 확인할 수 있다. 따라서 인터커넥트는 라우팅 정보를 변조해 컴퓨트 노드/가속기가 비할당 원격 메모리 영역으로 접근하도록 유도할 수 없다.

메모리 풀 매니저에서 capability transference 키에 의한 검사가 실패할 경우 (즉, 인터커넥트가 라우팅 정보를 변조했을 경우), 컴퓨트 노드/가속기는 자신이 올바르게 할당된 영역에 접근하려 했음에도 불구하고 원격 메모리를 사용하지 못하게 되는 문제가 발생한다. 이는 가용성의 침해가 발생한 상황이며, 본 연구에서는 이에 대해 다루지 않는다. 이러한 상황은 인터커넥트가 악의적일 경우 단순히 데이터를 전달하지 않는 방식으로 서비스를 거부하는 것과 동일하게 간주할 수 있다.

다) 메모리 접근 요청의 재생 공격 방어

인터커넥트가 전송되는 메모리 요청을 그대로 저장해 두었다 추후 원격 메모리에 그대로 복사된 요청을 다시 전송해 컴퓨트 노드/가속기의 오동작을 유도할 수 있다. 예를 들어, 인터커넥트가 컴퓨트 노드 A의 메모리 쓰기 요청 A-1 을 저장해 두었다 이후 해당 메모리에 잘못된 값을 저장하기 위해 요청을 리플레이 할 수 있다.

하지만 인터커넥트 로직과 메모리 풀은 순서 번호 윈도우를 통해 송수신된 요청의 정보를 기억한다. 따라서 중복된 요청은 순서 번호 윈도우에 의해 처리가 거부된다. 또한, 모든 메모리 요청의 순서 번호 필드는 capability transference 키로 HMAC을 계산할 때 포함된다. 그러므로 인터커넥트는 자신이 사전에 저장한 메모리 요청의 순서 번호를 임의로 바꿔 윈도우 내의 아직 미사용된 순서 번호를 쓰는 새로운 요청인 것처럼 위장시킬 수도 없다.

4) Secure Offloading의 설계

(1) 원격 인증과 키 교환

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

원격 인증을 통해 메모리 풀 매니저는 하드웨어 가속기의 진위성을 검증할 수 있다. 이는 Certificate Authority(CA)를 통해 장치의 인증서를 검증하는 것으로 이루어진다. 먼저 메모리 풀 매니저는 가속기의 인증서를 얻는다. 가속기는 키 저장소 내부의 root endorsement key(EK)에서 인증 공개키 및 개인키 쌍을 생성한다. 또한 가속기는 endorsement key를 통해 공개 인증 키에 서명하고, 이를 메모리 풀 매니저가 접근할 수 있는 메모리 버퍼에 쓴다. 메모리 풀 매니저는 가속기의 CA(제조사)를 통해 인증서를 검증한다. 공개키는 검증된 인증서에서 추출하고 키 교환에 사용한다.

원격 인증 후, 메모리 풀 매니저는 커맨드와 파라미터를 보낼 암호화된 채널을 생성하기 위해 대칭 세션 키를 생성하고 가속기와 공유한다. 세션 키는 공개 인증 키로 암호화되어 가속기로 보내진다. 우리는 통신 채널에 진위성과 무결성을 주는 AES-GCM과 같은 입증된 암호화 기법을 사용하고, 리플레이(replaying)에 저항성을 갖추기 위해 단조 증가하는 카운터를 사용한다. 또한 부채널(side-channel)을 방지하기 위해 채널에서는 고정된 통신 메시지(fixed communication message)를 이용한다.

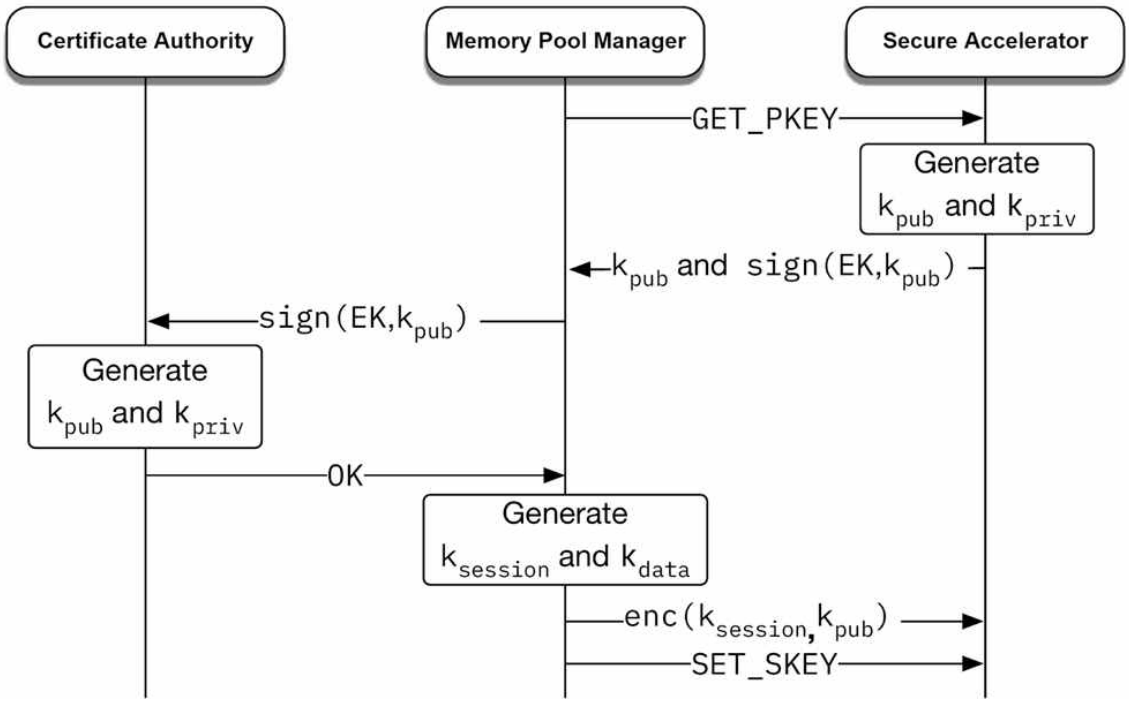


그림 15 원격 인증과 키 교환 흐름도

(2) 안전한 가속을 위한 하드웨어 교체

가속기가 안전한 가속을 지원하기 위해 하드웨어 교체가 필요하다. 먼저, 키 저장소는 root endorsement key(EK)를 저장하고 있다. 이 root endorsement key는 가속기의 진위성을 증명하는 신뢰점(root of trust)의 역할을 한다. 또한 GPU에 암호 확장을 도입했다. 먼저, 암호화와 복호화를 가속하기 위한 AES 엔진을 추가한다. 이는 데이터 전송에서 소프트웨어 기반 AES 암호화가 높은 오버헤드를 보이기 때문이다. 둘째, 키 교환 및 키 파생(key derivation)을 위한 암호 기본요

소(cryptographic primitive)이다. 이는 원격 인증에 사용되는 인증 키 쌍을 생성하기 위해서이다. 또한 커널 오프로딩 및 다른 프로토콜의 로직은 읽기 전용 메모리(Read-only memory, ROM)에 저장되어 있다.

(3) 중앙 관리 유닛을 이용한 접근제어

중앙 메모리 풀 매니저는 시스템의 안전한 가속 장치와 이들의 세션 키를 추적한다. 사용자는 메모리 풀 매니저를 통해 가속기에 접근을 요청한다. 그 후, 사용자는 메모리 풀 매니저를 가속기 사용을 위한 프록시로 사용한다. 사용자와 메모리 풀 매니저 간의 암호화된 채널이 이미 존재하므로, 사용자는 가속 장치와의 안전한 통신 채널을 만들 필요가 없다.

(4) 커널과 데이터 오프로딩

사용자는 가속 커널을 암호화해 메모리 풀 매니저에게 보낸다. 메모리 풀 매니저는 커널을 복호화하고, 목표 가속기의 세션 키로 다시 암호화해 가속기에 오프로드한다. 오프로딩 후, 메모리 풀 매니저는 장치에 측정을 요청해 가속기의 무결성을 검증한다. 측정값은 가속기가 사용하는 코드의 해시 체크섬으로, 온디바이스 메모리에 저장된다. 장치는 측정값을 암호화해 메모리 풀 매니저에게 보낸다. 메모리 풀 매니저는 커널이 장치 메모리에 올바르게 로드되었는지 검증하기 위해 앞서 계산된 커널의 체크섬으로 측정값을 검증한다.

그 후, 사용자는 가속기에 사용자 메모리에 접근할 수 있는 capability를 위임한다. 이는 사용자가 메모리 풀 매니저에게 다른 요청을 하면 수행된다. 그리고 메모리 풀 매니저는 사용자의 메모리에 접근하기 위한 권한을 나타내는 capability transference key를 생성해 가속기로 보낸다. 가속기는 이 키를 사용해 사용자의 메모리에 접근한다. 가속기는 사용자의 데이터를 온칩(on-chip) 메모리에 복사하고, 데이터를 복호화하고, 컴퓨팅을 수행하기 위한 DMA 요청을 수행한다.

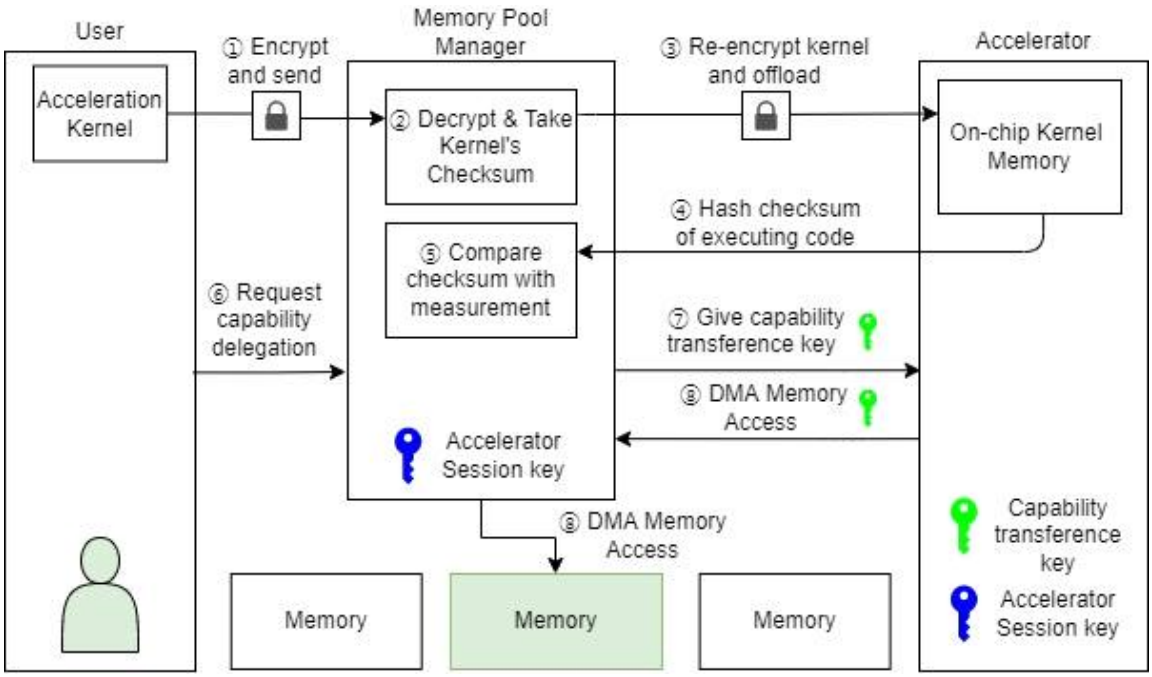


그림 16 커널과 데이터 오프로딩 흐름도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-2-4. 구현 개요

1) Capability 기반 접근제어의 구현

Capability 기반의 접근제어는 본 과제의 1년차 수행 과정에서 개발된 개별분리 시스템 시뮬레이터를 기반으로 구현되었다. 개별분리 시스템 시뮬레이터는 위에서 설명한 설계 가정 사항을 만족하도록 구현되었으며, cycle-accurate 한 동작을 통해 설계된 접근제어 메커니즘이 어느 정도의 성능 저하를 유발하는지 측정할 수 있다.

(1) 중앙 관리 유닛 구현

가) 시스템 초기화 시 Capability 생성

개별분리 시스템 시뮬레이터는 시스템 초기화 시점에 각 컴퓨트 노드와 원격 메모리 풀을 워크그룹(workgroup) 이라는 단위로 묶어서 할당한다. 이 과정에서, 메모리 용량 등 할당을 위한 정보는 시뮬레이터가 설정 파일로부터 읽어들인 이후, 시뮬레이터의 메모리 풀 관리자 클래스에 저장된다. 현재 시뮬레이터의 구현은 원격 메모리 풀의 동일한 영역을 서로 다른 두 개 이상의 컴퓨트 노드나 가속기가 공유하는 것을 지원하지 않는다.

시뮬레이터 상에서, 중앙 관리 유닛과 다른 시스템 구성 요소는 상호 작용할 수 있는 인터페이스가 구현되어 있다. 중앙 관리 유닛은 capability 할당 과정을 시뮬레이션하기 위해, 메모리 풀 관리자의 인터페이스 함수를 통해 사용할 수 있는 공간을 질의한다. 이는 의사 코드로 나타내면 다음과 같다.

```
mem_pool_manager->node_initialization_alloc(cid, size)
// cid: 저장 공간을 할당받을 컴퓨트 노드의 Component ID
// size: 할당받고자 하는 용량
```

의사 코드 1

메모리 풀 관리자는 가용한 공간이 있는지 확인한 후, 다음과 같은 자료구조로 정보를 반환한다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
struct NodeMemPoolAlloc {
    Collection<PermissionElement> perm_list, // 1개 이상의 영역 권한 디스크립터
    HMAC mac_checksum, // Capability transference key로 계산된 HMAC 값
}

struct PermissionElement {
    Addr address_begin, // 영역 주소 시작
    Addr address_end, // 영역 주소 끝
    Permission permission, // 접근 권한 비트
}
```

의사 코드 2

중앙 관리 유닛은 수신한 데이터의 mac_checksum 필드를 capability transference key로 확인한 이후, 문제가 없다면 다음 단계를 진행한다.
실제 시스템에서는 인터커넥트를 신뢰할 수 없으므로, 만약 인터커넥트가 전송되는 할당 정보를 변경했다면 이 과정에서 HMAC 검사 실패와 함께 시스템 초기화가 중단되는 상황이 발생할 수 있다 (공격 탐지). 그러나, 시뮬레이터 상에서 이러한 상황을 만드는 방법은 의도적으로 전송 과정 중간에 변조를 위한 코드를 삽입하는 것뿐이므로, 별도로 시험을 위해 설정하지 않는 이상 시뮬레이션 상에서 검사가 실패하는 경우는 발생하지 않는다.

다음 단계에서 중앙 관리 유닛은 자신의 capability sealing key로 할당 정보를 서명한다. 최종적으로 생성된 capability는 의사 코드로 나타내었을 때 다음과 같다.

```
struct MemPoolAllocCapability {
    NodeMemPoolAlloc allocation, // 위에서 받은 NodeMemPoolAlloc 정보
    Sign sign, // 중앙 관리 유닛의 capability sealing key로 서명된 값
}
```

의사 코드 3

나) 생성된 Capability를 인터커넥트 로직에 전달

생성된 capability는 각 컴퓨트 노드에 연결된 인터커넥트 로직에 전달된다. 시뮬레이터 상에서는 다음과 같은 인터페이스 함수로 설정된다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
interconnect_logic->set_capability(cap)
// cap: 위에서 생성된 capability
```

의사 코드 4

다) 시스템 성능 분석을 위한 레이턴시 부여

위 과정은 시뮬레이터 초기화 과정에서 일회성으로 실행된다. 따라서 전체 시스템 성능 측정에 있어 미치는 영향은 극히 미미하므로, 현재 시뮬레이터 구현에서는 별도의 레이턴시를 설정하지 않았다.

(2) 인터커넥트 로직 구현

가) 전달받은 capability의 검증

각 인터커넥트 로직은 전달받은 capability가 중앙 관리 유닛에 의해 생성되었는지를 검증한다. Capability sealing key는 공개키 기반으로 설계되었으므로, 각 인터커넥트 로직은 중앙 관리 유닛의 공개키를 사용해 MemPoolAllocCapability 구조체의 sign 필드를 확인할 수 있다.

실제 시스템에서는 인터커넥트를 신뢰할 수 없으므로, 위 과정에서 검증 실패로 인해 시스템 초기화가 중단되는 경우가 발생할 수 있다. 그러나 시뮬레이터 상에서는 의도적으로 오류를 주입하지 않는 이상 이 과정은 실패하지 않는다.

나) 메모리 패킷 전송시 capability의 권한과 비교

시뮬레이터 상에서 컴퓨트 노드가 원격 메모리 풀에 접근하는 과정은 메모리 패킷 구조체를 만들어서 인터커넥트 로직에 전달하는 것으로 시작한다. 이 과정에서 메모리 패킷의 물리 주소는 MemPoolAllocCapability 구조체의 allocation 필드가 가진 영역 권한 디스크립터 리스트와 비교된다. 만약 리스트 내의 아무런 디스크립터도 메모리 패킷이 접근하려는 주소를 포함하지 않으면, 인터커넥트 로직은 예외를 발생시킨다.

시뮬레이터 상에서는 의도적으로 오류를 발생시키지 않는 이상 항상 비교가 성공하기 때문에, 예외 처리는 시뮬레이션 로그 기록으로 구현되며 별도의 처리(예를 들어, 컴퓨트 노드에 외부 인터럽트 전달 등)는 추가로 수행되지 않는다. 로그는 추후 보안 메커니즘의 성능 평가를 위한 자료로 사용할 수 있다.

NodeMemPoolAlloc에서 1개 이상의 영역 권한 디스크립터는 Vector 등의 자료구조를 통해 저장될 수 있다. 시뮬레이터 상에서는 이를 임의의 자료구조로 정의할 수 있다. 예를 들어, 메모리 패킷이 접근하는 주소의 일부분을 사용해 해당되는 PermissionElement가 바로 인덱싱되는 배열 형태의 구조나, 순차 접근을 통해 해당되는 PermissionElement를 탐색할 수 있는 연결 리스트 형태의 구조 등이 가능하다. (실제 시스템에서는 효율적인 어느 하나의 자료구조로 고정되어 구현되어야 한다.)

다) Capability caching

위에서 PermissionElement를 포함한 자료구조의 종류에 따라 인터커넥트 로직

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

에서 하나의 메모리 패킷을 검사하는 데 걸리는 시간은 달라질 수 있다. 이때, 자주 사용되는 PermissionElement를 인터커넥트 로직에서 캐싱한다면 검사 시간이 더 줄어들 수 있다. 이를 위해서 현재 시뮬레이터 구현에서는 최근 사용된 PermissionElement를 추적한다. 추적할 PermissionElement의 개수와 cache eviction policy는 시뮬레이터 설정 파일에서 지정할 수 있다.

라) HMAC이 포함된 패킷 전달

Capability 검사를 통과한 메모리 패킷은 인터커넥트 로직에 의해서 인터커넥트를 거쳐 메모리 풀 매니저로 전달된다. 이 과정에서, 신뢰할 수 없는 인터커넥트에 의해 라우팅 정보가 변조되는 것을 방지하기 위해 메모리 패킷의 헤더를 포함한 데이터를 대상으로 capability transference 키를 사용해 HMAC을 계산한 이후, 메모리 패킷에 포함시킨다.

```
struct MemPacketWithHMAC {
    Packet original_packet,  // 컴퓨트 노드에서 생성한 원래 메모리 패킷
    HMAC hmac, // 위 패킷을 대상으로 생성된 HMAC
}
```

의사 코드 5

이때 HMAC의 계산에는 반드시 패킷의 순서 번호, 패킷을 생성한 컴퓨트 노드의 Component ID, 패킷을 받을 원격 메모리 풀의 Component ID가 포함된다.
악의적인 인터커넥트는 전송되는 패킷을 자신이 복사해 두었다가 이후 그대로 재전송함으로써 원격 메모리 풀의 데이터를 덮어쓰는 등의 재생 공격을 시도할 수 있다 (replay attack). HMAC 계산에 순서 번호를 포함시킴으로써, 신뢰할 수 없는 인터커넥트가 capability transference 키를 알지 못하는 이상 패킷의 재생 공격을 할 수 없도록 방지한다.
송신자와 수신자 Component ID를 포함함으로써, 신뢰할 수 없는 인터커넥트가 라우팅 정보를 변조할 수 없도록 방지한다.

마) 시스템 성능 분석을 위한 레이턴시 부여

현재 시뮬레이터 구현에서는 (3) Capability caching을 제외한 패킷 간의 상호작용이 없다는 가정하에서 레이턴시 계산이 이루어진다 (예를 들어, 여러 개의 패킷이 큐에 저장된 후 동시에 처리되는 것과 같은 일이 이루어지지 않는다). 따라서 각 메모리 패킷에 적용되는 레이턴시는 capability cache hit/miss 여부에 따라서 다른 상숫값을 적용한다. 인터커넥트 로직에서의 cache hit/miss 시의 레이턴시는 각각 시뮬레이션 설정 파일로부터 지정할 수 있다.
Capability cache는 설정 파일에 정의된 추적할 최근 PermissionElement 개수와 eviction policy에 따라 시뮬레이터 상에서 프로그램을 실행하며 상태가 바뀐다. Capability cache에 어느 주소가 로딩되어 있는지는 원격 메모리의 접근 패턴에 따라 달라지므로, 시뮬레이션할 프로그램이 다르다면 같은 주소의 패킷이라도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

hit/miss 레이턴시가 적용되는지가 달라진다.

(3) 메모리 풀 매니저 구현

가) 전송받은 메모리 패킷의 HMAC 검증

메모리 풀 매니저에서 전송받은 메모리 패킷은 신뢰할 수 없는 인터커넥트를 통해 전송되었으므로, 패킷의 헤더 정보에 대하여 capability transference 키를 사용해 HMAC을 검증한 이후 처리된다.

이때, 메모리 패킷에는 재생 공격을 방지하기 위한 순서 번호가 포함된다. 메모리 풀 매니저는 최근 정상 수신한 메모리 패킷의 순서 번호를 수신자 윈도우에서 제거한다. 따라서, 한번 정상 수신된 패킷이 중복으로 도착하면 해당 패킷은 처리하지 않고 무시한다(중복 도착의 원인으로는 인터커넥트의 재생 공격뿐만 아니라 인터커넥트 상의 전송 오류일 수도 있으므로, 시스템 정지와 같은 예외 처리는 적용하지 않는다).

나) 시스템 성능 분석을 위한 레이턴시 부여

메모리 풀 매니저에서는 별도의 capability 검증 없이 메모리 패킷의 HMAC과 순서 번호만 검증하면 되므로, 현재의 가정 (패킷 간의 상호작용 없음) 하에서는 패킷마다 검증에 상수 시간만큼의 레이턴시를 적용한다. 해당 레이턴시는 시뮬레이션 설정 파일로부터 지정할 수 있다.

(4) Capability key storage 구현

실제 시스템에서 capability transference 키와 capability sealing 키는 신뢰할 수 없는 요소와 격리된 영역에 존재한다. 이 경우, 키를 사용한 연산은 신뢰 실행 환경(Trusted Execution Environment, TEE) 또는 하드웨어 보안 모듈 (Hardware Security Module, HSM)과 같은 기술을 사용해 이루어질 수 있다. TEE나 HSM을 사용할 경우 추가적으로 성능 오버헤드가 발생할 수 있다.

시뮬레이터에서는 위 키를 사용하는 함수를 호출할 때마다 메모리 패킷에 상수 시간의 레이턴시를 추가로 부여하는 방식으로 구현하였다. 이는 성능 측정에 사용되는 (시스템 초기화 이후 전달되는) 메모리 패킷의 길이가 고정적이며, 암호 알고리즘 또한 사전 정의된 연산만을 수행한다는 점을 반영했다. 레이턴시와 키는 시뮬레이션 설정 파일로부터 지정할 수 있다.

2) Secure Offloading의 구현

(1) 가속기 구현

메모리에 대해 DMA를 수행할 수 있는 gem5의 DMA 장치로서 가속기를 구현했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
from m5.params import *
from m5.proxy import *
from m5.objects.Device import DmaDevice

class SecureAccelerator(DmaDevice):
    type = 'SecureAccelerator'
    cxx_header = "dev/x86/secure_accelerator.hh"
    ...
```

의사 코드 6

가속기의 로직은 C++로 작성된 gem5의 표준 PCI-E 장치로서 구현됐다. 가속기 메모리의 주소는 gem5 설정을 통해 다른 시스템이 발견할 수 있도록 메모리 버스에 매핑된다. 주소는 메모리 풀 매니저와 제어 레지스터 간의, 가속기에 커맨드를 보내는 통신에 사용되는 DMA 버퍼를 포함한다. 가속기는 이 커맨드를 기반으로 시작시의 원격인증과 같은 함수를 실행한다. 다음의 코드는 시스템 버스에 주소 0xfffef000와 4KB의 크기로 안전한 장치를 붙인다. 이를 통해 시스템의 다른 컴포넌트는 주소 내에서 통신할 수 있게 된다.

```
def makeX86System(mem_mode, numCPUs=1, mdesc=None,
workload=None, Ruby=False,):
        self = System()
        // ….
self.secure_acc = SecureAccelerator(pio_addr=0xFFFEF000,
pio_size="4kB")
self.secure_acc.dma = self.iobus.slave
self.secure_acc.pio = self.iobus.master
```

의사 코드 7

장치 로직은 C++로 작성됐다. 암호 함수는 OpenSSL을 사용해 내부 로직을 구현하고 확인된 오버헤드를 시뮬레이션에 도입했다. 키는 가속기 시뮬레이터의 private 변수로 추가했다. 또한 안전한 가속기는 사용자가 제어 커맨드를 전송하기 위한 제어 레지스터와 커맨드를 정의한다.

```
class SecureAccelerator : public DmaDevice {
private:
  RSA* privateKey;
  RSA* publicKey;
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
  RSA* endorsementKey;
  AES* sessionKey;
...
protected:
    enum ControlRegisters {
      CommandReg = 0x000,
      DmaTransferSizeReg = 0x008,
      InterruptStatusReg = 0x010,
      DmaBufferAddrReg = 0x018
    };
    enum ControlCommands {
      NoCommand = 0x0,
      RemoteAttest = 0x1,
      ExchangeKey = 0x2,

       Measurement = 0x3,

  };
}
```

의사 코드 8

각 커맨드를 수신하면, 커맨드를 구현한 함수를 실행하고, 장치 실행을 시뮬레이션하기 위해 딜레이를 추가한다. 예를 들어, 커널 측정값을 가져와 값으로 측정값에 서명하는 측정 함수는 다음과 같다.

```
SecureAccelerator::CalculateMeasurement(RSA* r) {
  unsigned char *hash = (unsigned char*)malloc(SHA256_DIGEST_LENGTH);
  SHA256_CTX sha256;
  SHA256_Init(&sha256);
  SHA256_Update(&sha256, LOADED_KERNEL, KERNEL_SIZE);
  SHA256_Final(hash, &sha256);
  std::string hash_str = (char*)hash;
  std::string signedHash = signMessage(hash_str, r);
  return hash_str + std::string("₩x00") + signedHash;
}
```

의사 코드 9

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-3. 3년차(2022년)의 연구 범위

본 과제의 3년차 연구 범위와 관련 기관은 다음과 같다.

2-3-1. 개발 목표

- **KAIST와 CISPA는 서로 협력하여 개별분리 시스템의 Access control과 프로토콜의 안정성 검증, 프로토타입 전체 성능 측정, 최종 평가, 미비점 보완 및 구현하지 못한 다른 보안요소 구성 방법 제안한다.**

  다양한 공격상황에 대비해 1, 2차년도 설계하고 적용한 보호 기법에 대한 실제 효과를 검증한다. CVE를 활용해 실제 공격시나리오를 설계하고 각 공격이 효과적으로 방어 가능한지 확인한다. Log와 Debugging data를 활용해 측정할 예정이다. 또한, 실제로 구현된 전체적인 프로토타입의 성능을 측정하기 위해 Test Application을 선정하고, 해당 Application의 통합 성능 저하율, 메모리 접근 지연 시간 감소율, 인터커넥션 대역폭 감소율, 비인가 접근 차단율로 측정할 예정이다. 최종적으로, 기관별 연구 결과물을 취합하고 평가 지표를 활용해 성능을 측정 후 전체적으로 프로토타입을 평가한다. 이후, 성능 측정 과정에서 발생한 미비점을 보완하고, 추가 공격시나리오 도출 및 새로운 보호 기법들을 제안한다.

- **성균관대학교와 CISPA는 서로 협력하여 개별분리 시스템의 CPU와 가속기간의 안전한 오프로딩 기술을 개발하고 최종 프로토타입을 구현하여 설계 내용을 검증한다.**

  본 연구에서는 성립된 암호화된 채널을 통해 CPU TEE에서 가속기에서 실행될 코드와 처리해야하는 데이터를 CPU 내에서 보호되는 신뢰 실행영역에서 암호화하여 원격인증된 가속기에 안전하게 오프로딩하는 프로토콜을 개발한다. 구체적으로는 CPU TEE에서 보내는 가속기 명령(예, GPU Command)과 데이터를 가속기 풀 컨트롤러가 복호화하여 해석하고 가속기에 전달하여 실행시킨 후, 결과를 전달받아 다시 암호화하여 CPU TEE에 반환하는 일련의 과정의 구현이 필요하다. 구현된 오프로딩 프로토콜을 여러 가지 공격 모델을 가정하고 가속기로 전달되는 데이터의 탈취 등이 가능한지를 분석한다. 분석 예정인 공격 모델로는 ① 가속기와 통신하는 프로세서 node 내부의 커널이 감염되어 시스템콜 수행을 통해 진행되는 I/O를 Eavesdropping 하는 공격, ② 다른 CPU 노드가 Man-in-the-middle 공격 등을 통해 데이터를 가로채는 공격 등 이다. 시뮬레이터 상으로 구현되었던 전체적인 아키텍처 (KAIST,CISPA)에 가속기 풀 컨트롤러를 소프트웨어 및 하드웨어적으로 구현 한 뒤 실제 GPU를 가속기로 사용하여 시뮬레이션의 내용을 검증한다. 다른 연구기관과의 호환성 및 성능 측면에서의 검증을 진행한다.

2-3-2. 검증 개요

본 연구팀은 1년차에 개발한 시뮬레이터의 다양한 파라미터 값과 설정 등을 이용하여 성능을 측정했다. 이러한 실험으로 개별분리 시스템을 설계하고 구현할 때 성능 하락

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

원인과 최소한의 성능 요구사항을 파악하는 것에 매우 큰 도움을 줄 수 있다. 본 연구팀은 실험을 5개의 세부 실험으로 나누어 진행했다. 실험1은 개별분리 시스템이 기존 머신 기반의 시스템과 어떠한 성능적 특성 및 차이를 가지는 지에 대한 실험이고, 실험2는 컴퓨트 노트가 점점 많아질 때 시스템 전체의 어떤 영향을 가지는 지에 대한 실험이다. 실험3은 메모리 풀 매니저에서 패킷을 처리하는 컴포넌트가 시스템의 병목현상을 유발시키고 이것을 여러개로 늘렸을 때 이러한 현상이 개선됨을 보여준다. 실험4은 메모리 컨트롤러와 모듈의 갯수가 증가함에 따라 전체 시스템의 성능 변화를 보여준다. 마지막으로 실험5는 인터커넥트의 소재에 따라 지연시간이 달라지는데 이것이 전세 시스템에 어떤 영향을 미치는 지에 대한 실험이다.

2-3-3. 검증 방법

본 연구팀은 본 과제에서 개발한 시뮬레이터를 이용해 표 2와 같은 파라미터 값과 설정으로 검증했다.

컴포넌트	파라미터 항목	파라미터 값
인터커넥트	링크 폭	288 bits/phit
	링크 지연시간	6.8 ns/packet
	전송 지연시간	10 ns/packet
	스위치 처리 지연시간	40 ns/packet
컴퓨트 노드	컴퓨트 노드 갯수	1
	컴퓨트 노드 당 코어 갯수	1
	L1, L2, L3 캐쉬 크기	32KB, 256KB, 2MB
	CPU 종류	Derive03CPU
	명령어 집합 아키텍처	x86
	주소 변환 지연시간	60 ns/packet
메모리 풀 매니저	패킷 처리 큐 갯수	1
	라우팅 지연시간	40 ns/packet
	프론트엔드 지연시간	1 ns/packet
	포워드 지연시간	0.5 ns/packet
	응답 지연시간	1 ns/packet
	헤더 지연시간	0.5 ns/packet
메모리 컨트롤러	메모리 컨트롤러 갯수	2
	쓰기 버퍼 임계값	50 - 85%
	프론트엔드 지연시간	10 ns/packet
	백엔드 지연시간	10 ns/packet
메모리 모듈	메모리 컨트폴러 당 메모리 모듈 갯수	2
	메모리 모듈 크기	256 MB

표 2 실험 파라미터 항목과 설정값

각 컴퓨트 노드는 x86 아키텍처와 L3 캐시를 갖춘 Derive03CPU 모델로 설정하였다. 이러한 컴퓨트 노드는 메모리 풀로부터 메모리를 제공받는다. 모든 실험은 2.2 GHz로 동작하는 2개의 AMD EPYC 7601 프로세서와 512 GB의 램, 그리고 우분투 18.04.5 (커

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

널 버전 4.15.0)의 환경에서 실험했다.

본 연구팀은 성능 측정을 위한 벤치마크 도구로써 SPEC CPU2006과 STREAM 벤치마크를 사용했다. 두 벤치마크 도구는 컴퓨터 아키텍처 분야에서 널리 사용되고 있는 도구이며, SPEC CPU2006은 실생활 어플리케이션의 대표로써 선택했고, STREAM 벤치마크는 그 특성상 매우 간단한 연산으로 동작하기 때문에 컴퓨트 노드의 규모를 증가시켜 실험하기 위해 선택했다. STREAM 벤치마크는 copy, scale, add, triad의 총 4가지 연산을 모두 실행 후 평균값으로 출력한다. 따라서 우리는 각 연산의 결과값을 얻기 위해 소스코드를 수정하여 한 가지 연산만 하는 각개의 어플리케이션으로 분리했다.

2-3-4. 종합적 안전성 및 보안성 검증 결과

- **Access control과 프로토콜의 안전성 검증**

1, 2차년도에 설계한 접근 제어 기법과 프로토콜의 안전성을 검증하기 위해서, 개별분리 시스템에서 발생할 수 있는 여러 보안 위협을 모델링하고 시뮬레이터 상에서 공격이 발생했을 때 방어가 적절한 방식으로 이루어지는지 검증했다.

아래는 3차년도 연구를 통해 파악된 개별분리 시스템에 대한 공격들이다. 각 공격의 동작 원리과 기구현된 보안 기능들이 이를 방지하는 원리에 대해 분석함으로써 보안성을 검증하였다.

보안 목적	공격	관련 보안 기능
기밀성	A1. 중간자 공격 (MiTM)	노드내 키 저장소, 노드간 원격인증 (Remote Attestation)
	A2. 패킷 도청	노드간 대칭키 암호 (AES-GCM)
무결성	A3. 패킷 헤더 변조	데이터링크 계층 HMAC
	A4. 패킷 데이터 변조	노드간 대칭키 암호 (AES-GCM)
	A5. 패킷 재전송 공격	데이터링크 계층 HMAC
기타	A6. 혼동된 대리인 공격	Capability기반 접근제어

표 3 보안성 검증 목록

본 개별분리 시스템에서는 Data Link Layer에서의 암호화를 통해 패킷에 대한 기밀성과 무결성을 보장한다. 또한, 중간자 공격(Man-in-the-Middle attack) 을 막기 위해 노드 간 원격 인증을 수행하며, 접근 제어의 경우 capability 기반의 방식을 사용해 혼동된 대리인 공격 (confused deputy) 을 방지한다. 아래는 각 공격과 방어 기법의 검증에 대한 내용이다.

- **노드간 원격 인증 및 안전한 통신 보장 (A1)**

전송되는 데이터를 보호하기 위한 세션 키를 올바르게 생성하기 위해서는 우선 노드간의 원격 인증 (remote attestation)을 통해 통신 상대의 진위성을 입증하는 과정이 선행되어야 한다. 이를 위해 가속기 등에는 Endorsement Key (EK) 가 안전한 내부 영역

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

에 저장되어 있어 세션 키 생성시에 원격 인증을 위해 사용된다. 만약 노드 간 통신 수행시에 두 노드의 중간에 위치한 공격자가 통신 대상으로 가장하는 방식의 공격 (Man-in-the-Middle attack) 을 시도한다고 하더라도, 중간 공격자는 EK를 얻을 수 없으므로 공격을 수행할 수 없다.
시뮬레이션으로 위 내용을 검증하기 위해, 가속기 노드와 메모리 풀 노드 사이에 임의의 노드를 삽입하고 중간자 공격을 실행하게 했다. 그러나 공개키 검증 과정에서 중간자 노드는 올바른 서명값을 생성할 수 없으므로, 공격이 실패하며 성공적으로 방어를 수행했다.

- **가속기 시큐어 오프로딩 (Secure offloading) (A2)**

컴퓨트 노드와 가속기간의 안전한 오프로딩을 위해 전달되는 가속 커널은 세션 키로 암호화된다. 따라서 신뢰할 수 없는 인터커넥트에 의해 도청이나 변조와 같은 공격이 불가능하며, 단조 증가하는 순서 번호의 포함으로 재생 공격 또한 방지된다.
시뮬레이터 상에서 상기된 공격과 방어를 검증하기 위해, 인터커넥트에서 전송중인 가속 커널을 추출하도록 했다. 그러나 전송되는 데이터가 암호화 되어 있으므로, 실제 사용이 불가능한 형태로만 가속 커널을 얻을 수 있었다. 또한, 재생 공격을 시연하기 위해 인터커넥트에서 추출한 가속 커널을 그대로 가속기에 다시 전송한 경우 복호화에 실패하면서 결과적으로 공격이 성공적으로 방어됨을 확인할 수 있었다.

- **물리계층 무결성 보장 및 재전송 방지 (A3, A4, A5)**

위협 모델에서 인터커넥트는 신뢰할 수 없기 때문에 전송되는 데이터의 변조가 일어날 수 있다. 이를 방지하기 위해 컴퓨트 노드와 메모리 풀 간 전송되는 데이터의 무결성은 인터커넥트 로직에 의해 보호된다. 또한, 인터커넥트가 전송되는 데이터를 저장한 이후 다시 전송하는 재생 공격을 방지하기 위한 방법 중 하나로 단조 증가하는 순서 번호가 사용된다.
시뮬레이터 상에서 상기된 공격과 방어를 검증하기 위해, 특정 컴퓨트 노드에서 생성된 패킷을 대상으로 인터커넥트가 패킷의 라우팅 정보를 변경하도록 했다. 그러나 인터커넥트가 변조한 내용에 대해 올바른 검증값을 생성할 수 없으므로, 결과적으로는 메모리 풀 매니저에서 패킷을 수신할 때 예외가 발생하면서 성공적으로 방어를 수행했다. 또한 재생 공격도 순서 번호 검사 과정에서 예외를 발생시키며 방어에 성공했다.
한 가지 유의할 점은 인터커넥트가 가용성을 침해하는 경우에 대해서는 고려되지 않았다는 것이다. 하지만 인터커넥트가 단순히 패킷을 손실시키는 방식으로 서비스 거부 공격을 수행할 수 있으므로, 가용성의 보장은 본 연구의 주제에서 벗어난다.

- **Capability기반 접근 제어를 통한 비인가 접근 차단 (A6)**

시뮬레이션 시작 시 각 컴퓨트 노드는 원격 메모리를 할당받음과 동시에 자신의 메모리 영역에 접근하기 위한 capability를 할당받는다. 이 capability는 인터커넥트 로직에 저장되어 모든 메모리 접근시마다 비교되는 방식으로 접근 제어가 수행된다. 이러한 접근 제어를 우회해 자신의 영역이 아닌 메모리에 접근하기 위해서는 capability를 위조하거나, 인터커넥트 로직을 거치지 않고 메모리에 접근할 수 있는 경로가 있어야 한

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다. 전자의 경우, 중앙 관리 유닛만이 capability sealing 키로 capability를 생성할 수 있으므로 불가능하며, 후자의 경우 하드웨어적인 침해로만 경로를 바꿀 수 있다.
시뮬레이터 상에서 상기된 공격과 방어를 검증하기 위해, 컴퓨트 노드에서 임의로 생성된 capability 를 인터커넥트 로직에 전달했다. 인터커넥트 로직은 capability 검증 과정 중 올바르지 않은 sealing 예외를 발생시키며 성공적으로 방어를 수행하였다. 또한, capability 에 명시되지 않은 메모리에 접근을 시도하는 경우도 마찬가지로 인터커넥트 로직에서 예외를 발생시키며 정상적으로 접근 제어를 수행하였다.

2-3-5. 프로토타입 전체 성능 측정 및 평가

- **실험 1: 머신 기반 시스템과 개별분리 시스템의 성능 비교**

그림 16은 SPEC CPU2006으로 측정한 기존 머신 기반의 시스템과 개별분리 시스템의 성능 차이를 보여준다. 이 실험으로 개별분리 시스템에서는 캐시 미스율이 클 수록 기존 머신 기반의 시스템 보다 훨씬 큰 성능하락이 생김을 알 수 있다. 최악의 경우 gobmk 어플리케이션은 7.85배의 성능하락과 2.93%의 캐시 미스를 발생시켰고, 지역성(locality)이 높은 hmer와 libquantum은 최소한의 성능하락으로 그쳤다.

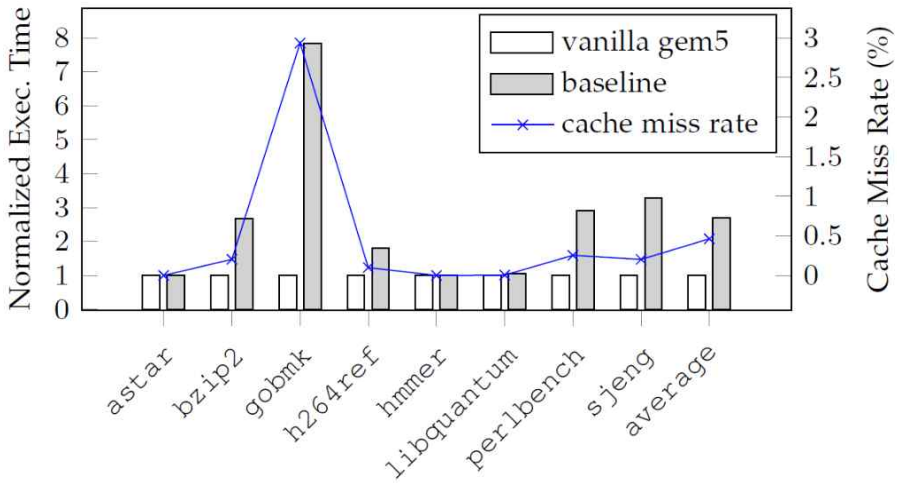


그림 17 기존 머신 기반의 시스템과 개별분리 시스템과의 성능 차이

실험1의 결과는 메모리 접근 지연시간이 개별분리 시스템에서 가장 큰 성능 하락의 원인임을 보여준다. 캐시 미스가 발생함에 따라 원격에 있는 메모리에 접근하는데 많은 지연시간이 생기기 때문이다. 메모리 접근 지연시간은 많은 요소로 인해 발생되는데, 예르들어 주소 변환 지연시간, 스위치 처리 시연시간 등이 있다. 이러한 지연시간을 최대한 극복하는 것이 개별분리 시스템이 실용화될 수 있는 관건이다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- **실험 2: 컴퓨트 노트 갯수와 전체 시스템의 성능 관계**

그림 17은 SPECint2006 벤치마크의 결과 중 컴퓨트 노드 갯수가 1, 2, 4, 8개로 증가할 때 컴퓨트 노드 1개에서 측정된 성능 하락을 보여준다. 컴퓨트 노드 서로가 공유된 네트워크를 사용하여 메모리에 접근하는 만큼, 각각의 컴퓨트 노드가 성능에 간섭이 생김을 알 수 있다. 특히 컴퓨트 노드 갯수가 가장 많은, 8개 컴퓨트 노트 실험에서는 gobmk 벤치마크에서 최대 8.59%의 성능 하락이 발생했다. 또한 그림 16의 실험과 같은 패턴의 결과가 발생함을 확인했다. 이러한 멀티 컴퓨트 노드 환경에서 패킷을 처리하는 성능에 따라 실험 3에서 확인했다.

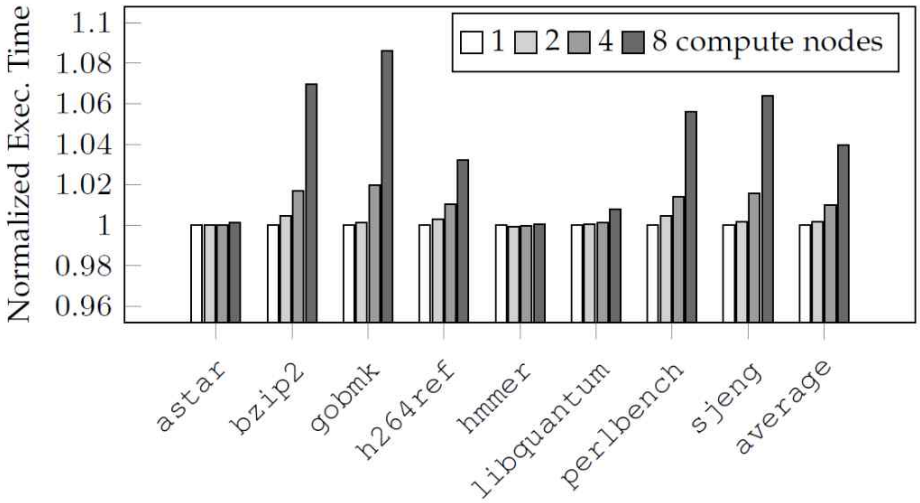


그림 18 멀티 컴퓨트 노드 환경에서 컴퓨트 노드 1개당 성능 하락

- **실험 3: 멀티 컴퓨트 노드 환경에서 패킷 처리 용량과 성능의 상관관계**

본 연구팀은 실험 2에서 멀티 컴퓨트 노드 환경에서 컴퓨트 노드가 많이질 수록 성능에 간섭이 생김을 확인했다. 이것의 원인을 파악하기 위해서 본 연구팀은 패킷 처리 큐의 갯수를 늘렸다. 패킷 처리 큐는 메모리 풀 매니저가 인터페이스로부터 패브릿 패킷을 수신하고 패킷 멀티플렉서로 보내기전에 패킷을 보관하는 컴포넌트이다. 패킷 처리 큐가 패킷을 보내지 못하고 계속 가지고 있는 현상이라고 가설을 세웠다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

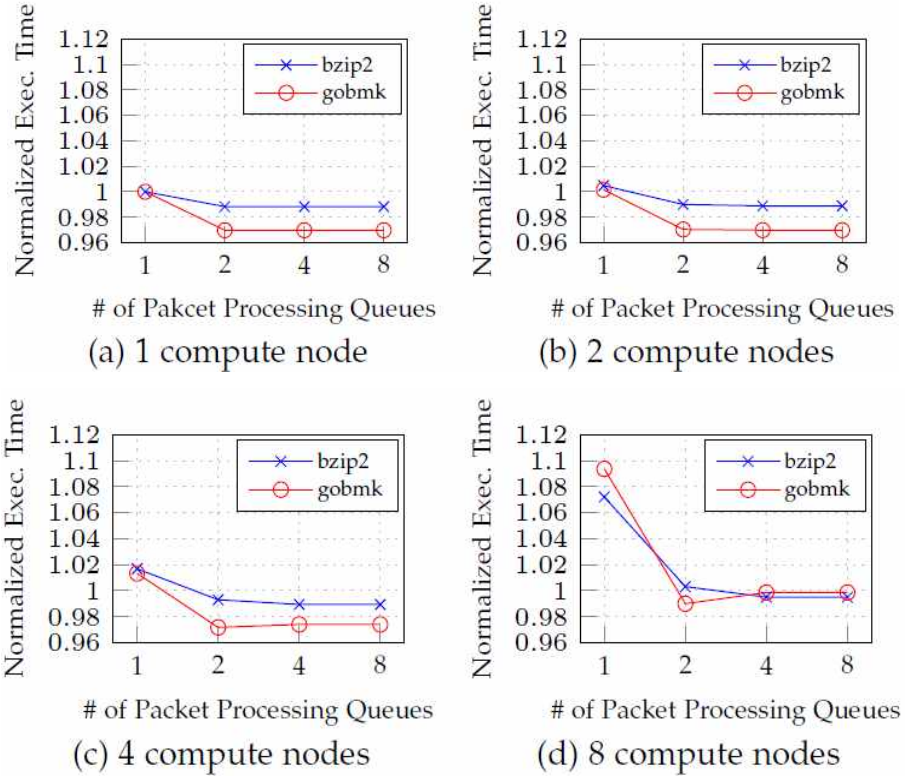


그림 19 SPECint2006 벤치마크 결과 중 패킷 처리 큐의 갯수 변경과 병목현상 개선 현상

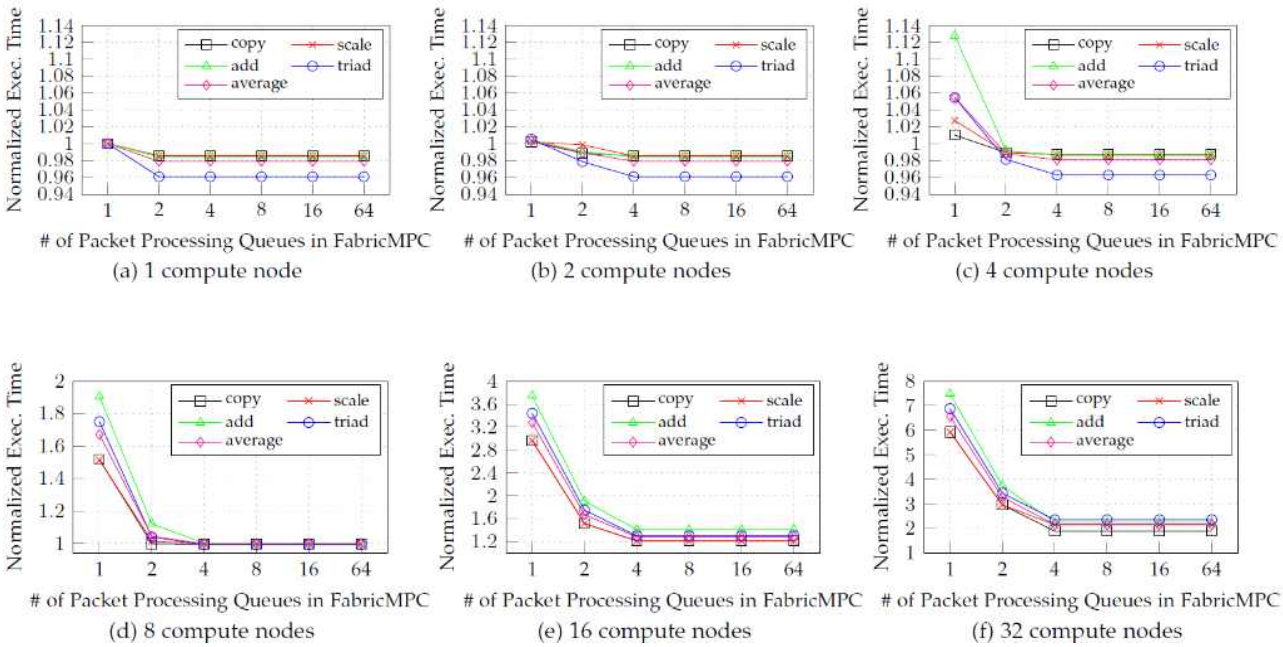


그림 20 STREAM 벤치마크 결과 중 패킷 처리 큐의 갯수 변경과 병목현상 개선 현상

그림 18과 그림 19는 패킷 처리 큐의 갯수와 병목현상의 상관관계를 보여준다. 패킷 처리 큐의 갯수가 많이질 수록 병목현상이 크게 개선됨을 알 수 있다. 특히 컴퓨트 노드가 많아질 수록, 지역성이 떨어져 원격 메모리 접근이 많을 수록 개선효과가 두드러진다. STREAM 벤치마크에서 copy와 scale은 배열 2개를 이용하고 add와 triad는 배열 3개를 이용하여 덧셈, 곱셈 등 간단한 연산을 한다. 이러한 벤치마크 특성은 실험 결과에도 잘 드러난다. 배열 3개를 이용한 add, triad는 배열 2개를 이용한 copy, scale 보

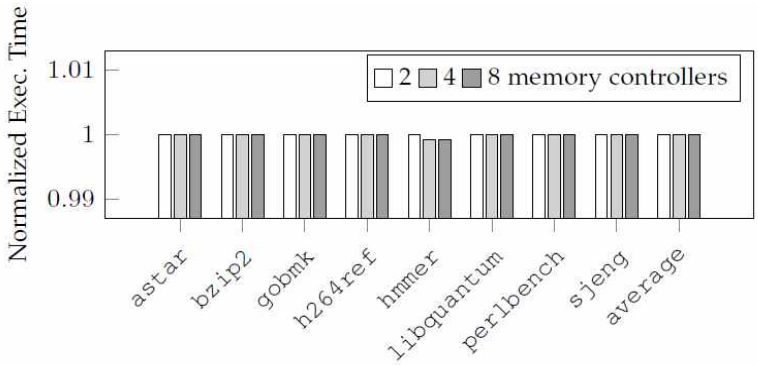
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

다 실행시간이 더 오래걸리는데, 원격 메모리 접근 횟수도 많아서 병목현상이 더 크게 나타난다. 이러한 병목현상을 줄이기 위해 패킷 처리 큐의 갯수를 증가시키면 그 효과가 더 크게 나타남을 알 수 있다.

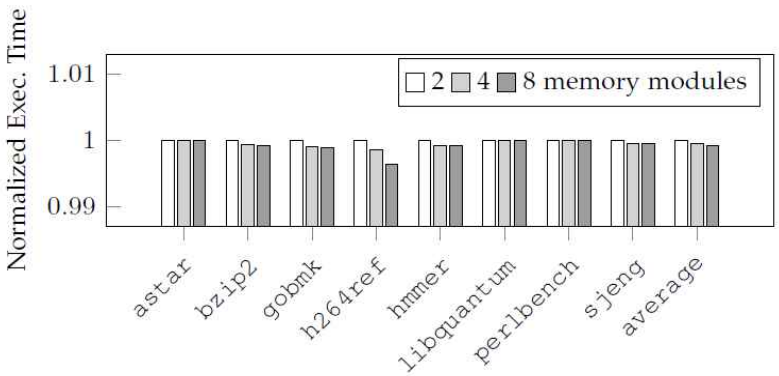
또한, STREAM 벤치마크 실험에서 시뮬레이터의 컴퓨트 노드를 32개로 설정하여 시스템의 스케일을 매우 높힐 수 있음을 확인했다. 많아진 패킷 처리 큐는 병렬처리 능력을 향상시켜 메모리 풀 매니저가 병목현상의 원인이 되는 것을 막아준다. 그러나 일정이상의 갯수를 충족하면 그 병렬성이 줄어들어 시스템 구조에 따라 적당한 갯수의 패킷 처리 큐를 설계하는 것이 중요하다. 또한 시스템 설계자는 이러한 병목현상을 줄이기 위해 시뮬레이터를 잘 활용하는 것이 중요하다.

- **실험 4: 메모리 컨트롤러와 메모리 모듈 및 성능의 상관관계**

본 연구팀은 메모리 컨트롤러와 메모리 모듈의 갯수에 따른 성능을 알아보기 위해 각각의 갯수를 달리하여 SPECint2006 벤치마크를 동작시켰다. 결과적으로, 그림 20과 같이 메모리 컨트롤러와 메모리 모듈의 갯수가 증가하더라도 시스템의 전반적인 영향은 미비했다. 즉, 메모리 컴포넌트들의 병렬성은 효과가 매우 적었다. 충분한 양의 메모리가 탑재된다면 메모리 컴포넌트가 시스템의 성능에 미치는 영향은 적다고 볼 수 있다.



(a) Performance overhead with the different numbers of *memory controllers* and the baseline configurations



(b) Performance overhead with the different numbers of *memory modules* and the baseline configurations

그림 21 메모리 컨트롤러 갯수 및 메모리 모듈과 성능의 상관관계

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- **실험 5: 인터커넥트의 소재에 따른 지연시간과 성능의 상관관계**

인터커넥트는 기존의 구리선 등을 이용한 전자적인 인터커넥트와 광섬유를 이용한 인터커넥트가 존재한다. 인터커넥트의 소재에 따라 종단간 지연시간이 매우 큰 차이를 보이며, 이것은 전체적인 시스템의 성능을 크게 좌우한다. 인터커넥트는 컴퓨트 노드, 메모리 풀 노드 등 수많은 노드들이 공유하는 자원이기 때문이다. 본 연구팀은 이 두 가지 소재의 지연시간을 시뮬레이터에 설정하고 실험했다. 그 결과, 그림 21과 같이 전자식 인터커넥트 스위치는 광학식 인터커넥트 보다 평균 약 5.78배 더 큰 지연시간이 발생했으며, 이것은 시스템 전반에 2.5배의 성능하락을 보여주었다. 시스템 설계자는 인터커넥트의 소재를 선택함에 있어서 이러한 성능하락을 예측하고 요구되는 성능에 부합하는 설계를 하는 것이 중요하다.

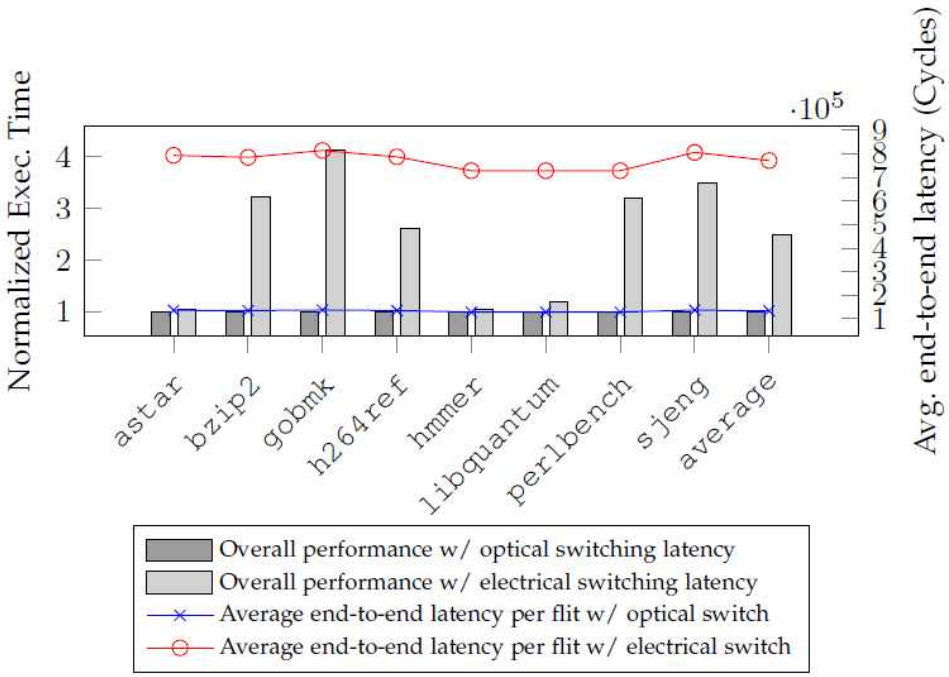


그림 22 인터커넥트의 소재에 따른 지연시간과 성능의 상관관계

2-3-6. 구현하지 못한 다른 보안요소 구성 방법 제안

- **외장 가속기의 내장 메모리 접근 제어**

본 과제에서 구현한 개별분리 시스템에서 GPU와 같은 가속기를 연결하기 위해서는 GPU의 공유 메모리에 대한 접근권한이 필요하다. 예를들어, 한 메모리 공간에 여러 CPU노드가 접근시 각 노드별로 권한을 할당해 자신의 메모리가 아닌 공간을 다른 노드가 접근하거나 쓰지 못하게 해 무결성을 보장해야만 한다. 이를 위해서는 GPU 메모리 공간의 접근 제어 시스템을 추후 구현할 필요가 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 연구개발과제의 수행 결과 및 목표 달성 정도

1) 연구수행 결과

(1) 정성적 연구개발성과

- **세계 최초 소프트웨어 기반 개별분리 시뮬레이터 설계 및 개발**

기존 연구에서는 하드웨어를 이용하여 개별분리 시스템을 구현하였지만 본 연구에서는 하드웨어 의존성이 없는 소프트웨어로 개별분리시스템을 설계 및 구현하였다. 또한 개별분리 아키텍처의 연산 속도 및 병목현상에 따른 한계 등을 분석하였다.

- **gem5 disaggregated 에뮬레이터 관련 내용**

컴퓨터 시스템 에뮬레이션 프로그램인 gem5를 이용하여 개별분리 아키텍처 에뮬레이터를 구현하였다. 또한 벤치마크 프로그램을 이용한 실험을 고안하는 것으로 해당 에뮬레이터가 의도한 바처럼 동작하며, 개별분리 아키텍처가 보이는 연산 속도 향상 및 병목현상에 따른 한계 등을 분석하였다. 추후 관련 기술을 삼성, 하이닉스 등이 소속되어 있는 개별분리 기술 연구 컨소시엄의 기업에 이전하는 것을 목표로 하고 있으며, 관련 구현과 관련된 논문을 출판하였다.

- **Unix to Linux(U2L) 서비스 이전을 위한 목적지 시스템 필요 성능 예측 방안**

성능 지표에 대한 case study 및 분석을 통해, 기존 Unix 시스템에서 동작하는 서비스를 Linux 환경으로 옮겨 실행하기 위한 필요 성능 예측에 대한 방안을 제시하였다. 해당 방안을 통해 최근 클라우드 컴퓨팅의 발전 및 기존 legacy 시스템의 한계로 인해 빈도가 증가한 Unix to Linux(U2L) 서비스 이전 상황에 기여하였고, SCI급 저널에 논문을 등재하였다.

- **에뮬레이터-실제 하드웨어 간 실행 불일치 이상 현상을 통한 에뮬레이터 탐지 기법 고안**

에뮬레이터와 실제 하드웨어 간의 실행 불일치로 인해 발생하는 이상 현상을 이용하여 에뮬레이터를 탐지하는 기법을 고안하였다. 또한, 이러한 이상 현상이 없는 에뮬레이터를 구현하기 위해서는 전반적인 구조의 수정과 심각한 성능 저하가 동반되어야 함을 증명하였다. 따라서, 우리는 효율적이며 정확한 에뮬레이션 기법의 필요성을 역설하였고, 그 기술의 독창성과 우수성을 인정받아 최상위 저널에 논문을 등재하였다.

- **국제공동연구 성과 (성균관대학교, CISPA 참여)**

독일의 헬름홀츠 사이버보안연구센터(CISPA)와의 국제 공동연구를 통해 본 과제를 수행하였으며 두 기관 간의 협력 증진과 연구 추진을 위해 CISPA에 학생을 파견 후 연구 진행하였다. 그리고 성균관대학교 / CISPA와의 주기적인 화상 미팅을 통한 연구 관련내용 토론 및 공유하였다. 연구기간 동안 얻은 배경지식과 상호 토론을 한 결과를 국제공동연구보고서로 작성하여 채택했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 정량적 연구개발성과

본 과제를 통해 SCI급 저널 5편(계획된 4편보다 1편 초과달성)을 저술 및 출판하였으며, 특허 5건 출원, 소프트웨어 등록 1건을 등록하였다.

< 정량적 연구개발성과표 >

(단위 : 건, 천원)

성과지표명 \ 연도			1단계 (YYYY~YYYY)	n단계 (YYYY~YYYY)	계	가중치 (%)
전담기관 등록·기탁 지표[1]	논문	실적(누적)			5	100%
	특허	실적(누적)			5	100%
	소프트웨어	실적(누적)			1	100%
연구개발과제 특성 반영 지표[2]		목표(단계별)				
		실적(누적)				
		목표(단계별)				
		실적(누적)				
계		실적(누적)			11	100%

* 1」전담기관 등록·기탁 지표: 논문[에스시아이 Expanded(SCIE), 비SCIE, 평균Impact Factor(IF)], 특허, 보고서원문, 연구시설·장비, 기술요약정보, 저작권(소프트웨어, 서적 등), 생명자원(생명정보, 생물자원), 표준화(국내, 국제), 화합물, 신품종 등을 말하며, 논문, 학술발표, 특허의 경우 목표 대비 실적은 기재하지 않아도 됩니다.
* 2」연구개발과제 특성 반영 지표: 기술실시(이전), 기술료, 사업화(투자실적, 제품화, 매출액, 수출액, 고용창출, 고용효과, 투자유치), 비용 절감, 기술(제품)인증, 시제품 제작 및 인증, 신기술지정, 무역수지개선, 경제적 파급효과, 산업지원(기술지도), 교육지도, 인력양성(전문 연구인력, 산업연구인력, 졸업자수, 취업, 연수프로그램 등), 법령 반영, 정책활용, 설계 기준 반영, 타 연구개발사업에의 활용, 기술무역, 홍보(전시), 국제화 협력, 포상 및 수상, 기타 연구개발 활용 중 선택하여 기재합니다
(연구개발과제 특성별로 고유한 성과지표를 추가할 수 있습니다).

평가 항목 (주요성능 Spec[1)])	단위	전체 항목에서 차지하는 비중[2)] (%)	세계최고 수준 보유국/ 보유기업 (미국/ HP)	연구개발 전 국내수준	개발 달성치			평가 방법
			성능수준	성능수준	1차 년도 (2020년)	2차 년도 (2021년)	3차 년도 (2022년)	
1. 개별분리 아키텍처 구현율 및 보안성 향상지표								
1-a 개별분리 시스템 인터커넥트 기능 구현율	%	12.5	미공개	최초	80	90	100	자체평가
1-b CPU pool 컨트롤러, 메모리 pool 컨트롤러, 가속기 pool 컨트롤러 기능 구현율	%	37.5	미공개	최초	50	75	100	자체평가
1-c 메모리 접근제어를 통한 보안성 향상	%	12.5	미공개	최초	-	90	100	자체평가
1-d 가속기 접근제어를 통한 보안성 향상	%	12.5	미공개	최초	-	90	100	자체평가
1-e 연산 오프로딩의 보안성 향상	%	12.5	미공개	최초	-	-	100	자체평가
1-f 컴포넌트 pool 간의 안전한 보안 프로토콜 설계	방어 가능 공격 유형 개수	12.5	미공개	최초	-	2	6 (3개 초과 달성)	자체평가
2. 보안 적용시 성능 저하 최소화 지표								

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-a 메모리 접근 지연 시간 증가량	%	50	미공개	최초	-	30 이내	20 이내	자체평가
2-b 인터커넥션 대역폭 감소량	%	50	미공개	최초	-	30 이내	20 이내	자체평가

(3) 세부 정량적 연구개발성과(해당되는 항목만 선택하여 작성하되, 증빙자료를 별도 첨부해야 합니다)

[과학적 성과]

□ 논문(국내외 전문 학술지) 게재

번호	논문명	학술지명	주저자명	호	국명	발행기관	SCIE 여부 (SCIE/비SCIE)	게재일	등록번호 (ISSN)	기여율
1	Quantitative Server Sizing Model for Performance Satisfaction in Secure U2L Migration	IEEE Access	조훈	9	국외	IEEE	SCIE	2021-10-11	2169-3536	100%
2	EmuID: Detecting presence of emulation through microarchitectural characteristic on ARM	Computer & Security	최예슬	113	국외	Elsevier	SCIE	2021-12-02	0167-4048	22.5%
3	DID We Miss Anything?:Towards Privacy-Preserving Decentralized IDArchitecture	IEEE TDSC	허시원	20	국외	IEEE	SCIE	2023-01-13	1545-5971	10%
4	Harnessing the x86 Intermediate Rings for Intra-Process Isolation	IEEE TDSC	이호준	20	국외	IEEE	SCIE	2023-01-13	1545-5971	10%
5	Towards Scalable and Configurable Simulation for Disaggregated Architecture	SIMPAT	김대경	125	국외	Elsevier	SCIE	2023-03-02	1569-190X	30%

□ 국내 및 국제 학술회의 발표

번호	회의 명칭	발표자	발표 일시	장소	국명

□ 기술 요약 정보

연도	기술명	요약 내용	기술 완성도	등록 번호	활용 여부	미활용사유	연구개발기관 외 활용여부	허용방식

□ 보고서 원문

연도	보고서 구분	발간일	등록 번호

□ 생명자원(생물자원, 생명정보)/화합물

번호	생명자원(생물자원, 생명정보)/화합물 명	등록/기탁 번호	등록/기탁 기관	발생 연도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[기술적 성과]

□ 지식재산권(특허, 실용신안, 의장, 디자인, 상표, 규격, 신품종, 프로그램)

번호	지식재산권 등 명칭 (건별 각각 기재)	국명	출원				등록			기여율	활용 여부
			출원인	출원일	출원 번호	등록 번호	등록인	등록일	등록 번호		
1	특허	안전한 개별분리 시스템을 위한 인터커넥트 설계 및 구현방법	강병훈 외 7인	2021-01-05	10-2021-0001091					100%	N
2	특허(해외)	Design of interconnect architecture for secure disaggregated system	강병훈 외 7인	2021-08-02	PCT/KR2021/010071					100%	N
3	특허	소프트웨어 기반의 개별분리 아키텍처 시스템 시뮬레이터	강병훈 외 8인	2021-12-29	10-2021-0191586					100%	N
4	특허(해외)	TOWARD SCALABLE AND CONFIGUREABLE SIMULATION FOR DISAGGREGATED ARCHITECTURE, AND METHOD OF THE SAME	강병훈 외 8인	2022-12-06	PCT/KR2022/019733					100%	N
5	특허	개별분리 아키텍처의 GPU 메모리 초기화를 이용한 보호 기법	강병훈 외 4인	2023-01-02	10-2023-0000303					100%	N

o 지식재산권 활용 유형

※ 활용의 경우 현재 활용 유형에 √ 표시, 미활용의 경우 향후 활용 예정 유형에 √ 표시합니다(최대 3개 중복선택 가능).

번호	제품화	방어	전용실시	통상실시	무상실시	매매/양도	상호실시	담보대출	투자	기타

□ 저작권(소프트웨어, 서적 등)

번호	저작권명	창작일	저작자명	등록일	등록 번호	저작권자명	기여율
1	개별분리 시스템 시뮬레이터	2023.03.12	한국과학기술원	2023.05.08	C-2023-020494	강병훈 외 4인	100%

□ 신기술 지정

번호	명칭	출원일	고시일	보호 기간	지정 번호

□ 기술 및 제품 인증

번호	인증 분야	인증 기관	인증 내용		인증 획득일	국가명
			인증명	인증 번호		

□ 표준화

o 국제표준 및 사실표준

번호	표준화 기구명	표준화 분과명[1]	표준 종류[2]	표준 단계[3]		제안(기고)한 표준[4]		반영(채택)된 표준[5]		제안자	승인(채택) 일자
				구분	단계명	표준번호	표준명	표준번호	표준명		
1	*ITU-T*	*SG5/WP1/Q2*	*제정*	*제안*	*NWI*	*C.xxxx*		*TD.xxxx*			

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 국제표준 진행 단계 중 해당하는 사항을 기재합니다.
(예) ITU-T의 경우, 신규 작업항목 제안 채택(NWI), 표준문서개발(Draft Recommendation), 국제표준승인(Recommendation)
* 4」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 제안(Contribution)한 표준문서 정보를 기재합니다.
* 5」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 반영(TD/Recommendation)된 표준문서 정보를 기재합니다.

ㅇ 국가표준 및 단체표준

번호	표준화 기구명[1]	표준종류[2]	표준명	표준번호	제안자	승인(채택) 일자
1	TTA	제정				

* 1」 제정 또는 개정한 표준화기구를 국립전파연구원(RRA), 국가기술표준원(KATS), 한국정보통신기술협회(TTA) 등에서 해당하는 사항으로 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.

ㅇ 국제표준 정책기고

번호	표준화 기구명	표준화 정책위원회명[1]	문서종류[2]	문서반영 단계구분[3]	결의안/기고명	문서번호	제안자	승인(채택) 일자
1	ITU-T	ITU-T WTSA	제정	ITU-T WTSA Resolution				

* 1」 정책위원회에 해당하는 ITU PP, ITU-T WTSA, ITU-T TSAG, ITU-R RA/WRC, ITU-R RAG/WP5D, ITU-D WTDC, ITU-D TDAG, APT, MC/APG/AWG/WTSA/ASTAP, JTC 1 총회 및 산하 자문그룹, ISO 총회, ISO TMB, IEC 총회, IEC SMB, 3GPP PCC/OP/TSGs, W3C 자문위원회 등에서 해당하는 사항으로 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 결의안, 국가선도기술 제안, 정책위원회 기고 반영의 진행 단계 중 해당하는 사항을 기재합니다.

ㅇ 오픈소스 연계 표준개발

번호	구분	표준문서 및 오픈소스 정보								과제검증 시험 결과서 번호[4]
	표준	표준화 기구명	표준화 분과명[1]	표준 종류[2]	표준단계		반영(채택)된 표준[3]		승인(채택) 일자	
					구분	단계명	표준번호	표준명		
1		3GPP	SG5/WP1/Q2	제정	제안	NWI				
	오픈 소스	프로젝트명			반영 소스명				승인(Commit) 일자	
		O-RAN								

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 제정 또는 개정 중 해당하는 사항을 기재합니다.
* 3」 제안(기고)한 표준문서가 제안/개발/승인 문서에 반영(채택)되었을 경우, 반영(TD/Recommendation)된 표준문서 정보를 기재합니다.
* 4」 표준문서와 오픈소스의 정합에 대하여 TTA에서 발급한 시험 결과서 번호를 기재합니다.

ㅇ 표준특허

번호	표준특허 구분[1]	표준							특허				
		표준화 기구	표준 단계[2]		표준명	표준번호	승인/개발 일자	IPR 선언 일자	특허 단계 구분[3]	특허명	특허번호	국명	출원/등록 일자
			구분	단계명									
1	국제표준 특허승인	3GPP	승인	TS					등록				

* 1」 국내/국제 표준특허 승인 또는 후보 중 해당하는 사항을 기재합니다.
* 2」 국내/국제표준 진행 단계 중 해당하는 사항을 기재합니다.
* 3」 특허 출원, 등록 중 해당하는 사항을 기재합니다.

ㅇ 표준 국제협력

번호	의장단 수임						위원회/그룹 신설			국제회의 국내유치		
	표준화 기구명	표준화 분과명[1]	의장단 직위[2]	신규/계속 여부	수임자	수임기간	표준화 기구명	표준화 분과명[1]	신설일자	표준화 기구명	표준화 회의명	회의일자
1	ITU-T	SG5/WP1/Q2	라포처	신규			ITU-T	SG5/WP1/Q2		ITU-T	FG abc	

* 1」 해당 표준을 진행한 표준화기구 분과명 및 분과번호를 하위 조직까지 기재합니다.
* 2」 의장단 수임 직위를 의장, 부의장, 라포처, 컨비너, 에디터, 프로젝트리더 중에서 해당하는 사항으로 기재합니다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[경제적 성과]

□ 시제품 제작

번호	시제품명	출시/제작일	제작 업체명	설치 장소	이용 분야	사업화 소요 기간	인증기관 (해당 시)	인증일 (해당 시)

□ 기술 실시(이전)

번호	기술 이전 유형	기술 실시 계약명	기술 실시 대상 기관	기술 실시 발생일	기술료 (해당 연도 발생액)	누적 징수 현황

* 내부 자금, 신용 대출, 담보 대출, 투자 유치, 기타 등

□ 사업화 투자실적

번호	추가 연구개발 투자	설비 투자	기타 투자	합계	투자 자금 성격*

□ 사업화 현황

번호	사업화 방식[1」]	사업화 형태[2」]	지역[3」]	사업화명	내용	업체명	매출액		매출 발생 연도	기술 수명
							국내 (천원)	국외 (달러)		

* 1」 기술이전 또는 자기실시
* 2」 신제품 개발, 기존 제품 개선, 신공정 개발, 기존 공정 개선 등
* 3」 국내 또는 국외

□ 매출 실적(누적)

사업화명	발생 연도	매출액		합계	산정 방법
		국내(천원)	국외(달러)		
합계					

□ 사업화 계획 및 무역 수지 개선 효과

성과				
사업화 계획	사업화 소요기간(년)			
	소요예산(천원)			
	예상 매출규모(천원)	현재까지	3년 후	5년 후
	시장 점유율	단위(%)	현재까지	3년 후	5년 후
		국내			
		국외			
	향후 관련기술, 제품을 응용한 타 모델, 제품 개발계획			
무역 수지 개선 효과(천원)	수입대체(내수)	현재	3년 후	5년 후
	수출			

□ 고용 창출

순번	사업화명	사업화 업체	고용창출 인원(명)		합계
			yyyy년	yyyy년	
합계					

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 고용 효과

구분			고용 효과(명)
고용 효과	개발 전	연구인력	
		생산인력	
	개발 후	연구인력	
		생산인력	

□ 비용 절감(누적)

순번	사업화명	발생연도	산정 방법	비용 절감액(천원)
합계				

□ 경제적 파급 효과

(단위: 천원/년)

구분	사업화명	수입 대체	수출 증대	매출 증대	생산성 향상	고용 창출 (인력 양성 수)	기타
해당 연도							
기대 목표							

□ 산업 지원(기술지도)

순번	내용	기간	참석 대상	장소	인원

□ 기술 무역

(단위: 천원)

번호	계약 연월	계약 기술명	계약 업체명	계약업체 국가	기 징수액	총 계약액	해당 연도 징수액	향후 예정액	수출/ 수입

[사회적 성과]

□ 법령 반영

번호	구분 (법률/시행령)	활용 구분 (제정/개정)	명 칭	해당 조항	시행일	관리 부처	제정/개정 내용

□ 정책활용 내용

번호	구분 (제안/채택)	정책명	관련 기관 (담당 부서)	활용 연도	채택 내용

□ 설계 기준/설명서(시방서)/지침/안내서에 반영

번호	구 분 (설계 기준/설명서/지침/안내서)	활용 구분 (신규/개선)	설계 기준/설명서/ 지침/안내서 명칭	반영일	반영 내용

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 전문 연구 인력 양성

번호	분류	기준 연도	현황										
			학위별				성별		지역별				
			박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타

□ 산업 기술 인력 양성

번호	프로그램명	프로그램 내용	교육 기관	교육 개최 횟수	총 교육 시간	총 교육 인원

□ 다른 국가연구개발사업에의 활용

번호	중앙행정기관명	사업명	연구개발과제명	연구책임자	연구개발비

□ 국제화 협력성과

번호	구분 (유치/파견)	기간	국가	학위	전공	내용

□ 홍보 실적

번호	홍보 유형	매체명	제목	홍보일

□ 포상 및 수상 실적

번호	종류	포상명	포상 내용	포상 대상	포상일	포상 기관

[인프라 성과]

□ 연구시설・장비

구축기관	연구시설/ 연구장비명	규격 (모델명)	개발여부 (○/×)	연구시설・장비 종합정보시스템* 등록여부	연구시설・장비 종합정보시스템* 등록번호	구축일자 (YY.MM.DD)	구축비용 (천원)	비고 (설치 장소)

* 「과학기술기본법 시행령」 제42조제4항제2호에 따른 연구시설・장비 종합정보시스템을 의미합니다.

[그 밖의 성과](해당 시 작성합니다)

- **국제공동연구보고서 작성 및 채택**

과제기간 중 CISPA/성균관대와의 협력내용을 기반으로 하여 연구 아이디어 및 추후 보완 사항등을 정리하여 보고서로 작성함.

- 제목: Resource Disaggregation: Challenges and Future Works
- 내용: 본 과제에서 다루고 있는 개별분리 아키텍처의 개요와 배경 지식, 관련 연구를 소개하고 최신 연구 및 앞으로의 개선할 부분 등에 대해 논의함.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(4) 계획하지 않은 성과 및 관련 분야 기여사항(해당 시 작성합니다)

해당없음

2) 목표 달성 수준

추 진 목 표	달 성 내 용	달성도(%)
○ 개별분리 시스템 설계, 구현 및 개발 ○ 개별분리 시스템의 벤치마크	○ 개발 완료 및 소스코드 공개 ○ SCIE급 저널 5편 출판 (계획된 4편보다 1편 초과 달성)	○ 100% ○ 125%
○ 특허 출원	○ 국내 3건, 해외 2건 출원, 총 5건 출원	○ 100%
○ 소프트웨어 등록	○ 소프트웨어 1건 등록	○ 50%

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 목표 미달 또는 미흡한 사항에 대한 원인분석

1) 목표 미달 또는 미흡한 연구결과에 대한 원인(사유) 자체분석 내용

- **최종 결과물 성능 평가를 위한 벤치마크 항목의 부족**

시뮬레이터와 개발된 접근제어 방법의 평가를 위해서 사용된 프로그램으로는 SPEC2006 벤치마크 세트 중 일부 벤치마크와 STREAM 메모리 대역폭 벤치마크가 사용되었다. 그러나 이 프로그램들은 실생활에서 접할 수 있는 대규모 워크로드에 대해 전체 시스템의 성능을 측정하기 위한 것에 초점을 맞춘 것이 아니다. 그보다는, 시스템 중 특정 하드웨어 요소의 성능을 분석하기 위한 목적으로 개발된 것이다 (마이크로벤치마크). 따라서 실제 서버 환경에서 최종 결과물의 성능을 평가하기에는 일부 부적합한 면이 있다.

이러한 한계가 있었던 근본적인 원인에는 시스템 설계를 위해 사용되었던 시뮬레이터가 하드웨어 요소의 세부적인 부분에서 걸리는 오버헤드까지 모두 측정하는 것을 목표로 개발되었기 때문이다. 이러한 방법론은 하드웨어 마이크로아키텍처의 어느 부분에서 성능 저하가 주로 일어나는지 (예를 들어, 메모리 접근 시 인터커넥트 로직 내부 큐 딜레이등)를 파악하기에는 적합하지만 대신 시뮬레이션 시간이 급증하는 단점이 있다. 따라서, 성능 평가를 위한 시뮬레이션 또한 수행 시간이 긴 실제 워크로드 대신 비교적 시간이 짧게 소요되는 마이크로벤치마크 위주로 실시되었다.

- **시뮬레이터와 실제 개별분리 시스템간의 직접적인 성능 비교 불가**

시뮬레이터가 정확한 결과를 산출하는지를 검증하기 위해서는 실제 시스템과의 직접적인 비교가 수행되어야 한다. 그러나 이번 과제 중 실제 개별분리 시스템과 개발된 시뮬레이터 간의 직접적인 비교가 수행될 수 없었다.

현재 개별분리 시스템은 확정된 표준안이 없어 구현에 따른 시스템 구조가 크게 차이가 난다. 또한, 과제 시작 시점에 존재했던 Gen-Z Consortium 이 과제 중간 시점에 CXL 표준으로 흡수 합병되는 등 변화도 상당히 빠르게 이루어지고 있다. 이러한 환경에서, 개별분리 시뮬레이터는 어느 하나의 시스템에 특화된 모델링을 제공한다기보다는 전반적인 공통점을 시뮬레이션하기 위한 틀을 제공하는 데 초점을 맞추어 개발이 진행되었다. 이에 따라 성능 평가 또한 실제 개별분리 시스템과의 직접 비교 분석보다는 전체적인 시스템 구조의 변화에 따른 성능 변화 측정에 초점을 맞추어 진행되어야 했다.

- **실제 개별분리 시스템에 대한 공격 및 방어 시연 불가**

위와 마찬가지로, 실제 개별분리 시스템의 표준안이 없어 현재 존재하는 개별분리 시스템은 보안 측면에서의 고려점이 각각 상이한 상황이다. RDMA와 같은 일부 시스템은 보안상 취약점과 방어 방법이 학회 논문 등으로 발표되었지만, 아직 실제로 구현되지 않았거나 비공개 구현체만이 존재하는 다른 시스템의 경우 보안 측면에서의 고려점이 잘 확인되지 않고 있다.

이러한 상황에서, 시뮬레이터로 어느 특정 공격과 방어 기법을 시연한다는 것은 특정 시스템에 대한 시뮬레이션을 수행한다는 것과 동일한 의미를 가지게 된다. 그러나 시뮬레이터 개발의 목표가 일반화 된 개별분리 시뮬레이션을 수행하는 것으로 정해졌기 때문에, 구체적인 공격 및 방어 시연은 최종 목표에서 제외되어야 했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 미달 또는 미흡한 결과물에 대한 보완 활동

- **최종 결과물 성능 평가를 위한 벤치마크 항목의 부족**

시뮬레이터로 실제 워크로드에 대한 시스템의 성능 평가를 수행하는 것은 소요되는 시간 문제 등으로 어려운 작업이다. 하지만, PARSEC 벤치마크 등 일부 벤치마크의 경우 실제 워크로드와 유사한 특정을 가지면서 시뮬레이터에서 실행 가능할 정도의 작은 입력을 가진 것으로 알려졌다. 따라서, 이러한 벤치마크를 실행해 성능 평가에 추가로 반영한다면 좀 더 현실적인 성능 평가가 가능할 것이다. 따라서, 해당 미흡점에 대해서는 추가 벤치마크 테스트를 통해 보완할 계획이다.

- **시뮬레이터와 실제 개별분리 시스템간의 직접적인 성능 비교 불가**

현재 시뮬레이터는 일반적인 개별분리 시스템을 시뮬레이션하기 위한 구조와 모델링 파라미터를 가지고 있다. 실제 개별분리 시스템과 직접적인 비교를 실시하기 위해서는 이를 일부 튜닝해서 실제 시스템과 일치하도록 맞춰야 하는 작업이 필요하다. 일부 사양 명세(specification) 이 공개된 시스템의 경우 이를 실시할 수 있으나, 비공개 시스템의 경우 작업이 어려운 측면이 있다. 따라서, 해당 미흡점에 대해서는 현재 사양이 공개된 시스템을 위주로 모델을 개선하는 방향로 보완할 계획이다.

- **실제 개별분리 시스템에 대한 공격 및 방어 시연 불가**

위와 마찬가지로, 시뮬레이터를 현재 사양이 공개된 시스템에 맞춰 일부 수정하고 시뮬레이터 상에서 공격 및 방어 기법을 실험하는 방향으로 해당 미흡점을 보완할 계획이다.

3) 연구개발 과정의 성실성

- **주 1회 연구 미팅 및 논문 세미나**

연구에 참여하는 연구원 간 원할한 지식 공유 및 프로젝트 방향성 설정을 위해 최소 주 1회 미팅 및 논문 세미나를 실시했다.

- **개발 과정 관리를 위한 도구의 사용**

최종 결과물인 개별분리 시스템 시뮬레이터의 형상 관리를 위해 Github와 같은 협업 도구를 적극 사용했으며, 코드 리뷰 시스템을 통해 각 연구원이 개발중인 부분에 대해서 의견 공유가 원활하게 이루어지도록 했다.

- **코로나19 대비 온라인 프로젝트 체계 구축**

과제가 진행된 2020년 - 2023년 동안 대부분의 기간 중 코로나 19의 유행으로 인해 오프라인 업무 및 연구가 크게 제한되는 일이 발생했다. 이에 대응해, 연구개발 과정을 100% 온라인으로 진행할 수 있도록 자체 메신저 및 미팅 프로세스 구축, 개발용 서버 공유와 같은 조치를 실시했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- **국제공동연구협력기관, 공동연구개발기관과의 협력 연구**

본 과제에 연구개발기관 외 기관으로 참여한 독일의 CISPA, 공동연구개발기관으로 참여한 성균관대학교 등과 활발한 협력 연구를 수행했다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

5. 연구개발성과의 관련 분야에 대한 기여 정도

- **일반화된 개별분리 시스템의 모델 제시**

현재 개별분리 시스템의 표준안이 정해져 있지 않아 각각의 시스템 별로 지원되는 기능, 사양 명세, 예상되는 사용례 등이 상이한 상황이다. 본 연구에서는 각 시스템 별 차이점과 공통점을 조사하여 일반화된 개별분리 시스템의 모델을 제시했다. 이는 후속 연구에서 특정 기능에 특화되도록 수정된 개별분리 시스템을 만들 때 유용하게 쓰일 수 있는 기본적인 토대로서 사용될 수 있다. 따라서, 후속 연구에서는 중복되는 설계 작업을 다시 반복할 필요 없이 빠른 프로토타입 제작 등이 가능해 질 것으로 예상한다.

- **시뮬레이터의 오픈소스 공개**

본 연구에서 개발된 시뮬레이터는 오픈소스로 공개되어 있다. 따라서 단순히 파라미터를 변경하는 수준의 시스템 변경 뿐만 아니라 새로운 설계의 개별분리 시스템 모델링 등 큰 규모의 수정이 필요한 작업에서 또한 후속 연구자들이 공개된 시뮬레이터를 유용하게 사용할 수 있을 것으로 예상한다.

- **개별분리 시스템에서의 보안 위협 모델링**

현재 개별분리 시스템은 보안 위협에 대한 고려가 아직 많이 논의되지 않은 상태이다. 본 연구과제에서는 개별분리 시스템에서 발생 가능한 보안 위협을 종류별로 정리했으며, 각 위협별로 가능한 방어 방법을 제시했다. 또한, 각 방어 방법이 실제로 시스템의 성능 측면에서 어떠한 영향을 미치는지를 시뮬레이션했다. 해당 결과는 실제로 개별분리 시스템을 설계할 때 보안 측면의 사양을 정하는 데 도움이 될 것이다.

- **인터커넥트 로직과 중앙 관리 유닛의 설계**

본 연구에서는 각 컴퓨트 노트가 인터커넥트에 연결될 때 반드시 거쳐야 하는 관문인 인터커넥트 로직과, 인터커넥트 로직을 관리하는 중앙 관리 유닛을 설계했다. 이 두 요소는 기존 개별분리 시스템에서 명확히 정의되지 않았던 신뢰할 수 있는 컴퓨팅의 기반 (rrust anchor) 역할을 한다. 따라서 개별분리 시스템에서의 보안 위협 정의와 방어 기법의 설계 시에 해당 요소 활용해 문제를 해결할 수 있는 기법을 제시할 수 있으며, 본 연구에서는 시뮬레이션을 통해 기법 일부의 효용성를 입증했다.

- **가속기를 포함한 개별분리 시스템의 보안 모델링**

지금까지 제시된 개별분리 시스템 대부분은 컴퓨트 노드와 메모리 풀 관리에 초점이 맞춰져 있었으며, 가속기와 주변 장치에 대한 고려는 부족한 측면이 있었다. 그러나 AI와 기계 학습등 고속/대규모 병렬 연산이 필요한 워크로드가 증가하면서 가속기는 고성능 시스템의 필수 요소로 자리잡았다. 본 연구에서는 개별분리 시스템에서 가속기가 연결되는 방식과 보안 측면에서의 고려에 대해 정의했으며, 발생 가능한 보안 위협에 대해 가능한 방어 기법 또한 제시했다. 실제 개별분리 시스템의 하드웨어 설계 또는 후속 개별분리 시스템 연구에서 가속기가 안전하게 사용되는 방법에 대해 본 연구의 결과를 활용 가능할 것이다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 연구개발성과의 관리 및 활용 계획

- **개발된 시뮬레이터의 활용 및 개선**

현재 개발된 시뮬레이터는 아직 개별분리 시스템의 표준안이 없는 관계로 일부 설계 측면에서 일반화된 개별분리 시스템을 모델링 한 것이다. 향후 기술의 발전에 따라 개별분리 시스템의 표준안이 확정되거나, 사실상의 표준 (de facto standard) 이 등장할 수 있다. 이에 맞춰 현재 시뮬레이터를 확장 및 보완할 계획이다.

- **개별분리 시스템에 연결 가능한 기타 하드웨어의 모델링 확장**

현재 시스템 모델은 크게 컴퓨트 노드, 인터커넥트, 원격 메모리 풀, 가속기 노드로 이루어져 있다. 그러나 개별분리 시스템에 대한 연구개발이 진척됨에 따라 영구 저장소 (persistent stroage), FPGA, 이기종 (heterogenous) 컴퓨트 노드 등 다른 종류의 하드웨어 또한 시스템에 연결될 수 있을 것이다. 현재 시뮬레이터는 새로운 하드웨어를 모델링할 수 있도록 유연하게 설계되었으며, 향후 연구개발 시에 이러한 방향의 확장을 고려할 계획이다.

< 연구개발성과 활용계획표 >

구분(정량 및 정성적 성과 항목)		연구개발 종료 후 5년 이내
국외논문	SCIE	1
	비SCIE	
	계	1
국내논문	SCIE	
	비SCIE	
	계	
특허출원	국내	1
	국외	
	계	1
특허등록	국내	1
	국외	1
	계	2
인력양성	학사	
	석사	
	박사	
	계	
사업화	상품출시	
	기술이전	
	공정개발	
제품개발	시제품개발	
국제 표준화	기고서 제출	
	기고서 채택	
	표준안 채택	
국내 표준화	기고서 제출	
	기고서 채택	
	표준안 채택	
성과홍보		
포상 및 수상실적		
정성적 성과 주요 내용		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1) 향후 5년간 성과활용·확산 활용방안 및 계획(활동계획)

- **시뮬레이터 활용 및 확산**

  본 과제를 통해 개발된 개별분리 아키텍처는 Github을 통해 오픈소스화하여 배포 중이며, 누구나 자유롭게 사용할 수 있다. 이는 관련 후속 연구에 적극적으로 사용되길 기대되며, 연구 뿐만 아니라 교육에도 사용될 것으로 예상된다. 이에 맞춰서 향후 확정되는 개별분리 아키텍처의 표준안과 연결 가능한 하드웨어에 맞춰서 확장 및 보완할 예정이다.

- **후속 연구**

  개별분리 아키텍처가 최근 많은 관심을 받는 것에 비해 보안 측면의 고려가 현재까지 부족한 상황이며, 새로운 보안 대책이 절실한 형편이다. 또한, 이러한 새로운 아키텍처의 특성이 오히려 기존 클라우드에서 풀 수 없었던 보안 문제들을 해결하는데 도움이 될 것으로 보인다. 이에 따라서 본 연구의 주요 성과인 개별분리 아키텍쳐 시뮬레이터를 활용한 보안 문제 해결에 초점을 맞춰 후속 연구를 진행할 예정이다.

  한 GPU 메모리에 여러 CPU노드가 접근시 각 노드별로 권한을 할당해 자신의 메모리가 아닌 공간을 다른 노드가 접근하거나 쓰지 못하게 해 무결성을 보장해야만 한다. 이를 위해서 GPU 메모리 공간의 접근 제어 시스템에 관한 후속 연구를 진행할 예정이다.

  이외에도 개별분리 아키텍처 시뮬레이터를 활용하여 많은 응용 연구가 가능할 것으로 기대되며, 다음의 후속 연구도 계획하고 있다.

  - 개별분리 아키텍처에서 동작하는 OS의 보안성 향상 연구
  - 개별분리 아키텍처의 side channel 연구
  - 개별분리 아키텍처의 secure layer 연구 등

<연도별 주요 활동계획(요약표)>

구분	종료+1 (2023년)	종료+2 (2024년)	종료+3 (2025년)	종료+4 (2026년)	종료+5 (2027년)
후속연구	연구 진행 및 논문 작성	연구 진행 및 논문 작성	연구 진행 및 논문 작성	연구 진행 및 논문 작성	연구 진행 및 논문 작성

2) 주요 성과활용 (기대)성과목표

(1) 정성적 목표

주요 성과 활용에 대한 기대 성과로는 후속 연구에 따른 논문 출간이 예상된다. 또한 개발된 시뮬레이터를 이용하여 국내외 연구 기관들과의 협업 연구도 기대되며, 이 결과로 매년 1편의 논문 출간을 목표로 하고 있다.

(2) 정량적 목표

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<연도별 성과활용 (기대)성과목표(요약표)>

구분	종료+1 (2023년)	종료+2 (2024년)	종료+3 (2025년)	종료+4 (2026년)	종료+5 (2027년)
해외 SCI급 논문 수	1	1	1	1	1

3) 성과활용·관리 추진체계

주관연구기관인 한국과학기술원에서 주요 성과인 시뮬레이터를 활용 및 관리할 예정이다.

4) 기타

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획

위의 6번 항목에서 언급되었듯이 개별분리 아키텍처 시뮬레이터를 활용한 보안 문제 해결에 초점을 맞춰서 후속 연구를 진행할 예정이다. 개별분리 아키텍처를 활용하여 개별분리 아키텍처의 OS 보안성 향상, side channel attack 그리고 secure layer 관련 연구도 진행할 예정이다.

본 과제에서 개발된 시뮬레이터는 향후 다양한 연구 및 교육에서 활용될 것으로 기대되기 때문에 주기적으로 확장 및 보완할 예정이다. 현재는 개별분리 아키텍처의 표준안이 정해져 있지 않지만 향후 정해진다면 이에 맞춰서 보완할 예정이다. 또한 추가로 개별분리 아키텍처에 연결할 수 있는 하드웨어에 대해서도 향후 추가로 보완 및 개발할 예정이다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

8. 연구개발비 사용실적

1) 연구개발비 사용명세서

(단위 : 원, %)

항목 \ 구분		연구개발비 현황					사용 금액 ⑥	사용 잔액 (⑤-⑥)	사용률 (⑥/⑤ ×100)	불용금액 (해당 시)
		전단계 이월액 (해당 시) ①	당초 연구개발비 ②	변경 연구개발비 ③	발생이자 중 연구개발비 산입금 ④	소계 ⑤ (①+② +③+④)				
가. 직접비	현금	3,665,842	198,162,000	0	0	201,827,842	200,603,141	1,224,701	99.4	
	현물	0	0	0	0	0	0	0	0	
	소계	3,665,842	198,162,000	0	0	201,827,842	200,306,141	1,224,701	99.4	
1) 인건비	현금	0	8,429,000	0	0	0	0	0	0	
	현물	0	0	0	0	0	0	0	0	
2) 학생인건비	현금	0	141,356,000	0	0	141,356,000	141,356,000	0	100	
3) 연구시설·장비비	현금	911,000	675,000	0	0	1,586,000	1,586,000	0	100	
	현물	0	0	0	0	0	0	0	0	
4) 연구재료비	현금	0	0	0	0	0	0	0	0	
	현물	0	0	0	0	0	0	0	0	
5) 위탁연구개발비[1]	현금	0	0	0	0	0	0	0	0	
6) 국제공동연구개발비	현금	0	0	0	0	0	0	0	0	
7) 연구개발부담비	현금	0	0	0	0	0	0	0	0	
8) 연구활동비	현금	2,754,842	15,003,000	0	0	17,757,842	16,536,141	1,221,701	93.1	
	현물	0	0	0	0	0	0	0	0	
9) 연구수당	현금	0	32,699,000	0	0	32,699,000	32,699,000	0	100	
나. 간접비	현금	0	51,838,000	0	0	51,838,000	51,838,000	0	100	
	소계	0	51,838,000	0	0	51,838,000	51,838,000	0	100	
합계	현금	3,665,842	250,000,000	0	0	253,665,842	252,441,141	1,221,701	99.4	
	현물	0	0	0	0	0	0	0	0	
	합계	3,665,842	250,000,000	0	0	253,665,842	252,441,141	1,221,701	99.4	

* 1」 위탁연구개발비 현황 및 사용금액은 주관연구개발기관만 기재합니다.

2) 발생이자 사용명세서

(단위 : 원)

구분	발생금액 (A)	사용용도	사용금액 (B)	사용잔액 (A-B)
연구개발기간 중 발생이자				
연구개발기간 후 발생이자				
합계				

3) 반납액 명세서

(단위 : 원, %)

연구개발비 반납액				발생이자 반납액		총 반납액 (C+E)
구분	현금	현물	소계			
사용잔액	1,221,701	0	1,221,701	연구개발기간 중		
불인정 사용금액	0	0	0	연구개발기간 후		
소계(A)	1,221,701	0	1,221,701	소계(D)		
정부지분율(B)	100	0	100	정부지분율(B)		
정부지분액(C=A×B)	1,221,701	0	1,221,701	정부지분액(E=D×B)		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주 의

1. 이 보고서는 과학기술정보통신부에서 시행한 정보통신 · 방송연구개발사업의 ○○연구개발과제 최종보고서이다.

2. 이 연구개발내용을 대외적으로 발표할 때에는 반드시 과학기술정보통신부(정보통신기획평가원)에서 시행한 정보통신 · 방송연구개발사업의 결과임을 밝혀야 한다.

3. 국가과학기술 기밀 유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 안 된다.